



In this chapter, we will tackle the problem of managing sequence control, an important part in defining the execution of program instructions in a generic abstract machine's interpreter.

In low-level languages, sequence control is implemented in an elementary way, by updating the value of the Program Counter register. In high-level languages, special language-specific constructs permit the structuring of control and the implementation of mechanisms more abstract than those available on the physical machine. Take, for example, the evaluation of an arithmetic expression: even a simple operation of this kind requires the use of control mechanisms specifying the order in which operands are evaluated, operator precedence, and so on.

In this chapter, we will consider the constructs for the explicit or implicit specification of sequence control. We will first consider expressions, spending some time on the syntactic aspects of the usual notation for representing expressions, as well as the semantic aspects of their evaluation. We will then move on to commands and, after discussing the concepts of variable and assignment, we will see the main commands for sequence control present in modern languages, showing the difference between structured and unstructured control and briefly illustrating the principles of structured programming. We will finally examine some aspects that are significant to recursion and clarify a terminological distinction between imperative and declarative languages.

Constructs for abstraction on control will be the subject of the next chapter.

6.1 Expressions

Expressions, together with commands and declarations, are one of the basic components of most programming language. We can say that expressions are the essential component of every language because, although there exist declarative languages in which commands are absent, expressions, numeric or symbolic, are present in every language.

First, let us try to clarify what sorts of object we are talking about.

Definition 6.1 (*Expression*) An expression is a syntactic entity whose evaluation either produces a value or fails to terminate, in which case the expression is undefined.

The essential characteristic of an expression, which differentiates it from a command, is therefore that its evaluation produces a value. Examples of numerical expressions are familiar: $4+3*2$, for example, is an expression whose evaluation may be seen as obvious. However, even in such a simple case, to obtain the correct result, we should make an implicit assumption (derived from mathematical convention) about operator precedence. This assumption— $*$ has precedence over $+$ (and, therefore, the result is 10 and not 14)—specifies a control aspect for evaluation of expressions. We will see below other more subtle aspects that contribute to modify the result of the evaluation of an expression.

Expressions can be non-numeric, for example in LISP, we can write `(cons a b)` to denote an expression which, when evaluated, returns the so-called pair formed by `a` and `b`.

6.1.1 Expression Syntax

In general, an expression is composed of a single entity (constant, variable, etc.) or of an operator (such as $+$, `cons`, etc.), applied to a number of arguments (or operands) which are also expressions. We saw in Chap. 2 how expression syntax can be precisely described by a context-free grammar and that an expression can be represented by a derivation tree, which also expresses semantic information relating to the evaluation of that expression. Tree structures are also often used to represent an expression internally inside the computer. However, if we want to use expressions in a conventional way in the text of a program, linear notations allow us to write an expression as a sequence of symbols. Various notations differ from each other by how they represent the application of an operator to its operands. We can distinguish three main types of notation.

Infix Notation

In infix notation, a binary operation symbol is placed between the expressions representing its two operands. For example, we write $x+y$ to denote the addition of x and y , or $(x+y)*z$ to denote the multiplication by z of the result of the addition of x and y . To avoid ambiguity in the application of operator to operands, brackets

Lisp and S-expressions

The programming language LISP (an acronym of LIST Processor), developed at the beginning of the 1960s by John McCarthy and by a group of researchers at MIT (Massachusetts Institute of Technology), is a language designed for symbolic processing, which has been particularly important in Artificial Intelligence. In the 1970s, the language Scheme was developed from a dialect of Lisp; Scheme is still in use in academic circles.

A Lisp program is composed of sequences of expressions to be evaluated by the language's interpreter. Some expressions are used to define functions which are then called in other expressions.

As the very name of the language implies, a LISP program mainly handles expressions constructed from lists. The basic data structures in LISP, in fact, is the dotted pair—a pair of data items written with a dot separating the two components, for example, $(A . B)$. Such a pair is implemented as a *cons* cell, or rather by the application of the `cons` operator to two arguments, as in `(cons A B)`. The two arguments of `cons` can be values of atomic types (integers, floating point numbers, character strings), or other dotted pairs. We can therefore represent *S-expressions* (symbolic expressions)—binary trees which allow the representation of lists as a particular case. For example, it is possible to construct the list `A B C` as `(cons A (cons B (cons C nil)))`, where `nil` is a particular value denoting the empty list. Among the many interesting characteristics of LISP, programs and data are represented using the same syntax and the same internal representation. This allows the evaluation of data structures as if they were programs and to modify programs as if they were data.

and precedence rules are required. For non-binary operators, infix representation is awkward and seldom used. A programming language which insists on infix notation even for user-defined functions is Smalltalk, one of the first object-oriented languages.

Infix notation is the one used by most programming languages, at least for binary mathematical operators. Often, however, this notation is only an abbreviation or, as we say, a *syntactic sugar* used to make code more readable. For example, in Ada, `a + b` is an abbreviation for `+(a, b)`, in C++ is an abbreviation for `a.operator+(b)`, and, similarly, in Python stands for `a.__add__(b)`.

Prefix Notation

In prefix notation (also called *prefix Polish notation*¹), the operation symbol precedes the symbols representing the operands (written from left to right, in the same way as text). Thus, to write the sum of x and y , we can write $+(x, y)$, or, without using parentheses, $+ x y$; to express the application of the function f to the operands a and b , we write $f(a b)$ or $f a b$.

With prefix notation, parentheses and operator precedence rules are of no relevance, provided that the arity (that is the number of operands) of every operator is known. There is no ambiguity about which operator to apply to any operands, because it is always the one immediately preceding the operands. For example, provided $+$ and $*$ are both binary operators, in

```
* (+ (a b) + (c d))
```

¹ The Polish mathematician W. Łukasiewicz extensively used and studied prefix notation in his mathematical logic writings.

or even

```
* + a b + c d
```

we can only mean the expression represented by $(a+b) * (c+d)$ in normal infix notation.

Most programming languages use prefix notation for unary operators (often using parentheses to group arguments) and for user-defined functions. Some languages use prefix notation also for binary operators. LISP represents functions using a particular notation known as *Cambridge Polish*, which places operators inside parentheses. In this notation, for example the last expression becomes:

```
( * ( + a b ) ( + c d ) ) .
```

Postfix Notation

In postfix (also *reverse Polish*) notation, the operator symbol is placed after the operands. For example, the last expression above when written in postfix notation is:

```
a b + c d + * .
```

Postfix notation is used in the intermediate code generated by some compilers. It is also used in programming languages (for example Postscript).

An advantage of two Polish notations over infix is that they can be used in a uniform fashion to represent operators with any number of operands. In infix notation, on the other hand, representing operators with more than two operands means that we have to introduce auxiliary operators. A second advantage, as we already said, is that we may omit parentheses completely, even if, for the sake of readability, both mathematical prefix notation $f(a\ b)$ and Cambridge Polish $(f\ a\ b)$ keep using parentheses. A final advantage of Polish notations is that it makes the evaluation of an expression particularly simple, as we will see shortly. For this reason, this notation became rather successful during the 1970s and 80s, when it was used for the first pocket-sized calculators.

6.1.2 Semantics of Expressions

The way in which we obtain a value out of an expression depends of course on the notation used. In infix representation the absence of parentheses can cause ambiguity problems, if the precedence rules for different operators and the associativity of every binary operator are not defined clearly. In the case of programming languages, expressions are often represented internally in the form of a tree.

Infix Notation: Precedence and Associativity

When using infix notation, we pay for the easiness of use with major complications in the evaluation mechanism. First of all, if parentheses are not used systematically, it is necessary to clarify the precedence of each operator.

If we write $4+3*5$, for example, clearly we intend the value of 19 as the result of the expression and not 35: mathematical convention, in fact, tells us that we have to perform the multiplication first; that is, the expression is to be read as $4 + (5 * 3)$ and not as $(4 + 3) * 5$. In the case of less familiar operators, present in programming languages, matters are considerably more complex. If, for example, in Pascal we write:

```
x=4 and y=5
```

where `and` is the logical operator and `=` is the comparison operator for equality, contrary to what many will probably expect, we will obtain an error (a static type error) because, according to Pascal's precedence rules, this expression is interpreted as

```
x=(4 and y)=5
```

and not as

```
(x=4) and (y=5) .
```

To avoid excessive use of parentheses (which, when in doubt it is good to use), programming languages employ *precedence rules* to specify a hierarchy between the evaluation order of the different operators. Languages differ considerably in their definition of such rules.

A second problem in expression evaluation concerns operator associativity. If we write $15-5-3$, we could intend it to be read as either $(15-5)-3$ or as $15-(5-3)$, with clearly different results. In this case, mathematical convention says that the usual interpretation should be the first. In more formal terms, the operator “ $-$ ” associates from left to right.² In fact, the majority of arithmetic operators in programming languages associate *from left to right* but there are exceptions. The exponentiation operator, for example, often associates from right to left, as in mathematical notation. If we write 5^{3^2} , or, using a notation more familiar to programmers, $5**3**2$, we mean $5^{(3^2)}$, or $5** (3**2)$, and not $(5^3)^2$, or $((5** 3)**2)$. Thus, when an operator is used, it is useful to include parentheses when in doubt about precedence and associativity.

Associativity is relevant also when mathematically it should make no difference. Consider the sum $a+b+c$. For the mathematician, the result does not depend on the order of evaluation of the two sums (addition is indeed associative). In a programming languages, however, there could be rounding errors involved in the evaluation, which could be very different if we evaluate our expression as $(a+b)+c$ or as $a+(b+c)$.

There are programming languages that present counter-intuitive behaviour. In APL, for example, the expression $15-5-3$ is interpreted as $15-(5-3)$. The reason for this apparent strangeness is that in APL there are many new operators (defined to operate on matrices) that do not have an immediate equivalents in other formalisms.

² A binary operator, `op`, is said to be associative if $x \text{ op } (y \text{ op } z) = (x \text{ op } y) \text{ op } z$ holds; that is, whether it associates from the right to the left or left to right makes no difference.

The designer of the language decided then it was better to have a uniform evaluation strategy, from right to left.

In the design of an algorithm to evaluate an expression in infix notation, the implicit use of precedence and associativity rules, together with the explicit presence of parentheses, complicates matters significantly. In fact, it is not possible to evaluate an expression in a single left-to-right scan (or one from right to left), given that in some cases we must first evaluate the rest of the expression and then come back. For example, in the case of $5+3*2$, when the scan from left to right arrives at $+$, we have to suspend the evaluation of this operator, evaluate $3*2$ and then go back to the evaluation of $+$.

Prefix Notation

Expressions written in prefix notation lend themselves to a simple evaluation strategy which proceeds by simply walking the expression from left to right using a stack to hold its components. Let's assume that the sequence of symbols that forms the expression is syntactically correct and initially not empty. The evaluation algorithm is described by the following steps, where we use an ordinary stack (with the push and pop operations) and a counter C to store the number of operands requested by the last operator that was read:

1. Read in a symbol from the expression and push it on the stack;
2. If the symbol just read is an operator, initialise the counter C with the number of arguments of the operator and go to step 1.
3. If the symbol just read it is an operand, decrement C .
4. If $C \neq 0$, go to 1.
5. If $C = 0$, execute the following operations:
 - (a) Apply the last operator stored on the stack to the operands just pushed onto the stack, storing the results in a register R , eliminate operator and operands from the stack and store the value of R on the stack.
 - (b) If there is no operator symbol in the stack go to 6.
 - (c) Initialise the counter C to $n - m$, where n is the number of the argument of the topmost operator on the stack, and m is number of operands present on the stack above this operator.
 - (d) Go to 4.
6. If the sequence remaining to be read is not empty, go to 1.

The result of the evaluation is located on the stack when the algorithm finishes. This algorithm assumes that we know in advance the number of operands required by each operator. This requires that we syntactically distinguish unary from binary operators. Furthermore, it is generally necessary to check that the stack contains enough operands for the application of the operator (Step 5.(c) in the algorithm above). This check is not required when using postfix notation, as we see below.

Postfix Notation

The evaluation of expression in postfix notation is even simpler. In fact, we do not need to check that all the operands for the last operator have been pushed onto the stack, since the operands are read (from left to right) before the operators. The evaluation algorithm is then the following (as usual, we assume that the symbol sequence is syntactically correct and is not empty):

1. Read the next symbol in the expression and push it on the stack.
2. If the symbol just read is an operator apply it to the operands immediately below it on the stack, store the result in R , pop operator and operands from the stack and push the value in R onto the stack.
3. If the sequence remaining to be read is not empty, go to 1.
4. If the symbol just read is an operand, go to 1.

This algorithm also requires us to know in advance the number of operands required by each operator.

6.1.3 Evaluation of Expressions

As we saw the start of the Chap. 2 and as we extensively discussed in that chapter, expressions, like the other programming language constructs, can be conveniently represented by trees. In particular, they can be represented by a tree (called the expression's *syntax tree*) in which:

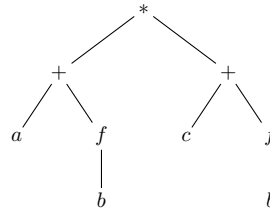
- Every non-leaf node is labelled with an operator.
- Every subtree child of a node N is an operand for the operator associated with N .
- Every leaf node is labelled with a constant, variable or other elementary operand.

Trees of this kind can be directly obtained from the derivation trees of an (unambiguous) grammar for expressions, by eliminating non-terminal symbols and by an appropriate rearrangement of the nodes. Given the tree representation, the linear infix, prefix, and postfix representations can be obtained by traversing the tree in a symmetric, prefix, or postfix order, respectively. The representation of expressions as trees clarifies (without needing parentheses) precedence and associativity of operators. The subtrees found lower in the tree constitute the operands and therefore operators at lower levels must be evaluated before those higher in the tree. For example the tree shown in Fig. 6.1 represents the expression:

$$(a + f(b)) * (c + f(b))$$

This expression can be obtained (parentheses apart) from the symmetric-order traversal of the tree (f is here an arbitrary unary operation).

For languages with a compilative implementation, the parser implements syntactic analysis by constructing a derivation tree. In the specific case of expressions then, infix representation in the source code is translated into a tree-based representation.

Fig. 6.1 An expression

This representation is then used by successive phases of the compilation procedure to generate the object code implementing runtime expressions evaluation. This object code clearly depends on the type of machine for which the compiler is constructed. In the case in which we have a traditional physical machine, for example, code of a traditional kind (i.e. in the form `opcode operand1 operand2`) is generated which uses registers as well as a temporary memory locations to store intermediate results of evaluation.

In some particular cases, on the other hand, object code can be represented using a prefix or postfix form which is subsequently evaluated by a stack architecture. This is the case for example in the executable code for many implementations of SNOBOL4 programs.³

In the case of languages with an interpretative implementation, it is also convenient to translate expressions, normally represented in the source code in infix notation, into a tree representation which can then be directly evaluated using a tree traversal. This is the case, for example, in interpreted implementations of LISP, where the entire program is represented as a tree.

It is beyond the scope of the present text to go into details on mechanisms for generating code or for evaluating expression in an interpreter. We only discuss below some points which often cause ambiguity. For convenience, we will fix on expressions represented in infix form. We will see that what we have to say applies equally to the direct evaluation of expressions represented as a tree, as well as to code generation-mechanisms.

Subexpression Evaluation Order

Infix notation precedence and associativity rules (or the structure, when expressions are represented as trees) do not say anything about the order to be used to evaluate an operator's operands (i.e., nodes at the same level). For example, in the expression in Fig. 6.1, nothing tells us which among $a + f(b)$ or $c + f(b)$ should be evaluated first. Moreover, nothing is said about whether operands should be evaluated before their operator; nor whether expressions that are mathematically equivalent can be

³ SNOBOL4 ("StriNg Oriented and symBOLic Language") is a language for string manipulation of the end of sixties, that had patterns as a first-class data type. Patterns in SNOBOL may describe arbitrary context-free grammars, as opposed to standard patterns of AWK, or Perl (the languages that took the place of SNOBOL in the eighties), which describe instead regular languages.

inter-substituted without modifying the result (for example, $(a-b+c)$ and $(a+c-b)$ could be considered equivalent).

While in mathematical terms these differences are unimportant (the result does not change), from our viewpoint these questions are relevant, for the following five reasons.

Side effects In imperative programming languages, expression evaluation can modify the state through so-called side effects. A side effect is an action that influences the result (partial or final) of a computation without otherwise explicitly returning a value in the context in which it is found. The possibility of side effects makes the order of evaluation relevant to the final result. In our example in Fig. 6.1, if the evaluation of the function f were to modify the value of its operand through side effects, first executing $a+f(b)$ rather than $c+f(b)$, could change the value produced by the evaluation (see Exercise 1). As far as side effects are concerned, languages follow various approaches. On the one hand, pure declarative languages do not permit side effects at all. In imperative languages, the most common case is that the presence of side effects is permitted, and hence the value of an expression depends on the evaluation order. Java and Python, for instance, state in their definition that evaluation always proceeds left-to-right. The definition of C, on the other hand, does not specify any order, meaning that each compiler may choose the order it deems more convenient.⁴

Finite arithmetic Given the set of numbers represented in a computer is finite (see also Sect. 8.3), reordering expressions can cause overflow problems. For example, if a has, as its value, the maximum integer representable and b and c are positive numbers such that $b > c$, right-to-left evaluation of $(a-b+c)$ does not produce overflow, while we have an overflow from the evaluation from left to right of $(a+c-b)$. Moreover, even if there is no overflow, the limited precision of computer arithmetic implies that changing the order of the operands can lead to different results (this is particularly relevant in cases of floating point computation).

Undefined operands When the application of operator to operands is considered, two evaluation strategies can be followed. The first, called *eager evaluation*, consists of first evaluating all the operands and then applying the operator to the values thus obtained. If we think of normal arithmetic operators, this strategy seems the most reasonable. In programming languages, however, we have expressions posing problems which do not normally appears in arithmetic expressions. One of the main issues is that an expression may be defined and have a value even when some of its components are undefined. Let us consider a conditional expression of the form:

```
a == 0 ? b : b/a
```

⁴ The definition of C does not even require that two copies of the same expression in a program are evaluated using the same evaluation order.

We can write this in C to denote the value of b/a when a is non-zero and b , otherwise. This expression results from the application of a single operator (expressed in infix notation using two binary operators `?` and `:`) to three operands (the boolean expression, $a \neq 0$, and the two arithmetic expressions b and b/a). We cannot use eager evaluation for such conditional expressions, because the sub-expression b/a would have to be evaluated even when a is equal to zero, producing an error.

In such a case, it is therefore better to use a *lazy evaluation* strategy which consists of *not* evaluating operands before the application of the operator, but in passing the un-evaluated operands to the operator, which, when it is evaluated, will decide which operands are required, and will only evaluate the ones it requires.

The lazy evaluation strategy, used in some declarative languages, is more expensive to implement than eager evaluation and for this reason, most languages use eager evaluation (with the significant exception of conditional expressions as we will see below). There are languages which use a mix of both the techniques (ALGOL, for example). We will discuss the various strategies for evaluating expressions in greater detail when we consider functional languages in Chap. 11.

Short-circuit evaluation The problem detailed in the previous point presents itself with particular clarity when evaluating Boolean expressions. For example, consider the following expression (in C syntax):

```
a == 0 || b/a > 2
```

If the value of a is zero and both operands of `||` are evaluated, it is clear that an error will result (in C, “`||`” denotes the logical operation of disjunction). To avoid this problem, and to improve the efficiency of the code, C, like other languages uses a form of lazy evaluation, also called *short-circuiting evaluation*, of boolean expressions. If the first operand of a disjunction has the value *true* then the second is not evaluated, given that the overall result will certainly have the value *true*. In such a case, the second expression is *short-circuited* in the sense that we arrive at the final value before knowing the value of all of the operands. Analogously, if the first operand of a conjunction has the value *false*, the second is not evaluated, given that the overall result has value *false*.

The main reason for a short-circuit evaluation of Boolean operators is not efficiency, however. With full evaluation, certain common program schemata would be incorrect, or require baroque coding. Let us make an example in Pascal, which does not use short-circuit evaluation: the linear scan of a sequential list.

```
p := list;
while (p <> nil) and (p^.value <> 3) do
  p := p^.next;
```

The intention of this code is to traverse a list until we have arrived at the end or until we have encountered the value 3. However, this is badly written code that can produce a runtime error. In the case in which we have $p = \text{nil}$, the second operand of the conjunction ($p^.value \neq 3$) yields an error when it dereferences a null pointer. Similar code, on the other hand, *mutatis mutandis*, can be written in C without causing problems. In order to avoid ambiguity, some

languages (for example C and Ada), explicitly provide different boolean operators for short-circuit evaluation.⁵ Finally, it should be noted that this kind of evaluation can be simulated using a conditional command (see Exercise 2).

Optimisation Frequently, the order evaluation of subexpressions influences the efficiency of the evaluation of an expression for reasons relating to the organisation of the physical machine. For example, consider the following code:

```
a = vector[i];  
b = a*a + c*d;
```

In the second expression, it is probably better first to evaluate $c*d$, given that the value of a has to be read from memory (with the first instruction) and might not be yet available; in such a case, the processor would have to wait before calculating $a*a$. In some cases, the compiler can change the order of operands in expressions to obtain more efficient, but semantically equivalent, code.

The last point explains many of the semantic problems that appear while evaluating expressions. Given the importance of the efficiency of the object code, it could be wise to give liberty to the compiler in the use of the expression evaluation method, without it being specified at the level of the semantic description of the language. This is the reason why, as we already said, C does not specify an evaluation order. On the other hand, with this under-specification, different implementations of the same language may produce different results for the same expression, or have errors at runtime whose source is hard to determine. And this is the reason why other languages, like Java or Python, do specify the evaluation order in their definition.

Wishing to capitalize in a pragmatic prescription what we have said so far, unless we know very well the language we use *and* the specific implementation we are using, it is wise to use all possible means to eliminate as many sources of ambiguity as possible in expression evaluation (such as brackets, parentheses, specific boolean operations, auxiliary variables in expressions, etc.).

6.2 The Concept of Command

If expressions are present in all programming languages, the same is not true for commands. They are constructs that are typically present in so-called imperative languages.

Definition 6.2 (*Command*) A command is a syntactic entity whose evaluation can have a side effect, and does not necessarily return a value.

⁵ In Python, the special, short-circuit evaluation of Boolean operators is the reason why those operators are not defined in terms of “magic” methods, like any other operator.

A command, or more generally, any other construct, has a side-effect if it influences the result of the computation but its evaluation returns no value to the context in which it is located. One of the simplest side effects is the one we obtain with output commands. Let us suppose that the `print` command could be used to print character strings supplied as argument. When the command `print "pippo"` is evaluated, we will not obtain a value but only a side-effect—the characters “pippo” appear on the output device.

The definition of command, just as the previous definition of expression, it is not precise, given that we have referred to an informal concept of evaluation (the one performed by the abstract machine of the language to which the command or the expression belongs). It is clear that we can always modify the interpreter so that we obtain some value as a result of the evaluation of a command. This is what happens in some languages (for example in C, assignment also returns the value to the right of `=`, see Sect. 6.2.2).

A precise definition and an exact distinction between expressions and commands on the basis of their semantics is possible only in the setting of a formal definition of a language. In Sect. 2.5, we saw a simple example. The evaluation of an expression, given an initial state, produces both a value and a (possibly modified) state (to take care of side effects inside the expression). On the other hand, the evaluation of a command, given an initial state, is a new state, produced by the side effects of the command itself. A command is therefore a construct whose purpose is the modification of the *state*, which, in the simple example of Sect. 2.5, takes into account the value of all the variables of the program.

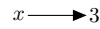
The assignment command is the elementary construct for modifying the state. Before dealing with it, however, it is necessary to clarify the concept of variable.

6.2.1 The Variable

In mathematics, a variable is a symbol (a *name*) that stands for a value, taken from some predetermined domain. In programming languages, it is still a name that represents an object, but the apparent simplicity of the concepts hides several subtleties. Moreover, imperative languages use variables differently than logic and functional languages (see Sects. 11.1 and 12.3).

The first notion we deal with is that of *modifiable variable*, common to most imperative programming languages. A modifiable variable is a name that represents (stands for) a “piece of mutable store”, a sort of container, or location (clearly referring to physical memory). The location contains a value (the “value of the variable”), which may change over time (whence the adjective “modifiable”), by the effect of assignment commands.

We may represent modifiable variables as in Fig. 6.2. The name `x` stands for the small box containing the value 3, which may be re-filled with other values. We stress that the name and the container are two different entities, although we usually say “the variable `x`” for “the container whose name is `x`”.

Fig. 6.2 A modifiable variable**Fig. 6.3** The reference model for a variable

Not all imperative languages use modifiable variables. In some of them, a variable is a name for a *reference* (that is, a mechanism allowing access) to a value.⁶ When this is the case, we will say that a language uses a *reference model* of variables.⁷ We may represent a variable in the reference model as in Fig. 6.3. Imperative languages with a reference model may have containers as values, thus allowing for side effects like the ones that are possible with the modifiable variable. Languages using a reference model include Python, Scala, and Java (only for class types).

Modifiable variables and variables in reference model behave differently in the context of the assignment command, as we will see in the next section.

(Pure) functional languages (see Sect. 11.1) use a concept of variable similar to the mathematical one—a variable is a name that stands for a value. It is often said that functional languages “do not have variables”, meaning that (in their pure forms) they do not have modifiable variables (or, equivalently, they have a reference model where values cannot be containers.)

Logic languages also use names associated to values as variables and, as with functional languages, once a link between a variable identifier and a value is created, it can never be eliminated. There is however a mode in which the value associated with a variable can be modified without altering the link, as will be seen in Sect. 12.3.

6.2.2 Assignment

Assignment is the basic command for the “modification of a variable”, that is, for changing the value (or the content) associated with a name. It is the most elementary command in every imperative language; still, there are various subtleties to be taken into account.

Let us first treat the case of the modifiable variable. In Pascal one may write

```
x := 2
```

to indicate that the variable *x* is assigned the value 2. After this command the container associated with the variable (whose name is) *x* contains the value 2 in place of the value that was there before. Since the command does not return any value, we say that the effect of the assignment is a “side effect”. From now on, the

⁶ A reference is the abstract version of a pointer, Sect. 8.4.6.

⁷ One of the earliest languages to use this model is CLU, where it was called the “object model” [1]. We borrow the term “reference model” from [2].

use of X (“an access to X ”) will return the value 2. Moreover, the value previously stored in the container is lost. Consider now the command

```
 $X := X + 1$ 
```

which assign to X its previous value incremented by 1. The same name, X , is used with a different meaning on the two sides of the assignment operator. The X on the left of the $:=$ symbol is used to indicate the container (the location) to be used for storing the new value. The occurrence of X on the right of the $:=$ denotes the value inside the container.

We thus distinguish two kind of values: *l-values* are those values that indicate locations and therefore are the values of expressions that can be on the left of an assignment command. On the other hand, *r-values* are the values that can be stored in locations, and therefore are the values of expressions that can appear on the right of an assignment command. If the language has a reference model for variables, *l-values* denote references, and not locations.

Assignment Semantics: Modifiable Variables Versus Reference Model

The assignment command has the form of a binary operator in infix form:

```
exp1 OpAss exp2
```

where `OpAss` indicates the symbol used in the particular language to denote assignment (e.g., `:=` in Pascal, `=` in C, FORTRAN, Python and Java, `←` in APL, etc.). The semantics of such a command, in the case of modifiable variables, include the following steps: compute the *l-value* of `exp1`, determining a container *loc*; compute the *r-value* of `exp2`, obtaining a value *val*; modify the contents of *loc* by substituting *val* for the one previously there.⁸

In the case of the reference model for variables, instead, we may describe the semantics of the same assignment as follows: compute the *l-value* of `exp1`, determining a reference *ref*; compute the *r-value* of `exp2`, obtaining a value *val*; change the reference *ref*, so that now it refers to *val*. Note that *val* is *not* copied (duplicated) as it happens for the modifiable variable.

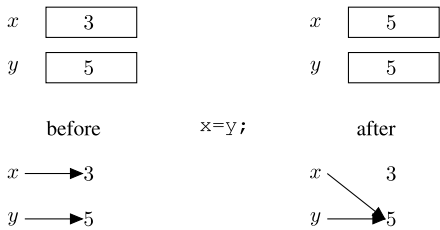
The different behavior in the two models becomes clearer if we consider an assignment between two variables:

```
 $x = y ;$ 
```

Under the modifiable variable model, the value “contained” in y is copied into the container for x . Under the reference model, instead, after the assignment x and y are two references to the same object (we have an *aliasing* situation). The behavior of assignment in the two contexts is graphically represented in Fig. 6.4.

⁸ The order in which these operations are performed—relevant in case `exp1` or `exp2` have side effects—is language specific. For example, Java allows the left-hand side to be evaluated before the right-hand side; C leaves this decision to the implementation (and then the same code could have different effects on different implementations); Python guarantees that `exp2` is evaluated before `exp1`.

Fig. 6.4 Assignment:
modifiable variable (top) vs
reference model (bottom)



As we said in the previous section, in the reference model we may have names that refer to containers (for example, records, or arrays, or instances of classes, etc.) If this is the case, after the assignment $x=y$; a modification performed on a value (a container) accessed using the name x becomes visible also through the name y , and vice versa.

Below, unless otherwise specified, when we talk about variables, we mean the modifiable variable.

Variations on the Theme

Which expressions denote (on the left of an assignment) an l-value depends on the programming language: the usual cases are variables, array elements, record fields (note that, as a consequence, calculation of an l-value can be arbitrarily complex because it could involve function calls, for example when determining an array index).

In some languages, for example C, assignment both produces a side effect, and returns the r-value computed for the right-hand side. Thus, in C the evaluation of

$x = 2$;

in addition to assigning 2 to x , returns the value 2. Therefore we can also write:

$y = x = 2$;

which should be interpreted as:

$(y = (x = 2))$;

This command assigns the value 2 to x as well as to y .

Forms of “chained assignment” are possible also in languages where assignment does not return a value. This is the case of Python,⁹ where a chained assignment

$x = y = \text{exp}$

is equivalent to (note that the leftmost name is assigned first)

⁹ Python 3.8 introduces assignment *expressions*: `name := exp` returns the value of `exp` and assign this value to `name`. Unless parenthesized, however, it cannot be used in all the contexts where a normal assignment (`=`) is used (e.g., at the top level, or in a chained assignment).

```
tmp=exp
x=temp
y=temp
```

Python also allows simultaneous multiple assignment, coherently with the presence of tuples as primitive data types:

```
x, y=exp1, exp2
```

evaluates first `exp1` and `exp2`, determining two values v_1 and v_2 ; then assigns v_1 to `x` and v_2 to `y`. Observe that

```
x, y=y, x
```

swaps the values associated to `x` and `y`.

In many languages (C and its descendants, Java, Python, and others) we find also *augmented* (or compound) assignments, to increase code readability or avoid unforeseen side effects. The usual assignment for incrementing a variable

```
x = x+1;
```

requires, in principle, two accesses to the variable `x`: one to determine the l-value, and one to obtain the r-value. Even if this can easily be optimised by a compiler, it remains an issue in presence of side-effects. Let us consider the code:

```
a[f()] = a[f()+1];
```

where `a` is a vector (of at least 3 elements) and `f` is a function defined as follows:

```
b = 0;
int f() {
    if b == 0 {
        b=1;
        return 1;
    }
    else return 2;
}
```

This function is defined in such a way that the non-local reference to `b` in the body of `f` refers to and modifies the global `b`. But then

```
a[f()] = a[f()+1]
```

does not increment the value of an element of `a`. Instead, it assigns the value of `a[1]+1` to `a[2]` whenever the evaluation of the right-hand side of the assignment *precedes* the evaluation of the left-hand side.¹⁰ Moreover, the compiler cannot optimise the computation (with a single call to `f`), because the programmer might have wanted this apparently anomalous behaviour.

Augmented assignments come to help on this subject. In the example above we could write

¹⁰ It assigns the value of `a[2]+1` to `a[1]` whenever the evaluation of the left-hand side precedes the evaluation of the right-hand side.

Environment and Memory

In Chap. 2, we defined the semantics of a command by referring to a simple notion of state, which we defined as a function that assigns to each variable present in the program the value it takes. This concept of state, although adequate for didactic purposes in mini-languages, is not sufficient when we want to describe the semantics of real programming languages which use modifiable variables and assignments. In fact, we have already seen in Chap. 4 (and will see in more detail in the next two chapters) that parameter-passing mechanisms as well as pointers can easily create situations in which two different names, for example X and Y , refer to the same variable, or rather the same location in memory. Such case of aliasing cannot be described using a simple function $State: Names \rightarrow Values$ because with a simple function it is not possible to express the fact that a modification of the value associated with (the variable denoted by) X also reflects on the value associated with Y . To correctly express the meaning of modifiable variables, we therefore use two separate functions. The first, called the *environment*, mostly corresponds to the concept of environment introduced in Chap. 4: in other words, it is a function $Environment: Names \rightarrow DenotableValues$ which maps names to the values they denote. The set (or, as one says in semantics jargon, the *domain*) of names often coincides with that of identifiers. The domain $DenotableValues$, instead, includes all values to which a name can be given; what these values are depends on the programming language but if the language provides modifiable variables then this domain certainly includes memory locations.

The values associated with locations are, on the other hand, expressed by a function $Memory: Locations \rightarrow StorableValues$ which (informally) associates every location with the value stored in it. In this case also, what exactly is a storable value depends on the specific language.

Therefore, when we say that “in the current state the variable X has the value 5”, formally we mean to say that we have an environment ρ and a memory σ such that $\sigma(\rho(X)) = 5$. Note that when a variable is understood to be an l-value, we are interested only in the location denoted by the name and, therefore, only the environment is used, while when we understand it as an r-value the store is also used. For example given an environment ρ , and a store σ , the effect of the command $X=Y$ is to produce a new state in which the value of $\rho(\sigma(Y))$ is associated with $\rho(X)$. Let us recall, for completeness, that a third value domain important in the language semantics are the *ExpressibleValues*: these are those values which can be the result of the evaluation of a complex expression.

```
a [ f ( ) ] += 1 ;
```

for saying that we want to call $f()$ once, and then increment $a[f()]$ by one. Augmented assignments come with a variety of possible operations ($+=$, $-=$, etc.). In all languages, the left-hand side is evaluated only once.¹¹

¹¹ It is language specific the time *when* the left-hand side is evaluated. It is interesting that in Python, while in a standard assignment $X=\text{exp}$, the right-hand side is evaluated before the left-hand side, in an augmented assignment $X+=\text{exp}$ it is the left-hand side to be evaluated first.

Imperative and Declarative Languages

Denotable, storable and expressible values, even if they have a non-empty intersection, are conceptually distinct sets. In effect, many important differences between various languages depend on how these domains are defined. For example, functions are denotable but not expressible in Pascal, while they are expressible in Lisp, ML, and Python. A particularly important difference between various languages concerns the presence of storable values and the *Memory* semantic function which we saw in the previous box. In fact, in a rather synthetic fashion, we can classify as *imperative* those languages which have environments as well as memory functions; those languages which have only environments are *declarative*. Imperative languages, while they are high-level languages, are inspired by the physical structure of the computer. In them the concept of memory (or state) is interpreted as the set of associations between memory locations and values stored in those locations. A program, according to this paradigm, is a set of *imperative* commands and computation consists of a sequence of steps which modify the state, using as its elementary command the assignment. The terminology “imperative” here has to do with natural language: as in an imperative phrase, we say “take that apple” to express a command, so with an imperative command we can say “assign to x the value 1”. Most programming languages normally used belong to the imperative paradigm (Fortran, ALGOL, Pascal, C, etc.).

Declarative languages were introduced with the aim of offering higher level programming paradigms, close to the notations of mathematics and logic, abstracting from the characteristics of the physical machine on which the programs are executed. In declarative languages (or at least in “pure” versions of them) there are no commands to modify the state, given that there are neither modifiable variables nor a semantic memory function. Programs are formed from a set of declarations (from which the name is derived) of functions or relations which define new values. According to the elementary mechanism used to specify the characteristics of the result, declarative languages are divided into two classes: functional and logic programming languages (the latter, also called logic languages for short). In the first case, computation consists of the evaluation of functions defined by the programmer using rules of a mathematical kind (mostly composition and application). In the second form of declarative language, on the other hand, computation is based on first-order logical deduction. Let us recall that there exist “impure” functional and logic languages that also have imperative characteristics (in particular, they include assignment).

6.3 Sequence Control Commands

Assignment is the basic command in imperative languages (and in “impure” declarative languages); it expresses the elementary computation step. Most other commands define sequence control, that is, are used to specify the order in which state modifications produced by assignments are to be performed. These other commands can be divided into three categories:

Commands for explicit sequence control These are the sequential command and `goto`. We treat in this category also the composite command, which allows us to consider a group of commands as a single one.

A Quibble About “;”

Being rigorous, the “;” used to separate commands is not always a sequential command. In C, C++ and Java, for example, it is a command *terminator* more than an operator expressing concatenation. In fact, this can be easily seen when we deal with the last command in a block. In these languages, the “;” is always required, even if the command is not followed by another as in

```
{ x = 1 ;
  x = x + 1 ;
}
```

In languages like Pascal, on the other hand, “;” is really an operator that sequentialises commands. The same example as above can be written as:

```
begin
    x := 1 ;
    x := x + 1
end
```

To insert a “;” without there being a command to follow it is, however, a venial programming sin which the compiler absolves by inserting an *empty command* between the “;” and the `end`. An empty command is not denoted by anything in the syntax and does not correspond to any action.

Conditional (or selection) commands These are the commands that allow the specification of alternative paths that the competition can take. They depend on the satisfaction of specific conditions.

Iterative commands These allow a given command to be repeated a pre-defined number of times or until certain conditions are met.

6.3.1 Commands for Explicit Sequence Control

Sequential Command

The *sequential command*, indicated in many languages by a “;”, allows us to specify the sequential execution of two commands. If we write:

```
C1 ; C2
```

the execution of C2 starts after C1 terminates. In languages in which the evaluation of a command also returns a value, the value returned by the evaluation of the sequential command is that of the second argument.

Obviously we can write a sequence of commands such as:

```
C1 ; C2 ; ... ; Cn
```

with the implicit assumption that the operator “;” associates to the left.

Composite Command

The *composite command* groups a sequence of commands into a single one. It is expressed by suitable delimiters such as those used by ALGOL:

```
begin  
...  
end
```

or those in C, Scala, Java:

```
{  
...  
}
```

Python expresses grouping of commands by the positive indentation of those commands.

Such a composite command, also called a *block*, can be used in any context in which a simple command is expected and therefore, in particular, can be used inside another composite command to create a tree-like structure of arbitrary complexity.

Goto

The `goto` command occupies a unique place in the panorama of sequential control commands. It was part of the first programming languages and still continues to be included in many used languages. This command is directly inspired by jump instructions in assembler languages and therefore by the sequence control of the hardware machine. The execution of the command

```
goto A
```

transfers control to the point in the program at which the label `A` occurs (different languages differ in what exactly constitutes a label but these differences are not relevant to us).

Despite its apparent simplicity and naturalness, the `goto` command has been at the centre of a considerable debate since the start of the 1970s (see, for example, a famous article by Dijkstra referred to in the bibliography), between its supporters and its detractors, who in the end won the match. To clarify the sense of this debate, let us, first of all, observe that the `goto` is not essential to the expressiveness of a programming language. A theorem due to Böhm and Jacopini in fact shows that any program can be translated into an equivalent one which does not use the `goto` (the formulation of the theorem, obviously, is much more precise than our account). This result, however, does not come down on one side or the other. On the one hand, we may use the theorem to assert that the `goto` is useless. On the other hand, we may observe that the transformations used by the theorem to obtain a `goto`-less program, completely destroy the structure of the original program. Hence, `goto` is “in principle” useless, but at the price of writing artificially complex programs.

The nexus of the question really is not of a theoretical nature but of a pragmatic one. Using `goto`, it is easily possible to write code which soon becomes incomprehensible and which still remains incomprehensible when all `gotos` are eliminated. We can think, for example, of a program of some considerable size where we have inserted

jumps between points which are some of thousands of lines of code apart. Or, we can think of a subprogram in which exits are made at different points based on some condition and the exits are performed by `gotos`. These and other arbitrary uses of this construct make the code hard to understand, and therefore hard to modify, correct and maintain; this has the obvious negative consequences in terms of cost. To all of this, we can add the fact that the `goto` with its primitive method of transferring control, does not accord well with other mechanisms present in high-level languages. What happens, for example, if we jump *inside* of a block? When and how is the activation record for this block initialised so that everything works correctly?

If `goto` were used in an extremely controlled fashion, locally to small regions of code, the majority of these disadvantages would disappear. However, the cases in which it can be useful to use this command, such as exit from loops, return from a subprograms, handling of exceptions, can, in modern programming languages, be handled by specific, more appropriate, constructs. We can therefore assert that in modern high-level languages, the `goto` is just an archeological relic. Java has been the first commercial language to not include `goto` into its set of commands. It has been followed by most other recent languages.

Other Sequence Control Commands

If `goto` is dangerous in its general form, there are local and limited uses of it which are useful in given circumstances. Many languages make available limited forms of jump to confront these pragmatic necessities without having to make use of the brute force of a `goto`. Among these commands (which take on different forms in different languages), we find constructs such as `break` (for terminating the execution of a loop, of a case, or, in some languages, of the current block), `continue` (for terminating the current iteration in an iterative command and force the starting of next iteration) or `return` (to terminate the evaluation of the function, returning control to the caller, sometimes also passing a value).

Finally, a more elaborate sequence control can be implemented using exceptions, which will be treated in Sect. 7.3.

6.3.2 Conditional Commands

Conditional commands, or selection commands, express one alternative between two or more possible continuations of the computation, based on appropriate logical conditions. We can divide conditional commands into two groups.

If

The `if` command is present in almost all languages, in various syntactic forms which can be reduced to the form:

```
if Bexp then C1 else C2
```

where `Bexp` is a boolean expression, and `C1` and `C2` are commands. Informally, the semantics of such a command expresses an alternative in the execution of the

computation, based on the evaluation of the expression `Bexp`. When this evaluation returns true, the command `C1` is executed, otherwise the command `C2` is executed. The command is often present in the form without the `else` branch:

```
if Bexp then C1
```

In this case, too, if the condition is false, the command `C1` is not executed and control passes to the command immediately after the conditional. As we saw in Chap. 2, nesting several `if` as in the command

```
if Bexp1 if Bexp2 then C1 else C2
```

causes problems of ambiguity, which can be resolved using a suitable grammar which formally describes the rules adopted by the language (for example, the `else` branch belongs to the innermost `if`; this is the rule used in almost every language). To avoid problems of ambiguity, some languages use a “terminator” to indicate where the conditional command ends, as for example in:

```
if Bexp then C1 else C2 endif
```

Furthermore, in some cases, instead of using a list of nested `if then else`s, use is made of an `if` equipped with more branches, analogous to the following:

```
if Bexp1 then C1
    elseif Bexp2 then C2
    ...
    elseif Bexpn then Cn
    else Cn+1
endif
```

The implementation of the conditional command poses no problems, and makes use of instructions for test and jump that are found in the underlying physical machine.¹²

Case

The `case` command is a specialisation of the `if` command, with more branches. In its simplest form it is written as follows:

```
case Exp of
    label1: C1;
    label2: C2;
    ...
    labeln: Cn;
else Cn+1
```

where `Exp` is an expression whose value is of a type compatible with that of the labels `label1`, ..., `labeln`, while `C1`, ..., `Cn+1` are commands. Each label is represented by one or more constants and the constant used in different labels are different from each other. The type permitted for labels, as well as their form, varies

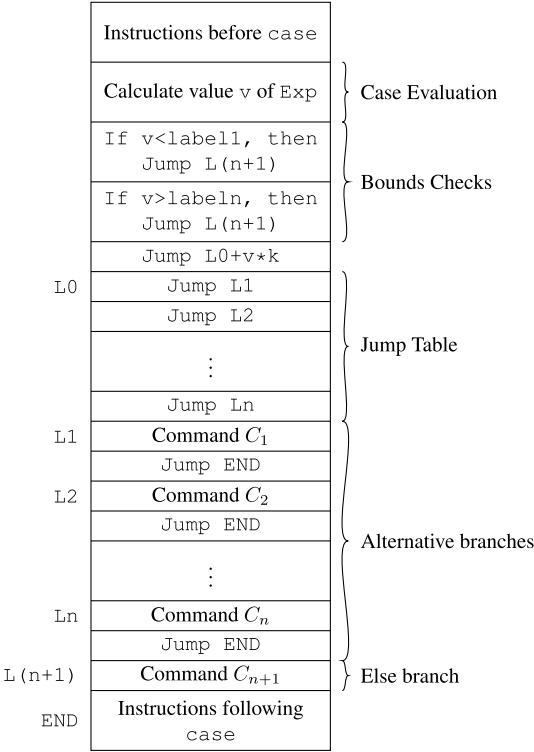
¹² At the assembly language level, and therefore in the language of the physical machine, there are jump operations, conditional or not, analogous to the `goto` in high-level languages.

from language to language. In most cases, a discrete type is permitted (see Sect. 8.3), including enumerations and intervals. So, for example, we can use the constants 2 and 4 to denote a label, but in some languages we can also write $2, 4$ to indicate either the value 2 or the value 4, or $2 \dots 4$ to indicate all values between 2 and 4 (inclusive).

The meaning of this command is analogous to that of a multi-branch `if`. Once the expression `Exp` has been evaluated to a value v , the command which occurs in the unique branch whose label includes v is executed. The `else` branch is executed whenever there is no branch containing v in the label.

Whatever can be done using a `case` can certainly be expressed using a nested series of `ifs`. Even so, many languages include some form of case in their commands, either to improve the readability of the code, or because it is possible to compile a `case` much more efficiently than a long series of nested `ifs`. A `case` is, in fact, implemented in assembly language using a vector of contiguous cells called a *jump table*, in which each element of the table contains the address of the first instruction of the corresponding command in the case's branches. The use of such a table is shown in Fig. 6.5, where, for simplicity, it is assumed that the labels `label1`, $\dots$, `labeln` are the consecutive constants 0, 1, $\dots$, $n - 1$. As should be clear in the figure, the expression which appears as an argument to the `case` is evaluated first of all. The

Fig. 6.5 Implementation of `case`



value thus obtained is then used as an offset (index) to compute the position in the jump table of the instruction which performs the jump to the chosen branch. The extension of this mechanism to the general case in which labels are sets or intervals is simple (see Exercise 3).

This implementation mechanism for `case` gives greater efficiency than a series of nested `ifs`. Using a jump table, once the value of the expression is calculated, two jump instructions are required to arrive at the code of the command to execute. Using nested `ifs`, on the other hand, for n alternative branches (in the worst case of an unbalanced `if`), it is necessary to evaluate $O(n)$ conditions and perform $O(n)$ jumps before arriving at the command of interest. The disadvantage of using a jump table is that, since it is a linear structure whose contiguous elements correspond to successive label values, it can consume a lot of space when the label values are dispersed over a fairly wide interval or when the individual labels are a type denoting a wide interval. In this case alternative methods can be used for calculating the jump address, such as sequential tests hashing or methods based on binary search.

Different languages exhibit significant differences in their `case` commands. In C, for example, the `switch` has the following syntax (also to be found in C++ and in Java):

```
switch (Exp) body
```

where *body* can be any command. In general, though, the *body* is formed from a block in which some commands can be labelled; that is they are of the form:

```
case label : command
```

while the last command of the block is of the form:

```
default : command
```

When the expression `Exp` is evaluated to v , the control is transferred to the command whose label is v ; if there are no labels with such a value, control passes to the command with the label `default`. If there is no `default` command, control passes to the first command following the `switch`. Observe that, with this semantics, once the branch of the `switch` has been selected, control then flows into the immediately following branches. To obtain a construct with semantics analogous to that of the `case` we discussed above, it is necessary to insert an explicit control transfer at the end of the block, using a `break`:

```
switch (Exp) {
    case label1: C1 break;
    case label2: C2 break;
    ...
    case labeln: Cn break;
    default: Cn+1 break;
}
```

If the value of `Exp` does not appear in any label, the entire command has no effect. Finally, lists or ranges of values are not permitted as labels. This however is no real limitation, given that lists of values can be obtained by exploiting the fact that control

passes from one branch to its successor when `break` is omitted. If, for example, we write:

```
switch (Exp) {  
    case 1:  
    case 2: C2 break;  
    case 3: C3 break;  
    default: C4 break;  
}
```

the command C2 is executed when `Exp` has value 1 or 2.

6.3.3 Iterative Commands

With the exception of `goto`, all the commands that we have seen up to this point only allow us to express finite computations, whose maximum length is determined statically by the length of the program text.¹³ A language which had only such commands would be of highly limited expressiveness. It would certainly not be Turing complete (recall Sect. 3.3), in that it would not permit the expression of all possible algorithms (consider, for example, scanning a vector of n elements, where n is not known a priori).

In low-level languages, the expressive power to express all possible algorithms is obtained through jump instructions, allowing the repetition of groups of instructions by jumping back to the start of the code. In high-level languages, avoiding the `goto` command, two basic mechanisms are employed to achieve the same effect: *structured iteration* and *recursion*. The first, which we consider in this section, is more familiar from imperative languages (they almost always allow recursion as well). Suitable linguistic constructs (which we can regard as special versions of the jump command) allow us to implement loops in which commands are repeated or *iterated*. At the linguistic level, we distinguish between *unbounded iteration* and *bounded iteration*. In bounded iteration, repetition is implemented by constructs that allow a determinate number of iterations, usually fixed before the iteration starts. In unbounded iteration, on the other hand, repetition is controlled by whether a condition becomes true. Bounded and unbounded iteration have different expressive power, at least in their “pure” form.

Recursion, which we will consider in the next section, allows the expression of loops in an implicit fashion, by allowing a function to call itself, thereby repeating its own body an arbitrary number of times. The use of recursion is more common in declarative languages (in many functional and logic languages there is no iterative construct).

¹³ It can easily be seen that the maximum length of the computation is a linear function of the length of the program.

Unbounded Iteration

Unbounded iteration is logically controlled iteration. It is implemented by linguistic constructs composed of two parts: a loop *condition* (or *guard*) and a *body*, which is a (possibly compound) command. When executed, the body is repeatedly executed until the guard becomes false (or true, according to the construct).

The most common form of unbounded iteration is that of the `while` command, originally introduced in ALGOL:

```
while (Bexp) do C
```

The meaning of this command is as follows: (1) the boolean expression `Bexp` is evaluated; (2) if this evaluation returns the value *true*, execute the command `C` and return to (1); otherwise the `while` command terminates.

In some languages there are also commands that test the condition *after* execution of the command (which is therefore always executed at least once). This construct is for example present in Pascal in the following form:

```
repeat C until Bexp
```

which can be seen as an abbreviation for:

```
C;  
while not Bexp do C
```

(`not Bexp` here indicates the negation of the expression `Bexp`). In C an analogous construct is `do`:

```
do C while (Bexp)
```

which corresponds to:

```
C;  
while Bexp do C
```

(note that the guard is not negated as in the case of `repeat`.)

The `while` construct is simple to implement, given that it corresponds directly to a loop that is implemented on the physical machine using a conditional jump instruction. This simplicity of implementation should not deceive us about the power of this construct. A programming language which contains only assignment, conditional commands, and unbounded iteration is Turing complete. Our mini-language from Chap. 2 is therefore Turing complete (or: it allows the implementation of all computable functions). The same is not the case with bounded iteration which we will now turn to.

Bounded Iteration

Bounded iteration (sometimes also called numerically controlled iteration) is implemented by linguistic constructs that are more complex than those used for unbounded iteration; their semantics is also more elaborate. Moreover, only few programming languages have the “pure” forms of bounded iteration that we are going to describe. The model that we adopt in this discussion is that of ALGOL, which was then adopted by many other languages of the same family (but *not* by C or Java).

Bounded iteration is expressed using some variant of the `for` command. It can be described as:

```
for I = start to end by step do
  body
```

where `I` is a variable, called the *index*, or counter, or *control variable*; `start` and `end` are two expressions (for simplicity we can assume that they are of integer type and, in general, they must be of a discrete type); `step` is a (compile-time) non-zero integer constant; *body* is the command we want to repeat. This construct, in the “pure” form we are describing, is subject to the important static semantic constraint that the control variable can not be modified (either explicitly nor implicitly) during the execution of the body.

The semantics of the bounded iteration construct can be described informally as follows (assuming that `step` is positive):

1. The expression `start` is evaluated, as is `end`. The values are frozen and saved, so that the programmer cannot modify them. We denote them, respectively, as `start_save` and `end_save`.
2. `I` is initialised with the value of `start_save`.
3. If the value of `I` is strictly greater than the value of `end_save`, execution of the `for` command is terminated.
4. Execute *body* and increment `I` by the value of `step`.
5. Go to 3.

In the case in which `step` is negative, the test in step (3) determines whether `I` is strictly less than `end_save`.

It is worth emphasising the importance of step (1) above and the constraint that the control variable cannot be modified in the body. Their combined effect is to *determine*—before the loop begins execution—the number of times the body will be executed. This number is given by the quantity, *ic* (*iteration count*), which is defined as:

$$ic = \left\lfloor \frac{\text{end} - \text{start} + \text{step}}{\text{step}} \right\rfloor$$

if the right hand side is positive, otherwise *ic* is 0. It can be seen, finally, that there is no way of producing an infinite cycle with this construct.

There are considerable differences, both syntactic and semantic, between the versions of this construct in different languages. First, not all languages require non-modifiability of the control variable and/or the freezing of the control expressions. Strictly speaking, such cases do not implement bounded iteration because they are unable to compute *ic* once and once only. It is common, though, to continue speaking of bounded iteration even when the language does not guarantee determinateness (in this case it is the programmer who must enforce determinateness, by not modifying either directly or indirectly the control variable, the start, end, and step expressions). Also several other aspects constitute important differences between languages, of which we mention four:

Number of iterations According to the semantics which we have just given, when `step` is positive, if the value of `start` is initially (strictly) greater than the value of `end`, `body` is not executed at all. This is the case for most languages, although some of them execute the test in Step 3 only after having executed once `body`.

Step The requirement that `step` is a (non-zero) constant is necessary for statically determining its sign, so that the compiler can generate the appropriate code for the test in step 3. Some languages (such as Pascal and Ada) use a special syntax to indicate that `step` is negative, for example using `downto` or `reverse` in place of `to`. Other languages, such as some versions of Fortran, do not have a different syntax for the negative step. They instead implement `for` directly using the iteration counter rather than the test of `I` and `end`. The value `ic` is computed and if this value is positive, it is used to control the loop, decrementing it by 1 until it reaches the value 0. If, on the other hand, `ic` has a negative value or is equal to 0, the loop is never repeated. It is the use of this implementation technique that suggests the name numerically controlled iteration.

Final index value Another subtle aspect concerns the value of the control variable `I` after the end of the loop. In many languages, `I` is a variable that is also visible outside of the loop. What is then its value outside the body, immediately after the loop terminates? One natural approach could be to let `I` maintain the last value assigned to it during execution of the `for`. Following the semantics we described above, and assuming that the loop terminates normally and the `step` is positive, the last value assigned to the index `I` is the first value greater than `end`. This choice, however, can generate type ambiguities or errors. Let us assume, for example, that `I` is declared of an interval type `1 . . 10` (from 1 to 10). If we use a command:

```
for I = 1 to 10 by 1 do
    body
```

the final value assigned to `I` would be 11, which is not an admissible value in the interval `1 . . 10`. An analogous problem occurs for integer values when the calculation of `I` causes an overflow. To avoid this problem, some languages (for example Fortran IV or Pascal) consider the value of `I` to be indeterminate on termination of the loop (that is, the language definition does not specify what it should be). In other words, each implementation of these languages is allowed to behave how it wishes, with the imaginable consequence of the non-portability of programs. Other languages (for example: ALGOL W, ALGOL 68, Ada, and, in certain circumstances, C++) cut the matter short, decreeing that the control variable is a variable that is *local* to the `for`; hence it is not visible outside the loop. In this case the header of the `for` implicitly declares the control variable with a type that is determined from that one of `start` and `end`. Still another choice is the one made by Python, in which the control variable is visible outside the loop, with the value it had during the last meaningful iteration (in the example above, `I` would have value 10 after termination of the loop).

Jump into a loop The last point which merits attention concerns the possibility of jumping into the middle of a `for` loop using a `goto` command. Most languages forbid such jumps for clear semantic reasons, while there are fewer restrictions on the possibility of using a `goto` for jumping out of a loop.

We have just considered a number of important aspects of the implementation of `for` loops. Particular tricks can be used by the compiler to optimise the code that is produced (for example, eliminating tests which involve constants) or by limiting overflow situations which could occur when the incrementing the index i (by inserting appropriate tests).

Expressiveness of Bounded Iteration

Using bounded iteration, we can express the repetition of a command for n times, where n is an arbitrary value not known when the program is written, but is fixed when the iteration starts. It is clear that bounded iteration allows us to compute functions that cannot be expressed using only conditional commands and assignment. Despite this increase in expressive power, bounded iteration on its own is insufficient to make a programming language Turing complete. Think, for example, of a simple function f which can be defined as follows:

$$f(x) = \begin{cases} x & \text{if } x \text{ is even,} \\ \text{does not terminate} & \text{if } x \text{ is odd.} \end{cases}$$

Certainly such a function is computable. Every programmer knows how to implement it using a `while` or a `goto` or a recursive call to obtain a nonterminating computation. However such a function is not representable in a language only having assignment, sequential command, `if` and (true) bounded iteration. Indeed, in such a language, all programs terminate for every input. In other words, in such a language, only total functions can be defined, while function f is partial.¹⁴ In order to obtain a language that is Turing complete, it is necessary to include unbounded iteration. The easy translation into machine language of `while` should not deceive us. From a formal viewpoint, `while` is semantically more complicated than `for`. As we saw in Chap. 2, the semantics of the `while` command is defined in terms of itself, something which, if at first sight it appears a little strange, finds its formal justification in fixed-point techniques which are beyond the scope of this text (and to which we will mention in the section on recursion, below). Even if we have not formally defined the semantics of `for`, the reader can convince themselves that this can be given in simpler terms (see Exercise 4). The major complication in the semantics of `while` with respect to `for` corresponds to the greater expressiveness of the first construct. It is in fact clear that every `for` command can easily be translated into a `while`.

It can now be asked why a language should provide a bounded iteration construct when unbounded iteration constructs allow the same things to be done. The reply is, once again, of a pragmatic nature. The `for` is a much more compact form of iteration—putting the three components of the iteration (initialisation, control and increment of the control variable) on the same line makes understanding what the

¹⁴ There are also *total* functions that are not definable using only assignment, sequential composition, `if` and `for`. The canonical example is the Ackerman function, for whose definition the reader is referred to texts on computability theory for example [3].

loop does a lot easier. It can also prevent some common errors, such as forgetting the initialisation, or the increment, of the control variable. The use of a `for` instead of a `while` can therefore be an important way to improve understanding and, therefore testing and maintenance of a program. There is also an implementation reason in some languages and on some architectures. A `for` loop can often be compiled in a more efficient way (and, in particular, be optimised better) than a `while`, particularly as far as register allocation is concerned.

The for in C

In C (and in its successors, among them Java), the `for` is not a bounded iteration construct. The general version is:

```
for (exp1; exp2; exp3)  
    body
```

Its semantics is the following:

1. Evaluate `exp`₁;
2. Evaluate `exp`₂; if it is zero, terminate execution of the `for`;
3. Execute `body`;
4. Evaluate `exp`₃ and go to (2).

The abstract machine does not freeze the value of the control expressions, nor is there any ban on the possibility of modifying the value of index (which, in the general case, need not even exist). It is clear how the semantics expresses that in C, `for` is only an abbreviation for a `while`.¹⁵

Making use of the fact that, in C, a command is also an expression, we obtain the most usual form in which the `for` appears in C programs:

```
for (i = initial; i <= final; i += step){  
    body  
}
```

This is an abbreviation of:

```
i = initial;  
while (i <= final){  
    body  
    i += step;  
}
```

For-each

One of the most common iterative program schemata is the sequential traversal of a data structure. For a typical example, let us consider a function computing the sum of the elements of an integer array, in C:

¹⁵ An abbreviation which, putting in the same line initialisation, control and increment of the control variable, makes a program more readable, prevents some trivial mistakes, allows for easier optimizations, etc.

```

int sum(int [] A) {
    int acc = 0;
    for (int i=0; i<length(A); i++)
        acc += A[i];
    return acc;
}

```

This function is full of details that the compiler knows: the first and last index of *A*, the specific check for *i* reaching its limit. The more detail that has to be added to a construct, the easier it is to make an error, and the more difficult to understand at a glance what the construct does. In the case of *sum*, the essential is simple: execute the body of the iteration to *every element of A*. Indexes, step, control on the bound *length(A)*, are all details that the compiler can obtain elsewhere.

Hence, some languages use a construct, called *for-each*, to perform this kind of operation. The *for-each* has the following general syntax:

```

foreach (ControlVar : Expression) Command

```

The *for-each* construct expresses the repetition of *Command* (in which the *ControlVar* can clearly appear) on each element of *Expression*, which must evaluate to some form of sequence, or “*iterable*”.

The previous function can now be written as:

```

int sum(int [] A) {
    int acc = 0;
    foreach (int e : A)
        acc += e;
    return acc;
}

```

We read the header of the iteration as “for every element *e* in *A*”. The vector index, together with all of the vector’s limits, is hidden in a more synthetic and elegant construct.

The use of the *for-each* construct is not limited to vectors, but can be applied to all collections over which the notion of iteration can be defined in a natural way. In addition to enumerations and sets (which we will see in Chap. 8), let us mention the particularly important case of languages which allow the user to define types which are “*iterable*”.

Java (from version 5) supports the *for-each* construct. The keyword used is simply *for*¹⁶ but the syntax is different and allows the construct to be disambiguated without problem. In Java the *for-each* (also called *enhanced for* in the documentation) is applicable to all subtypes of the library type *Iterable*.

¹⁶ The *for-each* construct has been added on the fly when the language was already distributed and in use for some years. In a case like this, the modification of the set of reserved words is not a good design decision. Old programs which used the new keyword would stop working.

The case of Python is particularly interesting, since in this language for-each is primitive, and numerically controlled bounded iteration is a special case. A command

```
for name in exp:  
    body
```

evaluates first `exp` to a value `itval`, which must be an *iterable*—that is an object on which the predefined function `next` is defined (standard sequences like tuples or lists are iterables, as it is an iterable `range(start, end, step)` which can be used to perform a numerically controlled iteration). A reference to `itval` is stored away, so that any further modification to `exp` does not change the iterable controlling the iteration. The body is then repeatedly executed: at the beginning of each evaluation of the body, to `name` is assigned the value obtained by a call to `next(itval)`, thus neutralizing possible modifications to the value of `name` that could occur inside `body`. When the iterable is “exhausted” (all values have been generated), iteration terminates, and `name` maintains the last meaningful value produced by `next(itval)`. The reader will recognize in this steps a close analogue of the semantics we detailed at the beginning of our treatment of bounded iteration. If Python iterable were always immutable, this would be a true bounded iteration (hence no divergent for loops). However, Python has mutable iterables—for instance, lists. The semantics we outlined guarantees only that the identity of `itval` is not modified during iteration. If `itval` is a mutable value (a container of some sort), we may modify its value and concoct infinite for loops.

6.4 Structured Programming

The rejection of the `goto` command during the 1970s was not an isolated phenomenon. Indeed, it was only one issue among many that contributed to a much wider debate which brought so-called *structured programming* to the fore. This can be considered as the antecedent of modern programming methodology. As the name itself suggests, it consists of a series of prescriptions aimed at allowing the development of software with a certain structure in code and, correspondingly, in the flow of control. These prescriptions have both a methodological nature, providing precise development methods for programs, and a linguistic component, suggesting that languages should prefer only some typologies for the commands used (in substance, all those seen here so far, with the exception of `goto`). Let us see in more detail some salient points about structured programming and its associated linguistic implications.

Top-down or hierarchical design of programs The program is developed by successive refinements, starting from a first (fairly abstract) specification adding successively extra detail at each step.

Code modularisation In a program, commands implementing to a specific feature should all be grouped together. To do this, all the linguistic mechanisms made available by the language are used. These range from compound commands to constructs for abstraction, such as procedures, functions, and modules, where the language supports them.

Use of meaningful names The use of meaningful names for variables, as well as for procedures, etc., greatly simplifies the process of understanding the code and therefore eases making any changes required during maintenance. Even if this appears (and is) obvious, in practice it is too often ignored.

Extensive use of comments Comments are essential for understanding, testing, verification, correction and modification of code. A program with no comments, becomes rapidly incomprehensible, once it has reached a certain length.

Use of structured data types The use of appropriate datatypes, for example records, to group and structure information, even if it is of an heterogeneous type, eases both the design of code and its later maintenance. For example, if we can use the single variable, of type *student record*, to store the information about family name, registration number and a subscription year for a student, then the structure of the program will be much clearer than it would be if one had to use four different variables to hold the information about a single student.

Use of structured control constructs This, from a linguistic viewpoint, is the essential aspect. To implement structured programming, it is necessary to use structured control constructs, which can be characterized as those having a single entry and a single exit point.

The last point is the one which interests us the most and merits extra study. The essential idea behind structured control constructs is that, by having a single entry and a single exit point, they allow structuring of the code in which the linear scanning of the program text corresponds to execution flow. If command C2 textually follows command C1, at the (unique) exit of command C1, when C1 terminates, control passes to the (unique) entry point of command C2. Each command internally can have complex structure: branching (as in an *if*) or loops (as in *for*) with a non-linear control structure or internal jumps. The important thing is that each elementary component, externally, is visible in terms only of an entry point and an exit. This property, which is fundamental for the understanding of code, is violated in the presence of a command such as *goto* which allows jumps forward and backwards in the program. In such a case, the code can rapidly reach a state which is called “spaghetti code”, where the control flow between the various program components, instead of being a simple graph with few edges (which connect the output of a command to the input of the following one), is described by a graph in which the edges resemble a plate of spaghetti.

The control constructs seen so far, except the *goto*, are all structured and are the ones left in the modern programming languages. From a theoretical position, they allow programs for all computable functions to be written, as we have already

observed. From the pragmatic point of view they are sufficient to express all types of control flow present in real applications. In particular, the constructs that were discussed at the end of Sect. 6.3.1 enable us to handle those cases in which we have to exit from a loop, a procedure, or, in some way, interrupt processing before “normal” termination occurs. All of these cases could be handled in a natural manner using a `goto`. For example, if we wished to process all the elements of a file that we have read from an external device, we could use code of the form¹⁷:

```
while true do {  
  read (X);  
  if X = end_of_file then goto end;  
  elaborate (X);  
}  
end: ...
```

It can be observed that this use of `goto` does not violate the single entry, single exit principle because the jump only anticipates the exit, which happens at a single point in the overall construct. The structured command `break` (or its analogues) is the canonical form of this “jump to end of loop”. When written in place of `goto end`, it makes the program clearer, and omits the label (the destination of the jump implicit in the `break` is the unique exit from the construct).

Let us finally recall that structured programming constituted a first reply to the demands of so-called programming in the large,¹⁸ given that it requires the decomposition of a system of vast dimensions into different components, each of which is assigned a certain level of independence. The amount of independence depends on the abstraction mechanisms being used. For example, if procedures are used, communication between the various components can happen only through parameters. More significant answers to the needs of programming in the large cannot, though, be given solely at the linguistic level of programming languages. Software engineering has studied many methodologies for managing projects and implementing big software systems. Some of these methodologies also have linguistic implications, which, however, cannot be completely accounted for in this book. The object-oriented paradigm, together with some specification formalisms for object-oriented projects (like UML), are some of the replies to issues we consider in this text.

¹⁷ Wishing to avoid `goto` and using only `while` and `if`, we find that we have to write code that is much less natural.

¹⁸ This term denotes the implementation of large-scale software systems.

Inductive Definitions

Using an axiomatic presentation due to Giuseppe Peano, the natural numbers (non-negative integers) $0, 1, 2, 3, \dots$, can be defined as the least set X , satisfying the following rules:

1. $0 \in X$
2. If $n \in X$, then $n + 1 \in X$,

where we assume as primitive the concept of 0 (zero), that of number (denoted by n) and that of successor of n (written $n + 1$). This definition of the naturals clearly provides an intuitive justification of the principle of induction, a fundamental tool in mathematics and also, in some ways, in computer science. This principle can be stated as follows. To prove that a property, $P(n)$, is true for all natural numbers, it is sufficient to show that the following two conditions are satisfied:

1. $P(0)$ is true;
2. For every natural, n , if $P(n)$ is true, then so is $P(n + 1)$.

In addition to the proof of properties, induction is a powerful tool for the definition of functions. In fact, it can be shown that if $g : (\mathbb{N} \times X) \rightarrow X$ is a total function, $\mathbb{N}$ denotes natural numbers and a is an element of X , then there exists a unique (total) function $f : \mathbb{N} \rightarrow X$, such that:

1. $f(0) = a$
2. $f(a + 1) = g(n, f(n))$

Such a pair of equations then provide an *inductive definition* of the function f .

What has so far been said about induction over the naturals can be generalised to arbitrary sets over which a well-founded ordering relation $<$ is defined, i.e. a relation which does not admit infinite descending chains $\dots x_m < \dots < x_1 < x_0$. In this case, the induction principle, called well-founded induction, can be expressed as follows. Let $<$ be a well-founded relation defined on a set A . To show that $P(a)$ has a value for every a belonging to A , it is sufficient to show the following implication:

For all $a \in A$, if $P(b)$ is true for every $b < a$, then $P(a)$ is true.

6.5 Recursion

Recursion is another mechanism, alternative to iteration, for obtaining Turing-equivalent programming languages. In empirical terms, a subprogram is recursive if its body contains a call to itself. One can have indirect recursion as well (it is best called *mutual recursion*) when a procedure P calls another procedure, Q , which, in its turn, calls P . In Chap. 5, we have already seen the example of the recursive function `fib` which compute the n th value of the Fibonacci function.

```

int fib (int n){
    if (n == 0)
        return 1;
    else
        if (n == 1)
            return 1;
        else
            return fib(n-1) + fib(n-2);
}

```

The fact that a function like `fib` is defined in terms of itself could raise some doubts about the mathematical nature of the function being defined. In reality, recursive definitions are fairly common in mathematics, where are also called inductive definitions. As the box on “inductive definitions” shows in more detail, the idea is that of describing the result of the application of a function f to an argument X in terms of the application of f itself to arguments which are “smaller” than X . The domain on which f is defined must be such that it does not allow infinite chains of successively smaller elements, thus ensuring that, after a finite number of applications of the function f , we arrive at a terminal case—directly defined. At this point, we can reconstruct the value of f applied to X . For example, recalling that the factorial of a natural number, n , is given by the product $1 \cdot 2 \cdots n - 1 \cdot n$, we can inductively define the function computing the factorial as follows:

$$\begin{aligned} factorial(0) &= 1, \\ factorial(n+1) &= (n+1) \cdot factorial(n). \end{aligned}$$

Here n is an arbitrary natural number. In an analogous fashion, we can define the function which computes the n th term in the Fibonacci series, for which we have first provided a recursive program.

If, then, inductive definitions in mathematics and recursive functions in programming languages are similar, there is still a fundamental difference. In the case of inductive definitions, not all possible definitions of function in terms of itself will work. If we write:

$$\begin{aligned} foo(0) &= 1, \\ foo(n) &= foo(n) + 1 \quad \text{for } n > 0 \end{aligned}$$

there is no total function over the naturals that satisfies these equalities. Hence they do not define a function, contrary to the equalities for the factorial function. If, on the other hand, we write:

$$fie(1) = fie(1)$$

the problem is now the opposite. Many functions satisfy these equalities, so once again this does not constitute a valid definition.

On the other hand it is perfectly legitimate to write the following function in a programming language supporting recursion:

```
int foo1 (int n){
    if (n == 0)
        return 1;
    else
        return foo1(n) + 1;
}
```

And also:

```
int fie1 (int n){
    if (n == 1) return fie1(1);
}
```

These are functions which, in some case, do not terminate (when $n > 0$ for `foo1(n)` and for $n = 1$ in the case of `fie1(n)`), but from the semantic viewpoint there is no problem because, as we saw in Chap. 3, programs define partial functions.

6.5.1 Tail Recursion

In Chap. 5, we saw how the presence of recursion in a programming language makes it necessary to include dynamic memory management since it is not possible to statically determine the maximum number of instances of a single function that will be active at the same time (and, therefore, to determine the maximum number of activation records required). For example, we have seen that for the call of our Fibonacci function `fib(n)`, we need as many activation records as the n th Fibonacci number, a value that cannot be known as compilation time. However, in some cases we can avoid the allocation of new activation records for successive calls of the same function, since we can reuse the same memory space. To understand this point, let us look at two recursive functions for the factorial. The first is the usual one:

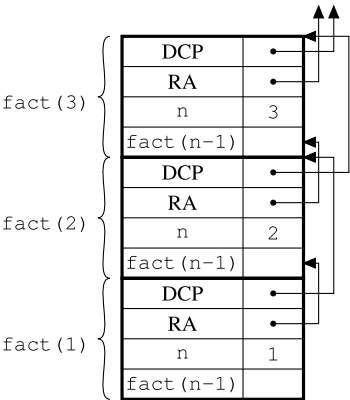
```
int fact (int n){
    if (n <= 1) return 1;
    else
        return n * fact(n-1);
}
```

The activation record for a call `fact(n)` is shown in a slightly simplified form in Fig. 6.6. The field *n* contains the value of the actual parameter to the procedure; the field *Intermediate Result* will contain the intermediate result produced by the evaluation of `fact(n-1)`; the field *Result address*, finally, contains the address of the memory area in which the result must be returned (that is, the address of the field *Intermediate Result* in the activation record of the caller, for all calls but the first). It is important to note that the value of the *Intermediate Result* field present in the activation record of `fact(n)`, can be determined only when the recursive call to `fact(n-1)` terminates. The value of this field is used in the computation of $n * \text{fact}(n-1)$, to obtain the value of `fact(n)`. In other words, when we are inside the call `fact(n)`, before it can terminate, we must know the value of `fact(n-1)`; in its turn, for the call to `fact(n-1)` to terminate we have to

Fig.6.6 Activation record for the fact function

Dynamic Chain Pointer (DCP)
Result Address (RA)
n
Intermediate Result: fact (n-1)

Fig.6.7 Activation record stack and for the call fib(3) and the two recursive calls fact(2) and fact(3)



know the value of $\text{fact}(n-2)$ and so on, recursively, right back to the terminal case $\text{fact}(1)$. Therefore all activation records for the recursive calls $\text{fact}(n)$, $\text{fact}(n-1), \dots, \text{fact}(1)$, must be at the same time on the stack, in distinct memory areas.

Figure 6.7 shows the stack of activation records created by the call to $\text{fact}(3)$, as well as the by the following calls to $\text{fact}(2)$ and $\text{fact}(1)$. When we reach the final case, the call to $\text{fact}(1)$, terminates immediately and returns the value 1, which, using the pointer contained in the *Result Address* of the activation record for $\text{fact}(1)$, will be returned to the *Intermediate result* field of the activation record for $\text{fact}(2)$, as is shown in Fig. 6.8. At this point, too, the call to $\text{fact}(2)$ can terminate, returning the value $2 \cdot \text{fact}(1) = 2 \cdot 1$ to the call $\text{fact}(3)$, as shown in Fig. 6.9. Finally, too, the call to $\text{fact}(3)$ will terminate, returning to the calling program the value $3 \cdot \text{fact}(2) = 3 \cdot 2 = 6$.

Consider now another function:

```
int factrc (int n, int res){
    if (n <= 1)
        return res;
    else
        return factrc(n-1, n * res)
}
```

This, if called with $\text{factrc}(n, 1)$ also returns the factorial of n .

Fig.6.8 Stack of activation records after termination of the call `fact(2)`

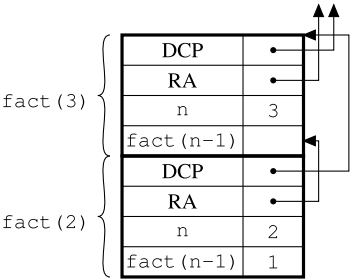
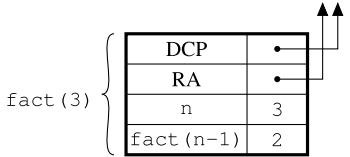


Fig.6.9 Activation record stack after the termination of the call `fact(2)`



In this case, also, the initial call `factrc(n,1)` produces $n - 1$ recursive calls `factrc(n-1,n*1)`, `factrc(n-2,(n-1)*n*1)`, ..., `factrc(1,2*...*(n-1)*n*1)`.

However, let us now observe that for $n > 1$, the value returned by the generic call `factrc(n,res)` is *exactly the same* as the value returned by the recursive call `factrc(n-1,n*res)` without there being any additional computation. The value finally returned from the initial call `factrc(n,1)` is therefore the same as that returned by the last recursive call `factrc(1,2*...*(n-1)*n*1)` (and is therefore $n \cdot (n - 1) \cdot (n - 2) \cdots 1$), without there being any requirement to “climb back up” the call chain using the intermediate results to compute the final value, as it happens, instead, in the case of `fact`.

From what has been said, it appears clear that, once `factrc(n,res)` has recursively called `factrc(n-1,n*res)`, there is no need to maintain the information in the activation record for the call `factrc(n,res)`, given that all the information necessary to perform the calculation of the final result is passed to `factrc(n-1,n*res)`. This means that the activation record for the recursive call `factrc(n-1,n*res)` can simply reuse the memory space allocated to the activation record for `factrc(n,res)`. Same considerations can be made for all the successive calls. In short, the function `factrc` needs a single memory area, to allocate a single activation record, independent of the number of recursive calls to be made. We have therefore obtained a recursive function for which memory can be allocated statically!

Recursion of the kind used in the function `factrc` is said to be *tail recursion*, since the recursive call is, so to speak, the last thing that happens in the body of the procedure. After the recursive call, no other computation is performed. We can give the following definition.

Definition 6.3 (*Tail recursion*) Let f be a function definition which, in its body, contains a call to a function g (different from f or equal to f). The call of g is said to be a *tail call* if the function f returns the value returned by g without having to perform any other computation. We say that a function definition for f is *tail recursive* if all the recursive calls present in f are tail calls.

For example, in the function:

```
int f (int n){
    if (n == 0)
        return 1;
    else
        if (n == 1)
            return f(0);
        else
            if (n == 2)
                return f(n-1);
            else
                return f(1)*2;
}
```

The first two recursive calls are tail calls, the third is not a tail call. Therefore the function f is not tail recursive.

Our interest in tail recursion lies in the possibility of implementing it using a single activation record and therefore using constant memory space. Our investigation of the `factrc` function, in fact, is completely general, and does not depend on the specific form of this function; it depends only on the fact that we are dealing with a tail recursive function. All of this, however, does not hold in the case in which we also consider higher-order functions (i.e. when there are functions that are passed as parameters), as we will see later in this section.

In general, it is always possible to transform a function definition which is not tail recursive into an equivalent tail recursive one, by complicating the definition. The idea is that all the computations which have to be made after the recursive call (and make the function non-tail recursive) should, as far as possible, be performed before the call. The part of the work which cannot be done before the recursive call (because, for example, it uses its results) is “passed” with appropriate additional parameters, to the recursive call itself. This technique is the same as the one we used in the case of the tail-recursive function `factrc`, where, instead of recursively calculating the product $n \cdot \text{fact}(n-1)$ in the body of the call to `fact(n-1)`, we have added a parameter `res` which allows us to pass to the product $n \cdot (n-1) \cdot (n-2) \cdots j$ to the generic recursive call `factrc(j-1, j*res)`. Therefore, in this case, the calculation of the factorial is performed incrementally by the successive recursive calls, in a way analogous to that performed by an *iterative* function such as the following:


```
int fact-it (int n, int res){
    res=1;
    for (i=n; i>=1; i--)
        res = res*i;
}
```

In an analogous fashion to this, we can also transform the `fib` function into a function with tail recursion `fibrc`, by addition of two additional parameters:

```
int fibrc (int n, int res1, int res2){
    if (n == 0)
        return res2;
    else
        if (n == 1)
            return res2;
        else
            return fibrc(n-1, res2, res1+res2);
}
```

The call `fibrc(n, 1, 1)` returns the n th value in the Fibonacci series. Clearly, both in the case of `fibrc` and in that of `factrc`, if we want to make the additional parameters invisible, we can encapsulate the functions inside others which have only the parameter n . For example, in the scope of the declaration of `fibrc`, we can define:

```
int fibrctrue (int n){
    return fibrc(n, 1, 1);
}
```

The transformation of one function into an equivalent one with tail recursion can be done automatically using a technique called *continuation passing style*, which basically consists of representing, at all points in a program, that part of the program that remain using a function called a *continuation*. If we want to convert a function into a tail-recursive one, it suffices to use a continuation to represent everything that remains of the computation, and pass this continuation to the recursive call. This technique, however, does not always produce functions that can be executed with constant memory requirements. The continuation, being a function, could contain the variable that should be evaluated in the environment of the caller, thus requiring the presence of the caller's activation record.

6.5.2 Recursion or Iteration?

Without going into detail on theoretical results, let us recall that recursion and iteration (in its most general form) are alternative methods for achieving the same expressive power. The use of the one or the other is often due to the nature of the problem. For the elaboration of data using rigid structures (matrices, tables, etc.), as normally happens in numerical or data processing applications, it is often easier to use iterative constructs. When, on the other hand, we need to process symbolic structures which

naturally lend themselves to a recursive definition (for example, lists, trees, etc.), it is often more natural to use recursion.

Recursion is sometimes considered much less efficient than iteration and therefore declarative languages are thought much less efficient than imperative ones. The argument presented above about tail recursion shows that recursion is not necessarily less efficient than iteration, both in terms of memory occupation and in terms of execution time. Certainly, naive implementations of recursive functions, such as those often resulting from the direct translation of inductive definitions, can be fairly inefficient. This is the case, for example, for our procedure `fib(n)` which has execution time and memory occupation that are exponential in n . However, as was seen, using recursive functions that are more “astute”, such as those with tail recursion, we can obtain performance similar to that of the corresponding iterative program. The function `fibrc` in fact uses constant space and runs in time linear in n .

6.6 Summary

In this chapter, we have analysed the components of high-level languages relating to the control of execution flow in programs. We first considered expressions and we have analysed:

- The types of syntax most used for their description (as trees, or in prefix, infix and postfix linear form) and the related evaluation rules.
- Precedence and associativity rules required for infix notation.
- The problems generally related to the order of evaluation of the subexpressions of an expression. For the semantics of expressions to be precisely defined, this order must be precisely defined.
- Technical details on the evaluation (short-circuit, or lazy evaluation) used by some languages and how they can be considered when defining the correct value of an expression.

We then passed to commands, seeing:

- Assignment. This is the basic command in imperative languages. Even though it is fairly simple, we had to clarify the notion of variable in order to understand its semantics.
- Various commands which allow the management of control (conditionals and iteration) in a structured fashion.
- The principles of structured programming, stopping to consider age-old questions about the `goto` command.

The last section, finally, dealt with recursion, a method that stands as an alternative to iteration for expressing algorithms. We concentrated on tail recursion, a form of recursion that is particularly efficient both in space and time. This must clear up the

claim that recursion is a programming method that is necessarily less efficient than iteration.

In the various boxes, we examined a historic theme that has been important in the development of modern programming languages, as well as a semantic issue that precisely clarifies the difference that exists between imperative, functional and logic programs.

We still have to deal with important matters concerning control abstraction (procedures, parameters and exceptions) but this will be the subject of the next chapter.

6.7 Bibliographical Notes

Many texts provide an overview of the various constructs present in programming languages. Among these, the most complete are [2,4].

Two historical papers on the `goto` question are [5] (in which Böhm and Jacopini's theorem is proved) and Dijkstra's [6] (where the “dangerousness” of the jump command is discussed).

An interesting paper, even though not for immediate reading, which delves into themes relating to inductive definitions is [7]. For a gentler introduction to recursion and induction, see [8].

According to Abelson and Sussman [9], the term “syntactic sugar” is due to Peter Landin, a pioneer in Computer Science who made fundamental contributions in the area of programming language design.

6.8 Exercises

1. Define, in any programming language, a function, f , such that the evaluation of the expression $(a + f(b)) * (c + f(b))$ when performed from left-to-right has a result that differs from that obtained by evaluating right-to-left.
2. Show how the `if then else` construct can be used to simulate short-circuit evaluation of boolean expressions in a language which evaluates all operands before applying boolean operators.
3. Consider the following case command:

```

case Exp of
  1:      C1;
  2, 3:   C2;
  4..6:   C3;
  7:      C4
else:   C5

```

Provide an efficient pseudocode assembly program that corresponds to the translation of this command.

4. Define the operational semantics of the command

```
for I = start to end by step do body
```

using the techniques introduced in Chap. 3. Suggestion: using values of the expressions, start, end and step, the following can be computed before executing the for: the number, ic , of repetitions which must be performed (it is assumed, as already stated in the chapter, that the possible modification of I, start, end and step in the body of the for have no effect upon their evaluation). Once this value, n , has been computed, the for can be replaced by a sequence of ic commands.

5. Consider the following function:

```
int ninetyone (int x){
    if (x>100)
        return x-10;
    else
        return ninetyone(ninetyone(x+11));
}
```

Is this tail recursive? Justify your answer.

6. The following code fragment is written in a pseudo-language which admits bounded iteration (numerically controlled) expressed using the for construct.

```
z=1;
for i=1 to 5+z by 1 do{
    write(i);
    z++;
}
write(z);
```

What is printed by write?

7. Say what the following code block prints. This code is written in a language with static scope and call by name. The definition of the language contains the following phrase: “The evaluation of the expression $E_1 \circ E_2$ where $\circ$ is any operator, consists of (i) the evaluation of E_1 ; (ii) then the evaluation of E_2 ; (iii) finally, the application of the operator $\circ$ to the two values previously obtained.”

```
{int x=5;
  int P(name int m) {
      int x=2;
      return m+x;
  }
  write(P(x++) + x);
}
```

References

1. B. Liskov, J. Guttag, *Abstraction and Specification in Program Development*. MIT Electrical Engineering and Computer Science Series (MIT Press, Cambridge, MA, 1986)
2. M.L. Scott, *Programming Language Pragmatics* (Morgan Kaufmann Publishers, Burlington, MA, 2000)
3. H. Rogers, *Theory of Recursive Functions and Effective Computability* (McGraw Hill, New York, 1967). Reprinted by MIT Press
4. T.W. Pratt, M.V. Zelkowitz. *Programming Languages: Design and Implementation*, 4th edn. (Pearson, London, 2000)
5. C. Böhm, G. Jacopini, Flow diagrams, turing machines and languages with only two formation rules. *Commun. ACM* **9**(5), 366–371 (1966)
6. E.W. Dijkstra, Go to statement considered harmful. *Commun. ACM* **11**(3), 147–148 (1968)
7. P. Aczel, An introduction to inductive definitions, in *Handbook of Mathematical Logic*. ed. by J. Barwise (North-Holland, Amsterdam, 1977), pp.739–782
8. G. Winskel, *The Formal Semantics of Programming Languages* (MIT Press, 1993)
9. H. Abelson, G.J. Sussman, *Structure and Interpretation of Computer Programs* (MIT Press, Cambridge, MA, 1996)

Control Abstraction

7

The concept of abstraction is a recurrent theme in this text. Right from the first chapter, we have encountered *abstract* machines and their hierarchies. For them, we used the terms “abstract” as opposed to “physical”, denoting by abstract machine, a set of algorithms and data structures not directly present in any specific physical machine, but executable on it by means of interpretation. Moreover, the fact that the concept of abstract machine, to some extent, hides the underlying machine is fundamental.

“To abstract” means to hide something. Often, abstracting from some concrete data relating to some object, one succeeds in bringing out with more clarity a concept common to that object. Each description of a phenomenon (natural, artificial, physical, etc.) is not based on the set of *all* data relating to the phenomenon itself, otherwise it would be like a 1:1 geographical map, extremely precise but useless. Every scientific discipline describes a certain phenomenon, concentrating only on some aspects, those which have been found to be the most relevant to the agreed aims. It is for this reason that scientific language uses appropriate mechanisms to express these “abstractions”. Programming languages, themselves the abstractions of the physical machine, are no exception. Rather, expressiveness depends in an essential way on the mechanisms of abstraction which the languages provide. These mechanisms are the principal instruments available to the designer and programmer for describing in an accurate, but also simple and suggestive, way the complexity of the problems to be solved.

In a programming language, in general, two classes of abstraction mechanisms are distinguished. Those that provide *control abstraction* and those that provide *data abstraction*. The former provide the programmer the ability to hide procedural data; the latter allow the definition and use of sophisticated data types without referring to how such types will be implemented. In this chapter we will be concerned with the mechanisms for control abstraction, while data abstraction will be the subject of Chap. 9, after we have seen mechanisms for data structuring in the next chapter.

Fig. 7.1 Definition and use of a function

```
int foo (int n, int a) {
    int tmp=a;
    if (tmp==0) return n;
    else return n+1;
}
...
int x;
x = foo(3,0);
x = foo(x+1,1);
```

7.1 Subprograms

Every program of any complexity is composed of many components, each of which serves to provide part of a global solution. The decomposition of a problem into subproblems allows better management of complexity. A more restrictive problem is easier to solve. The solution to the global problem is obtained by appropriate composition of the solutions to these subproblems.

In order for all of this to be efficient, however, it is necessary that the programming language provides linguistic support which facilitates and makes possible such subdivisions and, therefore, re-composition. This linguistic support allows the expression of decomposition and re-composition directly in the language, transforming these concepts from simple methodological suggestions into real instruments for design and programming.

The key concept provided by all modern languages is that of the subprogram, procedure, or function.¹ A *function* is a piece of code identified by name, is given a local environment of its own, and is able to exchange information with the rest of the code using *parameters*. This concept translates into two different linguistic mechanisms—the *definition* (or declaration) of function, and its *use* (or call). In Fig. 7.1, the first five lines are the definition of the function named `foo`, whose local environment is composed from three names `n`, `a` and `tmp`.² The first line is the *header*, while the remaining lines constitute the *body* of the function. The last two lines of Fig. 7.1 are the uses (or calls) of `foo`.

A function exchanges information with the rest of the program using three principal mechanisms: parameters, return value, nonlocal environment.

¹ These three terms assume different meanings in different languages. For example, “subprogram” is usually the most general term. In languages of the ALGOL family and their descendants, a procedure is a subprogram which modifies the state, while a function is a subprogram that returns a value. In this chapter we will use the three terms as synonyms.

² The name `foo` is part of the nonlocal environment of the function.

Fig. 7.2 Renaming of formal parameters

```
int foo (int m, int b){  
    int tmp=b;  
    if (tmp==0) return m;  
    else return m+1;  
}
```

Parameters

We distinguish between *formal parameters*, which appear in the definition of a function, and *actual parameters*, which appear, instead, in the call. The formal parameters are always names which, as far as the environment is concerned, behave as declarations local to the function itself. They behave, in particular, as *bound variables*, in the sense that their consistent renaming has no effect on the semantics of the function. For example, the function `foo` in Fig. 7.1 and that in Fig. 7.2 are indistinguishable, even though the second has different names for its formal parameters.

The number and type of actual and formal parameters must, in general, coincide, although many type compatibility rules can be brought into play (see Sect. 8.7). There is sometimes also the possibility of declaring functions with a variable number of parameters.

Return Value

Some functions exchange information with the rest of the program by returning a value as a result of the function itself, as well as through the use of parameters. Our function, `foo`, for example, returns an integer. The language makes available, in this case, the mechanism which allows the “return of value” to be expressed (for example the `return` construct which has also the effect of terminating the execution of the current function). In some languages, the name “function” is reserved for subprograms which return a value, while those subprograms which interact with the caller just via parameters or the non-local environment are called “procedures”.

In languages which derive their syntax from C, all subprograms are, linguistically, functions. If the result type of a function is `void`, the function returns no meaningful value (the command to return such a value and to terminate execution is `return`).

Nonlocal Environment

This is a less sophisticated mechanism with which a function can exchange information with the rest of the program. If the body of the function modifies a nonlocal variable, it is clear that such a modification is felt in all parts of the program where this variable is visible.

Static Variables

We have always assumed that the local environment of a function has the same lifetime as the function itself. In such a case, there is no primitive mechanism that one instance of a function can use for communicating information to another instance of the *same* function. The only way for this to happen is through the use of a nonlocal variable.

In some languages, however, it is possible to arrange for a variable (which must be local to a function) to retain its value between one call to the function and another. In C, for example, this is achieved using the `static` modifier (Fortran uses `save`, ALGOL `own`, etc.). A `static` variable allows programmers to write functions with memory. The following function, for example, returns the number of times it has been called:

```
int how_many_times() {
    static int count;
    /* C guarantees that static variables are
       initialised to zero by the first
       activation of the function */
    return count++;
}
```

The declaration of a `static` variable introduces an association with unlimited lifetime into the environment (in the lifetime of the program, clearly). It should be clear that a static variable cannot be memorized in the activation record created for each function call and is, instead, stored together with the global variables.

A `static` variable provides greater data abstraction than a global variable, since it is not visible from outside the function. The visibility mechanisms guarantee, therefore, that it is only modifiable inside the function body.

7.1.1 Functional Abstraction

From a pragmatic viewpoint, subprograms are mechanisms which allow the software designer to obtain *functional abstraction*. A software component is an entity which provides services to its environment. The clients of such a component are not interested in *how* the services are delivered, only in how to use them. The possibility of associating a function with each component allows the separation of what the client needs to know (expressed in the header of the function: its name, its parameters, its result type, if present) from what it does not need to know (which is in the body). We have a true functional abstraction when the client does not depend on the body of a function, but only on the header of it. In this case, the substitution (e.g. for efficiency reasons) of the body by another with the same semantics is fully transparent to the system software. When a system is based on functional abstraction, the three acts of specification, implementation and implementation and use of a function can be performed independently of each other without knowledge of the context in which the other actions are performed.

Functional abstraction is a methodological principle, whose functions provide linguistic support. It is clear that this is not a definitive support. It is necessary for the programmer to use these functions correctly, for example by limiting the interactions

between function and caller only to parameter passing, because the use of the non-local environment to exchange information between caller and callee would destroy functional abstraction.

7.1.2 Parameter Passing

The mode in which actual parameters are paired with formal parameters, and the semantics which results from this, is called the *parameter passing discipline*. According to what is now traditional terminology, a specific mode is composed of the kind of communication that it supports, together with the implementation that produces this form of communication. The mode is fixed when the function is defined and can be different for each parameter; it is fixed for all calls of the function.

From a strictly semantic viewpoint, the classification of the type of communication permitted by a parameter is simple. From a subprogram's viewpoint, three parameter classes can be discerned:

- Input parameters.
- Output parameters.
- Input/output parameters.

A parameter is of input type if it allows communication which is only in the direction from the caller to the function (the “callee”). It is of output type if it permits communication only in direction from the callee to the caller. Finally, it is input/output when it permits bidirectional communication.

Note that this is a linguistic classification, part of the definition of the language; it is not derived from the use to which parameters are put. An input/output parameter remains that way even if it is used only in a unidirectional fashion (e.g., from caller to callee).

Each of these types of communications can be obtained in different ways. The specific implementation technique constitutes the “call mode”, which we will now subject to analysis, describing for each mode:

- What type of communication it allows.
- What form of actual parameter is permitted.
- The semantics of the mode.
- The canonical implementation.
- Its cost.

Of the modes that we will discuss, the first one (by value) is the most important and, by far, the most used. Also the mode by reference is relevant. Most of the others are variations on the theme of call by value. An exception is call by name, which we will discuss last and will allow us to present a simple case of what it means to “pass an environment” to a procedure.

Fig. 7.3 Passing by value

```

int y = 1;
void foo (int x) {
    x = x+1;
}
...
y = 1;
foo(y+1);
// here y = 1

```

Call by Value

Call by value is a mode that corresponds to an input parameter. The local environment of the procedure is extended with an association between the formal parameter and a new variable. The actual parameter can be an expression. When called, the actual parameter is evaluated and its r-value obtained and associated with the formal parameter. On termination of the procedure, the formal parameter is destroyed, as is the local environment of the procedure itself. During the execution of the body, there is no link between the formal and the actual parameter. There is no way of make use of a value parameter to transfer information from the callee to the caller.

Figure 7.3 shows a simple example of passing by value. Like in C, C++, Pascal and Java, when we do not explicitly indicate any parameter-passing mode for a formal parameter, it is to be understood that parameter is to be passed by value. The variable `y` never changes its value (it always remains 1). During the execution of `foo`, `x` assumes the initial value 2 by the effect of passing the parameter. It is then incremented to 3, finally it is destroyed with the entire activation record for `foo`.

If we assume a stack-based allocation scheme, the formal parameter corresponds to a location in the procedure's activation record in which the value of the actual parameter is stored during the calling sequence of the procedure.

Let us note how this is an expensive mode when the value parameter is bound to a large data structure. In such a case, the entire structure is copied to the formal.³ On the other hand, the cost of accessing the formal parameter is minimal, since it is the same as the cost of accessing a local variable in the body.

Passing by value is a simple mechanism with clear semantics. It is the default mechanism in many languages and is the only way to pass parameters in C, Java, and Python (but see the box on “Call by object reference”, below).

Call by Reference

Call by reference (also called *by variable*) implements a mechanism in which the parameter can be used for both input and output. The actual parameter *must* be an expression with l-value (recall the definition of l-value on Sect. 6.2.2). At the point of call, the l-value of the actual parameter is evaluated and the procedure's local environment is extended with an association between the formal parameter

³ The reader who knows C should not be misled. In a language with pointers, as we will discuss below, often the passing of a complex structure consists of passing (by value) a pointer to the data structure. In such a case, it is the pointer that is copied, not the data structure.

Call by Reference in C

C admits only call by value but also allows the manipulation of pointers and addresses. Making use of this fact, it is not difficult to simulate call by reference. Let us consider the problem of writing a simple function which swaps the values of two integer variables which are passed to the function as parameters. With only call by value, there is no way to do this. We can, though, combine call by value with pointer manipulation, as in the following example:

```
void swap (int *a, int *b) {
    int tmp = *a; *a=*b; *b=tmp;
}
int v1 = ...;
int v2 = ...;
swap (&v1, &v2);
```

The formal parameters to `swap` (both by value) are of type pointer to integer (`int *`). The values of the actual parameters are the addresses (that is, the l-value) of `v1` and `v2` (obtained using the operator `&`). In the body of `swap`, the use of the `*` operator performs dereferencing of the parameters. For example, we can paraphrase the meaning of `*a = *b` as: take the value contained in the location whose address is contained in `b` and store it in the location whose address is stored in `a`. Our `swap` therefore simulates call-by-reference.

Fig.7.4 Passage by reference

```
int y = 0;
void foo (reference int x) {
    x = x+1;
}
y=0;
foo(y);
// here y = 1
```

and the actual parameter's l-value (therefore creating an *aliasing* situation). The most common case is that in which the actual parameter is a variable. In this case, the formal and the actual are two names for the same variable. At the end of the procedure, the connection between the formal parameter and the actual parameter's l-value is destroyed, as is the environment local to the procedure. It is clear that call by reference allows bidirectional communication: each modification of the formal parameter is a modification of the actual parameter.

Figure 7.4 shows a simple example of call by reference (which we have notated in the pseudocode with the `reference` modifier). During the execution of `foo`, `x` is a name for `y`. Incrementing `x` in the body is, to all effects, the incrementing of `y`. After the call, the value of `y` is therefore 1.

It can be seen that, as shown in Fig. 7.5, the actual parameter need not necessarily be a variable but can be an expression whose l-value is determined at call time. In a way similar to the first case, during the execution of `foo`, `x` is a name for `v[1]` and the increment of `x` in the body, is an increment of `v[1]`. After the call, the value of `v[1]` is, therefore, 2.

Fig. 7.5 Another example of passage by reference

```
int[] V = new V[10];
int i=0;
void foo (reference int x) {
    x = x+1;
}
...
V[1] = 1;
foo(V[i+1]);
// here V[1] = 2
```

In the stack-based abstract machine model, each formal is associated with a location in the procedure's activation record. During the calling sequence, the l-value of the actual is stored in the activation record. Access to the formal parameter occurs via an indirection which uses this location.

This is a parameter passing mode of very low cost. At the point of call, only an address need be stored; every reference to the formal is achieved by an indirect access (implemented implicitly by the abstract machine) which can be implemented at very low cost on many architectures.

Call by reference is a low-level operation. It is possible in Pascal (`var` modifier) and in many other languages. It has been excluded from more modern languages, where, however, some form of bidirectional communication between caller and callee can be obtained by exploiting the interaction between parameter passing and other mechanisms (the most important being the model chosen for variables) or the availability of pointers in the language. Two boxes discuss simple examples in C, Python, and Java. Without some form of bidirectional communication between caller and callee, subprograms would not be that versatile essential programming tool they are.

Call by Constant

We have already seen how call by value is expensive for large-sized data. When, however, the formal parameter is not modified in the body of the function, we can imagine maintaining the semantics of passing by value, implementing it using call by reference. This is what constitutes the *read-only* parameter mode.

This is a mode that establishes an input communication and in which arbitrary expressions are permitted as actual parameters. The formal parameters passed by this mode are subject to the static constraint that they cannot be modified in the body, either directly (by assignment) or indirectly (via calls to functions that modify them). From a semantic viewpoint, call by constant coincides with call by value, while the choice of implementation is left to the abstract machine. For data of small sizes, call by constant can be implemented as call by value. For larger structures, an implementation using a reference, without copy, will be chosen.

Call by constant is an good way to “annotate” a given parameter to a procedure. By reading the header, one immediately has information about the fact that this parameter is input only. Furthermore, one can count on static semantic analysis to verify that this annotation is really guaranteed.

Call by Object Reference, or by Assignment

When a language uses a reference-based model for variables (this is the case of Python and Java—limited to names of a class type in Java), the call by value mode takes the name of call *by object reference*, or also call *by assignment*. In the reference-based model, a name is a reference to a value. When an expression is used as an actual parameter in a subprogram call, it is a reference to the value of the expression to be passed (by value) and bound to the formal parameter. In the following Python fragment

```
def f ( x ) :
    x = val
f ( exp )
```

the expression `exp` is evaluated when `f` is called, determining a value `v`; the formal `x` is bound to `v` in the usual reference-based model. The body of `f` changes the binding of `x` from `v` to `val`. Upon termination, the new binding for `x` (and thus `val`) is destroyed, in accordance to the standard semantics of pass by value.

It is important to realize, however, that since *there may exist modifiable values*, one can modify a value inside the body of a function, and see that modification in the caller:

```
L = [ 1 ]
def f ( x ) :
    x [ 0 ] = val
f ( L )
```

After the fragment ends, `L` is bound to the list `[val]`.

One can see in the examples that indeed parameter passing acts as an assignment of the value of the actual parameter to the formal parameter, from which the name “call by assignment”. Since we are in a reference-based model, it is the reference that is passed and copied (and not the referenced value).

Similar examples that exploit modifiable values may be given using class instances, either in Python or Java. In Java:

```
class A {
    int v;
}
```

The following method swaps the values of the field `v` in two instances of `A`:

```
void swap ( A a, A b ) {
    int tmp = a.v; a.v = b.v; b.v = tmp;
}
```

What is passed (by value) to `swap` are two references to objects of class `A`.

Observe, in both cases, that we may make a function modify a value in the caller, but we cannot give a true simulation of call by reference, as instead it is the case with `C`.

Call by Result

Call by result is the dual of call by value. This is a mode that implements *output-only* communication. The procedure’s local environment is extended with an association between the formal parameter and a new local variable. The actual parameter must be an expression that evaluates to an l-value. When the procedure (normally) ter-

Fig.7.6 Call by result

```

void foo (result int x) {x = 8;}
...
int y = 1;
foo(y);
// here y is 8

```

Fig.7.7 Call by value-result

```

void foo (valueresult int x){
    x = x+1;
}
...
y = 8;
foo(y);
// here y is 9

```

minates, before the destruction of the local environment, the current value of the formal parameter is assigned to the location obtained as l-value of the actual parameter. As in call by value, the following questions about evaluation order must be answered. If there is more than one result parameter, in which order (for example, from left to right) should the corresponding “backward assignment” from the actual to formal be performed? Finally, when is the actual parameter’s l-value determined? It is reasonable to determine it both when the function is called and when it terminates.⁴

During the execution of the body, there is no link between the formal and actual parameters. There is no way to make use of a result parameter to transfer information from the caller to the callee. An example of call by result is shown in Fig. 7.6. The implementation of call by result is analogous to call by value, with which it shares its advantages and disadvantages. From a pragmatic viewpoint, the by-result mode simplifies the design of functions that must return (that is provide as result) more than one value, each in a different variable.

Call by Value-Result

The combination of call by value and call by result produces the call by value-result mode. It implements bidirectional communication using the formal parameter as a variable local to the procedure. The actual parameter must be an expression that can yield an l-value. At the call, the actual parameter is evaluated and the r-value thus obtained is assigned to the formal parameter. At the end of the procedure, immediately before the destruction of the local environment, the current value of the formal parameter is assigned to the location corresponding to the actual parameter. An example of call by value-result is shown in Fig. 7.7.

The canonical implementation of call by value-result is analogous to that of call by value, even if some languages (Ada, for example) choose to implement it as call by reference in the case of large-sized data, so that the problems of cost of call by value can be avoided.

⁴ Construct an example which gives different results if the l-value of the actual is determined at call time or when the procedure terminates.

Fig. 7.8 Call by value-result
is not call by reference

```
void foo (reference/valueresult int x,
         reference/valueresult int y,
         reference int z){
    y = 2;
    x = 4;
    if (x == y) z = 1;
}

...
int a = 3;
int b = 0;
foo(a, a, b);
```

The implementation of call by value-result using a reference is, however, not semantically correct. Consider, for a moment, the fragment in Fig. 7.8. At first sight, the conditional command present in the body of `foo` seems useless, for `x` and `y` have just received distinct values. The reality is that `x` and `y` have different values only *in the absence of aliasing*. If, on the other hand, `x` and `y` are two different names for the same variable, it is clear that the condition `x == y` is always true.

If, therefore, `x` and `y` are passed by value-result (there is no aliasing), the call `foo(a, a, b)` terminates without the value of `b` being modified. If, on the other hand, `x` and `y` are passed by reference (where there *is* aliasing), `foo(a, a, b)` terminates by assigning the value 1 to `b`.

Call by Name

Call by name was introduced in ALGOL 60 as a semantically cleaner way of passing parameters than by reference. After a hiatus of several decades, it has been reintroduced in some modern languages (e.g., Scala), and can be obtained at no cost in languages allowing anonymous functional parameters (lambda expressions), see Sect. 7.2.3. It is a conceptually important mode worth the effort of studying in detail because of its properties and its implementation.

The problem that the designers of ALGOL 60 set themselves was to give a *precise* semantics to procedure definition and call. The semantics should specify in an elementary fashion what the effect of a call to a function with specified parameters would be. The solution that they chose was to define the semantics of function call using the so-called *copy rule*. We will state it in the case of a function of one argument:

Let f be a function with a single formal parameter, x , and let a be an expression of a type compatible with that of x . A call to f with an actual parameter a is semantically equivalent to the execution of the body of f in which all occurrences of the formal parameter, x , have been replaced by a .

It is a simple rule, that reduces the semantics of function call to the syntactic operation of expanding the body of the function after a textual substitution of the actual for the formal parameter. This notion of substitution, however, is not as simple as it seems, because it must take into account the presence of several different variables with the same name. Consider, for example, the function in Fig. 7.9. If we


```

int x=0;
int foo (name int y){
    int x = 2;
    return x + y;
}
...
int a = foo(x+1);

```

Fig.7.9 Which environment should be used to evaluate $x+1$ in the body of `foo`?

```

int i = 2;
int fie (name int y) {
    return y+y;
}
...
int a = fie(i++);
// here i has value 4; a has value 5

```

Fig.7.10 Side effects of call by name

blindly apply the copy rule, the call `foo(x+1)` results in the execution of `return x+x+1`. This command, executed in the local environment of `foo`, returns the value 5. But it is clear that this is an incorrect application of the copy rule because it makes the result of the function depend on what the name of the local variable is. If the body of `foo` had been:

```
{ int z=2; return z+y; }
```

the same call would result in the execution of `return z+x+1`, with the result 3.

In the first substitution that we suggested, we say that the actual parameter, x , was *captured* by the local declaration. The substitution used in the copy rule must therefore be a substitution that *does not capture variables*. Since it is impossible to ban different variables with the same name, we can obtain a non-capturing substitution by requiring that the formal parameter, even after substitution is evaluated in the environment of the caller and not in that of the callee.

We can therefore define call by name as that parameter-passing mode whose semantics is given by the copy rule, where the concept of substitution is always understood without capture. Equivalently, we can say that what is substituted is not merely the actual parameter, but the actual parameter *together with* its own evaluation environment which is fixed at the moment of call.

Note, moreover, that the copy rule requires that the actual parameter is evaluated *only* when (and only if) the formal parameter is encountered during execution.⁵ Moreover, the actual parameter is evaluated *each time* the formal parameter is encountered during execution. Consider the example of Fig. 7.10, where the construct `i++` has the semantics of returning the current value of the variable `i` and then incrementing the value of the variable by 1. The copy rule requires that the `i++` construct is

⁵ We say that call by name is a *by need* parameter passing mode.

```

void fiefoo (valueresult/name int x, valueresult/name int y) {
    x = x+1;
    y = 1;
}
...
int i = 1;
int[] A = new int[5];
A[1]=4;
fiefoo(i,A[i]);

```

Fig. 7.11 Call by name is not call by value-result

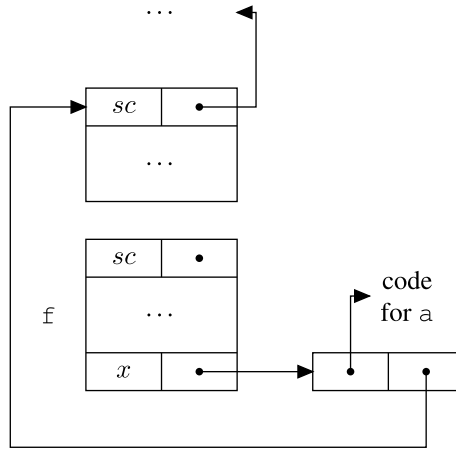
evaluated twice. Once for every occurrence of the formal parameter y in `fie`. The first time, its evaluation returns the value 2 and increments the value of i by one. The second time, it returns the value 3 and increments i again.

The example we have just discussed shows how it is an error to consider call by name as a complicated way of describing call by value-result. The two modes are semantically different, as Fig. 7.11 shows. Let us assume, first, that we are to execute `fiefoo` with parameters passed by value-result. On termination, we will have `A[1]` with value 1 and i with value 2, while the rest of the array `A` has not been touched. If, on the other hand, we execute the same procedure with the two parameters passed by name, on termination we have `A[1]` with value 4, i with value 2 and, what is more important, the element `A[2]` has been updated to the value 1. In this case, call by reference exhibits the same behaviour as call by value-result.

It remains to describe how call by name can be implemented. We have already seen that the caller must pass not only the textual expression forming its actual parameter but also the environment in which this expression must be evaluated. We call *closure*⁶ such a pair (expression, environment) in which the environment includes all the free variables in the expression. We can therefore say that, in the case of call by name, the caller passes a closure, formed from the actual parameter (in the form of a textual expression) and the current environment. When the callee encounters a reference to the formal parameter, it is resolved using the evaluation of the first component of the closure in the environment given by the second component. Figure 7.12 describes this situation in the particular case of an abstract machine with a stack. In this case, a closure is a pair formed from two pointers: the first points to the code that evaluates the expression of the formal parameter, the second is a pointer to the block which forms the local environment in which to evaluate the expression. When a procedure f with a formal name parameter, x , is called with actual parameter a , the call constructs a closure whose first component is a pointer to the code for a and whose second component is a pointer to the caller's activation record. It then binds this closure (for example, using another pointer) to the formal parameter x which resides in the called procedure's activation record.

⁶ The term "closure" comes from mathematical logic. A formula is closed when it does not contain free variables. A closure is a canonical method for transforming a piece of code containing nonlocal (that is, "free") variables in a completely specified code.

Fig. 7.12 Implementation of call by name



We can finally summarise what we know on call by name. It is a mode which supports input and output parameters. The formal parameter does *not* correspond to a variable that is local to the procedure; the actual parameter can be an arbitrary expression. It is possible that the actual and formal parameters can be aliased. The actual parameter must evaluate to an l-value if the formal parameter appears on the left of an assignment. The semantics of call by name was established by the copy rule which allows the maintenance of a constant link between formal and actual parameters during execution. The canonical implementation uses a closure. The procedure's local environment is extended with an association between the formal and a closure, the latter being composed of the actual parameter and the environment in which the call occurs. Every access to the formal parameter is performed via the *ex novo* evaluation of the actual parameter in the environment stored in the closure. Call by name is an expensive parameter-passing mode, both because of the need to pass a complex structure and, in particular, because of the repeated evaluation of the actual parameter in an environment that is not the current one.

7.2 Higher-Order Functions

A function is *higher order* when it accepts as parameters, or returns another function as its result. Although there is no unanimous agreement in the literature, we will say that a programming language is higher-order when it allows functions either as parameters or as results of other functions. Languages with functions as parameters are fairly common. On the other hand, languages that allow functions to return functions as a result are less common. This last type of operation, however, is one of the fundamental mechanisms of *functional* programming languages (which we will deal with extensively in Chap. 11). We will, in this section, discuss linguistic and implementation problems in these two cases. We treat each of them separately.

Jensen's Device

Call by name allows side effects to be exploited to obtaining elegant and compact code, even though it often results in code that is difficult to understand and maintain. This is the case with the so-called Jensen's Device which makes use of pass by name to pass a complex expression and, at the same time, a variable which appears in the same expression, in such a way that modifications to variable change the value of the expression. Let us consider the following example:

```
int summation (name int exp; name int i;
               int start; int stop) {
    int acc = 0;
    for (i=start, i<= stop, i++)
        acc = acc + exp;
    return acc;
}
int x = ...;
...
int y = summation(2*x*x - 2*x + 1, x, 1, 10);
```

The side effects of passing a parameter by name are such that, in the body of the loop in `summation`, the value of `exp` can depend upon the value of `i`. A moment's reflection shows that the call on the last line is equivalent to the calculation of the sum:

$$y = \sum_{x=1}^{10} 2x^2 - 2x + 1.$$

Jensen's Device allows call by name to be used as a way to derive powerful and specialisable "higher-order" procedures at call time (in the case of the example, by indicating the expression in which to calculate the sum).

7.2.1 Functions as Parameters

The general case is that of a language with functional parameters, nested environments and the ability to define functions at every nesting level.⁷ Let us consider the example shown in Fig. 7.13.

Using the notation `void g (int h(int n)){ ... }`, we mean, in our pseudo-language, the declaration of the function `g` with a single formal parameter, `h`, which, in its turn, is specified to be a function returning an `int` with its own formal parameter of type `int`.⁸ The two key points of the example are: (i) the fact that `f` is passed as an actual parameter to `g` and later called through the formal parameter `h`; and (ii) the name `x` is defined more than once, so it is necessary to establish which

⁷ C allows functions as parameters, but it is possible to define a function only in the global environment. With this limitation, the problem becomes considerably simplified (and this simplicity is one of the reasons why C does not allow nested functions).

⁸ The name of the formal parameter of `h` (in this case `n`), is of no relevance and there is no way in which the programmer can use it in the body of `g`.

```

{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}

```

Fig. 7.13 Functional parameters

is the (nonlocal) environment in which f will be evaluated. Concerning this second question, the reader will not be surprised if we observe that there are two possibilities for selecting the nonlocal environment to use when executing a function f invoked using a formal parameter h :

- Use the environment that is active at the time when the *link between h and f is created* (which happens on line 11). We say, in this case, that the language uses a *deep binding policy*.
- Use the environment that is active when the *call* of f using h occurs (which happens on line 7). In this case, we say that the language uses a *shallow binding policy*.⁹

Although the two alternatives for binding immediately recall the distinction between static and dynamic scope, we emphasise that the binding policy (in the case of higher-order functions) should be considered independent of scope policy. All common languages that use static scope also use deep binding (because the choice of a shallow policy appears contradictory at the methodological level). The matter is not as clear for languages with dynamic scope, among which there are languages with deep as well as shallow binding.

Returning to the example of Fig. 7.13, the different scope and binding policies yield the following behaviours:

- Under static scope and deep binding, the call $h(3)$ returns 4 (and g returns 6). The x in the body of f when it is called using h is the one in the outermost block;

⁹ The terminology, however, is not uniform across the literature. In particular, the terms deep and shallow binding are also used, in a special way, in the LISP community to indicate two different implementation techniques for dynamic scope.

- Under dynamic scope and deep binding, the call $h(3)$ returns 7 (and g returns 9). The x in the body of f when it is called using h is the one local to the block in which the call $g(f)$ occurs;
- Under dynamic scope and shallow binding, the call $h(3)$ returns 5 (and g returns 7). The x in the body of f at the moment of its call through h is the one local to g .

Implementation of Deep Binding

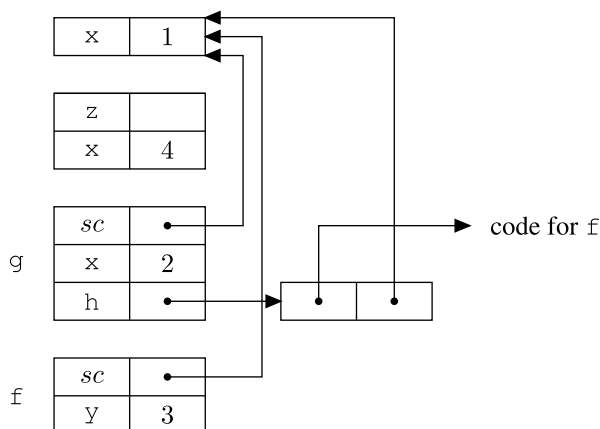
Shallow binding does not pose additional implementation problems to the technique used to implement dynamic scope. It is enough, at least conceptually, to look for every name's last association in the environment. Things are different, though, for deep binding, which requires auxiliary data structures in addition to the usual static and dynamic chains.

So as to fix our ideas, let us consider the case of a language with static scope and deep binding (the case of dynamic scope is left to the reader, see Exercise 6). From Sect. 5.5.1, we already know that to any direct invocation (one that is not of a call to a formal parameter) of a function f , there is statically associated information (an integer) which expresses the nesting level of the definition of f with respect to the block in which the call occurs. This information is used dynamically by the abstract machine to initialise the static chain pointer (that is, the nonlocal environment) in the activation record for f . When, on the other hand, a function f is invoked using a formal parameter, h , no information can be associated to the call because it is called via a formal parameter. Indeed, in the course of different activations of the procedure in which it is located, the formal can be associated with different functions (this is the case, for example, with the call $h(3)$ in Fig. 7.13).

In other words, it is clear that with deep binding, the information about the static chain pointer must be determined at the moment the association between the formal and actual parameters is created. With the formal h must be associated not only the code for f but also the nonlocal environment in which the body of f is to be evaluated. Such a nonlocal environment can be determined in a simple fashion: corresponding to a call of the form $g(f)$ (the procedure g is called with the functional actual parameter f), we can statically associate with the parameter f the information about the nesting level of the definition of f within the block in which the call $g(f)$ occurs. When this call is performed, the abstract machine will use this information to associate with the formal parameter corresponding to f both the code for f , and a pointer to the activation record of the block inside which f is declared (this pointer is determined using the same rules that were discussed in Sect. 5.5.1).

The formal function parameter is therefore associated logically with a (text, environment) pair, represented at the implementation level by a pair (pointer to code, pointer to an activation record). We have already seen when we discussed the implementation of call by name that such a data structure is called a *closure*. Therefore, a closure is associated with the formal functional parameter. When the formal parameter is used to invoke a function (that is statically unknown), the abstract machine finds in the first component of the pair the code to which to transfer control and

Fig. 7.14 Closure and static chain



assigns the content of the second component of the closure to the static chain pointer of the activation record for the new invocation.

Figure 7.14 shows the situation on the activation stack for the code in Fig. 7.13 at the moment at which the function `f` is entered (it is called using `h` from the body of `g`). When the function `g` is called with actual parameter `f`, a closure is linked to `h`. `f` is declared at distance 1 from the place in which it appears as an actual parameter (it is in fact declared inside the block which *contains* the one in which it appears as an actual parameter). The second component of the closure is therefore determined by 1 step along the static chain (obtaining a pointer to the outer block). When `f` is called through the name `h`, the corresponding activation record is pushed onto the stack. The value of the static chain pointer is taken from the second component of the closure.

The reader will recall, once more, that the problems we are discussing appear only when the language allows the definition of functions with nonlocal environments, that is allows the definition of functions *inside nested blocks*. In the contrary case, for example in C, there is no nonlocal environment, so there is no need for closures. To pass a function as a parameter, it is enough to pass a pointer to its code. All nonlocal references in the body of the function will be resolved in the global environment.

Binding Policy and Static Scope

We have already observed how all languages with static scope use deep binding. At first sight it could rather seem that deep or shallow binding make no difference in the case of static scope. After all, the nonlocal environment of a function is determined from the (static) position of its declaration and not by the way in which it is invoked. In the case in Fig. 7.13, it is the scope (and not binding) rule that establishes that every invocation of `f` (whether direct, using its name, or indirect, using a formal parameter) is evaluated in the outermost nonlocal environment.

In general, however, it is not like this. The reason for this is that there can be many activation records for the same function simultaneously present on the stack

Thunks

The parameter-passing rule required for functional parameters is similar to call by name. In fact, a formal name parameter can be considered as a kind of functional parameter (without arguments). Analogously, the corresponding actual parameter can be considered as the definition of an anonymous argumentless function. During execution of the body, every occurrence of the name parameter corresponds to an *ex novo* evaluation of the actual parameter in the environment fixed at the moment the association between the actual and formal name parameter is made. A process that is analogous to a new call to the anonymous function corresponding to the actual parameter.

In ALGOL 60 jargon, the name *thunk* was used for a structure like this—a function without arguments associated to its evaluation environment. In call by name, therefore a connection between the formal parameter and a thunk is introduced.

Fig. 7.15 The binding policy is necessary for determining the environment

```
void foo (int f(), int x){
    int fie(){
        return x;
    }
    if (x==0) return f();
    else return foo(fie,0);
int g(){
    return 1;
}
print(foo(g,1))
```

(this clearly happens when we have recursive or mutually recursive functions). If a procedure is passed out from one of these activations, it is possible to create a situation in which the scope rules alone are not enough to determine which nonlocal environment to use in invoking the functional parameter. As an example, we will discuss the code in Fig. 7.15, which, as usual, we assume was written in a pseudo-language with static scope.

The heart of the problem is the (nonlocal) reference to *x* inside *fie*. The scope rules tell us that such an *x* refers to the formal parameter to *foo* (which, as it happens, is the only *x* declared in the program). But when *fie* is called (using the formal *f*), there are two active instances of *foo* (and therefore two instances of its local environment). A first activation from the call *foo(g, 1)*, in which *x* is associated to the value 1, and a second one from the (recursive) call *foo(fie, 0)*, in which *x* is associated with the value 0. It is inside this second activation that the call to *fie* through *f* is made. We can argue that the scope rules say nothing about which of the instances of *x* should be used in the body of *f*. It is at this point that the binding policy intervenes. Using deep binding, the environment is established when the association between *fie* and *f* is created, that is when *x* is associated with the value 1. The call to *f()* will therefore return 1 (which is also the value printed by the program). To help in understanding this example, Fig. 7.16 shows the stack and the closures when *fie* is executed.

In the case of shallow binding (which, let us repeat, is not used with static scope), the environment would be determined at the time *f* is invoked and *z* would be assigned the value 0.

The Environment in C

The structure of the environment in C is particularly simple. A C program consists, in fact, of a sequence of variable and function declarations. The variables declared in this way (which in C jargon are called *external* variables) are visible at any point in the program. They are global variables, according to the terminology of Sect. 4.2.2. Functions are structured internally as blocks, and in each block local variables can be declared, but the definition of functions inside other functions is not permitted.

The environment of a function, therefore, is composed of a local and a global part. Each reference to a nonlocal name is resolved in a unique fashion in the global environment. With this simplified structure, environment handling is very simple. The static chain does not have to be maintained and to pass a function as a parameter to another function, it is sufficient to pass a pointer to its code.

For reasons of efficiency, furthermore, there is no management of in-line blocks. Variables declared in any block in a function are allocated in the activation record of the function.

Execution efficiency is one of the primary objectives for C. To avoid the cost of activation record stacks, the compiler can choose to translate a function call using the expansion of its body (in the case of a recursive function, the expansion happens once only).

What Defines the Environment

Before closing this section, let us consider again the problem encountered in Chap. 4, of which rules are used to determine the environment. We can finally complete the ingredients which contribute to the correct determination of the evaluation environment for a block-structured language. The following are necessary:

- Visibility rules, normally guaranteed by block structure.
- Exceptions to the visibility rules (which take into account, for example, redefinitions of names and the possibility or not of using a name before its declaration).
- Scope rules.
- The rules for the parameter passing method.
- The binding policy.

7.2.2 Functions as Results

The possibility of generating functions as the results of other functions allows the dynamic creation of functions at runtime. A function returned as a result cannot be represented at execution time by its code alone, since it is also necessary to include the environment in which the function will be evaluated. Let us consider a first simple example in Fig. 7.17. Let us fix, first of all, the notation: by `void->int` we denote the *type* of the functions which take no argument (`void`) and return a value of type integer (`int`). The second line of the code is therefore the declaration of a function `F` which returns a function of no arguments which also returns an integer (note that `return g` returns the “function”, not its application). The first line after the body

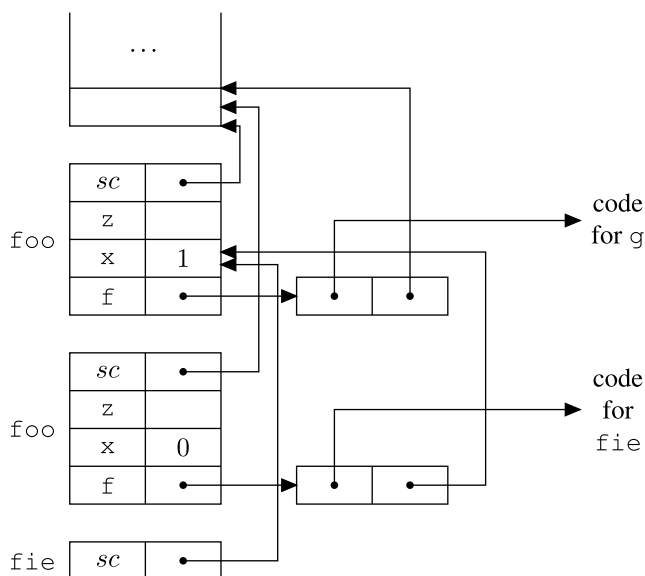


Fig. 7.16 Activation stack for Fig. 7.15

Fig. 7.17 Functions as results

```

{int x = 1;
void->int F () {
    int g () {
        return x+1;
    }
    return g;
}
void->int gg = F();
int z = gg();
}

```

of F is the declaration of the name gg with which the result of the evaluation of F is dynamically associated.

It should not be difficult to convince ourselves that the function gg returns the successor of the value of x . Using the static scope regime, this x is fixed by the structure of the program and not by the position of the call to gg , which could appear in an environment in which another definition of the name x occurs.

We can, therefore, say that, in general, when a function returns a function as result, this result is a *closure*. In consequence, the abstract machine must be appropriately modified to take into the account calls to closures. Analogous to what happens when a function is called via a formal parameter, when a function whose value is obtained dynamically (like gg), the static chain pointer of its activation record is determined using the associated closure (and not the canonical rules discussed in Sect. 5.5.1, which would be of no help).

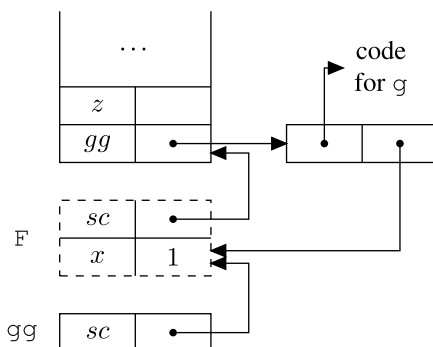
Fig. 7.18 Functions as result
and stack discipline

```

void->int F () {
    int x = 1;
    int g () {
        return x+1;
    }
    return g;
}
void->int gg = F();
int z = gg();

```

Fig. 7.19 Activation records
for Fig. 7.18



The general situation, moreover, is more complex. If it is possible to return a function from the inside of a nested block, it is possible to arrange that its evaluation environment will refer to a name that, according to stack discipline, is going to be destroyed. Let us consider, indeed, the code in Fig. 7.18, where we have moved the declaration of `x` to the inside of `F`. Figure 7.19 shows the activation record arrangement when `gg()` is called.

When the result of `F()` is assigned to `gg`, the closure which forms its value points to an environment containing the name `x`. But this environment is local to `F` and will therefore be destroyed on its termination. How is it possible, then, to call `gg` later, without producing a dangling reference to `x`? The reply can only be drastic: abandon stack discipline for activation records, so that they can then stay alive indefinitely, because they could constitute the environments for functions that will be evaluated subsequently. In languages with the characteristics that we have just discussed, local environments have an *unlimited* lifetime.

The most common solution in this case is to allocate all activation records in the heap and to leave it to a garbage collector to deallocate them when it is discovered that there are no references to the names they contain.

Functional languages are constructed around the possibility of returning functions as results—they must therefore take this problem head on. On the contrary, returning functions as results in imperative languages is less common, for the purpose to maintain a stack discipline for activation records. Older imperative languages which permit functions as results add restrictions aimed at guaranteeing that it is never possible to create a reference to an environment that has become deactivated—for example: no

Generators in Python

In Python, a “function” definition whose body contains a `yield` command, defines a *generator*—an object to be used for the generation of a sequence of values. The command `yield exp` (which, like a `return`, may appear only inside a function definition) stops the evaluation of the body, and returns the value of `exp` to the caller. Differently from `return`, however, a `yield` does not force the destruction of the activation record of the function call.

```
def f(x):
    while x < 10:
        yield x
        x = x + 1
F = f(2)
print(next(F))
print(next(F))
```

The call `f(2)` does *not* execute the body of `f` (but for the binding of the formal to the actual parameter). It returns instead a generator object, which the assignment binds to the name `F`. When the predefined function `next` is called on a generator, the body of the generator is executed until a `yield` command is found. In our example, a first `next(F)` will start the execution of `f`, stopping at `yield x`: the current value of `x` is returned as value of `next(F)` and thus 2 is printed.

When `yield` is executed, the activation record of its generator is not destroyed. Moreover, the program location where the `yield` occurred is saved, so that a subsequent `next` on the same generator will resume the execution from where it was paused and with the environment and memory in which it was paused. In the example, the second call `next(F)` will return 3. We can thus keep calling `next(F)`, obtaining all the values of the generator. After producing the last value (9, in the example) the generator is *exhausted*—a further call `next(F)` will make the execution reach the end of the body (that is, an implicit `return`) and a `StopIteration` exception is raised.

The name `StopIteration` makes explicit that generators may be used as sequences controlling a for iteration, see Sect. 6.3.3.

nested functions (C, C++), return only non-nested functions (Modula-2, Modula-3), appropriately restrict the scope of those nested functions that are returned (Ada). More modern languages, instead, allow a full functional programming style, and therefore implements also non-stack-based activation records. The case of Python is particularly interesting. In addition to the possibility of returning functions from nested scopes, it also allows subprograms to return a value and be put on hold (hence without their activation record being destroyed) so that their execution could be later resumed. It is the case discussed in the box on *generators*.

7.2.3 Anonymous Functions: Lambda Expressions

In older imperative languages, subprograms have a different status than other primitive values. For instance, in C we may introduce a function only through a function definition, thus giving *also* a name to that function. On the other hand, we have

Fig. 7.20 Python: lambda expressions simulate pass by name

```
x=0
def foo(y):
    x = 2
    return x + y()
a = foo(lambda : x+1)
```

unnamed expressions denoting integer values.¹⁰ More modern languages, where functions are first-class citizens, allow also unnamed expressions denoting functions, usually called *lambda* expressions (see Sect. 11.6 for the motivation for the name). In Scala the following line

```
(x: Double) => x * x
```

is an expression denoting the function taking a floating-point number and returning its square. The analogous expression in Python would be written

```
lambda x: x * x
```

We may use this expression as is, e.g. by applying it to an argument:

```
(lambda x: x * x) (2.0)
```

evaluates to 4.0. Or we may bind it to a name via an assignment

```
sq = lambda x: x * x
```

which is equivalent to a standard function definition:

```
def sq(x):
    return x * x
```

However, one of the main uses of lambdas is as arguments to higher-order functions, avoiding the definition of a function that otherwise would never be called on its own. As such, lambdas are also a way to obtain the same behavior of parameter passing by name. Figure 7.20 rephrases in Python, using lambdas, the program we gave with pass by name in Fig. 7.9. Note how the lambda without formal parameters (`lambda : x+1`) encodes a thunk (see the box on Sect. 7.1.2).

It should be clear, by now, that similarly to pass by name, the implementation of a lambda expression is a closure, since its body may well contain names that need an environment to make sense. As a final incarnation of closures, let us mention Scala's definitions with deferred evaluation. The following definition

```
def foo: Int = {
    println("test")
    10
}
```

introduces the integer name `foo`, bound to the code on the right of `=`, which is not evaluated at definition time. Whenever `foo` is evaluated, the code is evaluated

¹⁰ Using the terminology of the box on Sect. 6.2.2, functions in C are denotable, but not expressible. Integer values are both denotable and expressible (and also storable).

anew (thus obtaining the value 10 *and* the printing side-effect). The body of the `def` can use names defined in the outer environment, thus requiring a closure to be implemented.

7.3 Exceptions

An exception is an event that is checked during the execution of a program and which must not (or cannot) be handled in the normal flow of control. Such events could be checking that a dynamic semantic error has occurred (e.g., a division by zero, overflow, etc.) or checking that a situation has occurred for which the programmer explicitly decides to terminate the current computation and transfer control to another point in the program, often outside the currently executing block.

First-generation languages did not provide structures for handling such situations. They typically treat them by means of jumps (`gotos`). On the other hand, many modern languages such as C++, Scala, Java, and Python have *structural* mechanisms for exception handling which appear as real abstraction constructs. These constructs allow the interruption of a computation and the shifting of control outside of the current construct, block or procedure. Often, this mechanism also allows data to be passed across the jump, resulting in a very flexible (and also often efficient) tool for handling those cases of exceptional termination of a computation which the normal control constructs (loops and conditionals) are unable to handle properly. Devised for handling the unusual or exceptional cases which can present themselves in a program, exceptions are also useful, as we will see, when giving compact and efficient definitions of some ordinary algorithms.

The mechanisms for handling exceptional situations vary greatly from language to language. We will restrict ourselves here to describing some common approaches and we do not pretend to be exhaustive. In general, to correctly handle exceptions, a language must:

1. Specify *which* exceptions can be handled and how they can be defined.
2. Specify how an exception can be *raised*, that is which mechanisms cause the exceptional termination of a computation.
3. Specify how an exception can be *handled*, that is what are the actions to perform when an exception occurs and where to transfer control of execution.

On the first point, we find both exceptions raised directly by the abstract machine (when some dynamic semantic condition is violated) and exceptions defined by the user. The latter can be values of a special type (as is the case in Ada and in ML), or any value whatsoever (as in C++) or something in the middle (in Java and Python, an exception is an instance of some subclass of a class predefined for this purpose—`Throwable` in Java, `Exception` in Python). Usually, an exception can contain a dynamically generated value which is passed to the handler.

```

class EmptyExcp extends Throwable {int x=0;};

int average(int[] V) throws EmptyExcp(){
    if (length(V)==0) throw new EmptyExcp();
    else {int s=0; for (int i=0, i<length(V), i++) s=s+V[i];}
    return s/length(V);
};
...
try{...
    average(W);
    ...
}
catch (EmptyExcp e) {write('Array_empty');}

```

Fig. 7.21 Exception handling

Once an exception is defined, it can be raised implicitly if it is an abstract machine exception, or explicitly by the programmer using an appropriate construct.

Finally, for point (3), the handling of an exception requires two different constructs:

- A mechanism that defines a capsule around a portion of code (the *protected block*), with the aim of intercepting the exceptions that are to be handled inside the capsule itself.
- The definition of a handler for the exception, statically linked to the protected block. Control is transferred to the handler when the capsule intercepts the exception.

Let us examine the example in Fig. 7.21 (written in the usual pseudo-language inspired by Java and C++). The first line is the definition of the exception. In the case we are considering, all instances of class `EmptyExcp` can be an exception. To an approximation, we can imagine an instance of such a class as a record with a single internal field, labelled by `x`. Passing an exception involves the creation of a such a value and then raising the exception proper.

The second line is the definition of the `average` function. The keyword `throws` introduces the list of exceptions that *can* be thrown in the body of the function (in our case `EmptyExcp`). This clause (necessary in Java but optional in C++) has an important function as documentation: it shows the clients of the function that, in addition to the integer result, it could result in anomalous termination as signalled by the exception itself.

The construct that raises an exception is `throw`. A protected block is introduced by the keyword `try`, while the handler is contained in a block introduced by `catch`. The `average` function computes the arithmetic mean of the elements of the vector `V`. In the case in which the vector is empty, the function, instead of returning something arbitrary, raises an exception of class `EmptyExcp`. In checking such an event, the language's abstract machine interrupts the execution of the current command (in this case, the conditional command) and propagates the exception. All blocks entered during execution are exited until a `try` block trapping (`catch`) *this* exception

is found. In the case of Fig. 7.21, the `average` function would be terminated, as would every block appearing between the only `try` present and the call to `average`. When the exception is intercepted by a `try`, control passes to the code in the `catch` block. If no explicit `try` trapping the exception is encountered, it is captured by a default handler which then terminates execution of the program and prints some error message.

The handler (the code in the `catch`) block is statically linked to the protected block. When an exception is detected, execution of the handler replaces the part of the protected block which has still to be executed. In particular, after execution of the handler, control flows normally to the first instruction which follows the handler itself.¹¹

As far as these questions are strongly dependent on the language, let us make two important observations:

1. An exception is not an anonymous event. It has a name (often, rather, as in our case, it is a value in one of the language's types) which is explicitly mentioned in the `throw` construct and is used by constructs of the form `try-catch` to trap a specific class of exception.
2. Although the example does not show it, the exception could include a value which the construct that raised the exception passes in some way to the handler as an argument on which to operate in order to react to the exception that has just occurred (in our case, the value of the field `x` could have been modified when the exception is raised).

It is easy to convince ourselves that the propagation of an exception is not a simple jump. Let us assume that the exception is raised inside some procedure. If the exception is not handled inside the currently executing procedure, it is necessary to terminate the current procedure and to re-raise the exception at the point at which the current procedure was invoked. If the exception is not handled by the caller either, the exception is propagated along the procedure's call chain until it reaches a compatible handler or reaches the highest level which provides a default handler (which results in an error termination).

A subtle aspect which is worth explicitly considering is that exceptions propagate along the dynamic chain, even if the handler is statically associated with the protected block. To illustrate this point, let us consider the code in Fig. 7.22, where we have assumed that we have already declared an exception class `X`.

The exception `X` is raised inside two protected blocks. The outer `try` is the one in the body of `g`. This is the one that traps the exception. The program prints the string "in `g`". Figure 7.23 shows the stack of activation records and handlers when the

¹¹ This way of working is, in the literature, called "handling with termination" (because the construct where the exception is determined is terminated). Some older languages, PL/I for example, (one of the first languages to introduce exception-handling mechanisms) follow a different approach, called "handling with resumption". In this case, the handler can arrange that control returns to the point where the exception was raised. The scheme with resumption can lead to flagging errors that are very difficult to locate.


```

{
void f() throws X {
    throw new X();
}

void g (int sw) throws X {
    if (sw == 0) {f();}
    try {f();} catch (X e) {write("in_g");}
}
...
try {g(1);}
catch (X e)  {write("in_main");}
}

```

Fig. 7.22 Exceptions propagate along the dynamic chain

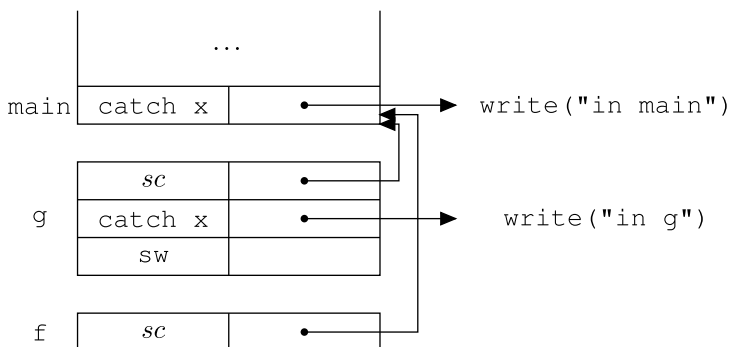


Fig. 7.23 System stack for Fig. 7.22

body of function **f** is executed. Considering Fig. 7.22, note that, in the case in which the argument of **g** in the protected block is a variable whose value is execution-dependent, it is not statically determinable which will be the handler to invoke.

Summarising, an exception is handled by the last handler that has been placed on the execution stack. This is a reasonable behaviour. The exception is trapped at the closest possible point to the one at which it was detected.

Pragmatics

We have discussed the use of exceptions in handling error cases. There are ordinary cases, though, in which the cautious use of exceptions makes for more elegant and efficient programs. We will limit ourselves to presenting one example, that of walking a binary tree where it is desired to calculate the product of the integers which label the nodes. The most obvious code is that shown in Fig. 7.24, where a depth-first search of the tree is used and it is assumed that the type **Tree** is a structure (a record or a class) with three fields of which the first is an integer and the others are of type **Tree** and which link the structure together. The generic null value is represented by **null**.

Easier to Ask Forgiveness Than Permission

Python provides a fast and efficient exception-handling mechanism, and an abstract machine that checks many common problems, raising the corresponding predefined exceptions. These two factors make the use of exceptions a popular programming style.

In the Python community a coding style characterized by the presence of many `try` and `except` statements is called EAFP (easier to ask forgiveness than permission) and it is recommended over a programming style where one checks beforehand for all possible limit cases, characterized by many `if` statements (which would result in a “look before you leap” style, LBYL).

```

type Node = {int key;
              Node FS;
              Node FD;
            }

int mul (Node alb){
    if (alb == null) {return 1;}
    return alb.key * mul(alb.FS) * mul(alb.FD);
}

```

Fig. 7.24 Anticipated visit to a binary tree

The function `mul` correctly returns the product of the nodes, but is inefficient in the cases of very large trees where there is significant chance that some nodes are zero, given that in such a case the function could immediately terminate with the result zero. We can make use of the exception mechanism to force this expected termination.

Figure 7.25 shows the function `mulAus` which raises an exception (of class `Zero`, which we assume to be defined elsewhere) when it encounters a node labelled with zero. `mulAus` is called by `mulEff`¹² inside to the protected block which handles the exception returning zero.

```

int mulAus (Node alb) throws Zero{
    if (alb == null) {return 1;}
    if (alb.key == 0) {throw new Zero();}
    return alb.key * mulAus(alb.FS) * mulAus(alb.FD);
}

int mulEff (Node alb){
    try {return mulAus (alb);}
    catch (Zero e) {return 0;};
}

```

Fig. 7.25 A more efficient traversal

¹² The function `mulEff` is the function “exported” to clients in this program. In jargon, we say that `mulEff` is a *wrapper* of `mulAus`.

7.3.1 Implementing Exceptions

The simplest and intuitive way in which an abstract machine can implement exceptions is one that uses the stack of activation records. When a block is entered at runtime, a pointer to the corresponding handler (together with the type of exception to which it refers) is inserted into the activation record (of the current procedure or of the current anonymous block). When a normal exit is made from a protected block (that is, because control transfers from it in the usual way and not through raising an exception), the reference to the handler is removed from the stack. Finally, when an exception is raised, the abstract machine looks for a handler for this exception in the current activation record. If one is not found, it uses the information in the record to reset the state of the machine, removes the record from the stack and rethrows the exception. This is a conceptually very simple solution but it has a not insignificant defect: each time that a protected block is entered, or is left, the abstract machine must manipulate the stack. This implementation, therefore, requires explicit work even in the normal (and more frequent) case in which the exception *is not* raised.

A more efficient runtime solution is obtained by anticipating a little of the work at compile time. In the first place, each procedure is associated with a hidden protected block formed from the entire procedure body and whose hidden handler is responsible only for clearing up the state and for rethrowing the exception unchanged. The compiler then prepares a table, *EH*, in which, for each protected block (including the hidden ones) it inserts two addresses (*ip*, *ig*) which correspond to the start of the protected block and the start of its handler. Let us assume that the start of the handler coincides with the end of the protected block. The table, ordered by the first component of the address, is passed to the abstract machine. When a protected block is entered or exited normally, nothing need be done. When, on the other hand, an exception is raised, a search is performed in the table for a pair of addresses (*ip*, *ig*) such that *ip* is the greatest address present in *EH* for which $ip \leq pc \leq ig$, where *pc* is the value of the program counter when the exception is detected. Since *EH* is ordered, a binary search can be performed. The search locates a handler for the exception (recall that a hidden handler is added to each procedure). If this handler re-throws the exception (because it does not capture this exception or because it is a hidden handler), the procedure starts again, with the current value of the program counter (if it was a handler for another exception) or with the return address of the procedure (if it was a hidden handler). This solution is more expensive than the preceding one when an exception is detected, at least by a logarithmic factor in the number of handlers (and procedures) present in the program (the reduction in performance depends on the need to perform a search through a table every time; the cost is logarithmic because a binary search is performed). On the other hand, it costs nothing at protected block entry and exit, where the preceding solution imposed a constant overhead on both of these operations. Since it is reasonable to assume that the detection of an exception is a rarer event than simple entry to a protected block, we have a solution that is on average more efficient than the preceding one.

Exceptions and Static Scope

In languages with static scope, the combination of the dynamic association of handlers with protected blocks and the static scope for exception name definitions, may create subtle interactions.

Let us discuss the Java code in Fig. 7.26. On a superficial reading, the code seems syntactically correct and we may presume that an invocation of `g` on an instance of `Q` results in the string “in `g`” being printed. Yet, if compilation is attempted, the compiler finds two static semantic errors around line 12: (i) the exception `X` which must be caught by the corresponding `catch` is not raised in the `try`; (ii) the exception `X`, raised by `f`, is not declared in the (absent) `throws` clause of `g`.

Exception names (`X` in this case) have normal static scope. Method `f` raises the exception `X` declared on line 1; while the `catch` on line 12 traps the exception `X` declared on line 9 (and which is more correctly denoted by `Q.X`, since it is a nested class within `Q`). Just so as to avoid errors caused by situations of this kind, Java imposes the requirement on every method that it must declare all the exceptions that can be generated in its body in its `throws` clause.

The analogous situation can be reproduced in C++, *mutatis mutandis*. In C++, however, the `throws` clause is optional. If we compile the C++ code corresponding to that in Fig. 7.26, in which `throws` clauses are omitted, compilation terminates properly. But an invocation to the method `f` throws an exception different from that caught in the body of `g`. An invocation of `g` on an instance of `Q` results in an exception `X` (of the class declared on line 1) which is then propagated upwards.

In Python, on the other hand, the program situation of Fig. 7.26 would print the string “in `g`”. But not because exception names do not follow a static scope discipline. The fact is that the scope of any name local to a *class* definition *does not include the blocks nested into the class*. Hence, the exception name `X` defined inside `Q` is not visible inside `g`, where, however, is visible the `X` defined at top level. To reproduce the C++ behavior we may move the definition of `X` from the first line of `Q` to the first line of the definition of `g`. Now a call to `g` on an instance of `Q` results in an uncaught exception.

That class scopes do not include nested blocks (thus introducing an important exception to the general visibility rule for block-structured languages we introduced in Sect. 4.2.2) is a controversial Python feature.

7.4 Summary

The chapter has dealt with a central question for every programming language: mechanisms for functional abstraction. It has discussed, in particular, two of the most important linguistic mechanisms to support functional abstraction in a high-level language: procedures and constructs for handling exceptions. The main concepts introduced are:

- *Procedures*: The concept of procedure, or function, or subprogram, constitutes the fundamental unit of program modularisation. Communication between procedures is effected using return values, parameters and the nonlocal environment.
- *Parameter passing mode*: From a semantic viewpoint, there are input, output and input-output parameters. From an implementation viewpoint, there are different ways to pass a parameter:
 - By value.
 - By reference.

```

class X extends Exception {};
class P{
    void f() throws X{
        throw new X();
    }
}

class Q{
    class X extends Exception {};
    void g(){
        P p = new P();
        try {p.f();} catch (X e){
            System.out.println("in_g");
        }
    }
}

```

Fig. 7.26 Static scope and exception names

- By means of one of the variations on call by value: by result, by constant or by value-result.
- *Higher-order functions*: Functions that take functions as arguments or return them as results. The latter case, in particular, has a significant impact on the implementation of a language, forcing the stack discipline for activation records to be abandoned.
- *Binding policy*: When functions are passed as arguments, the binding policy specifies the time at which the evaluation environment is determined. This can be when the link between the procedure and the parameter is established (deep binding) or when the procedure is used via the parameter (shallow binding).
- *Closures*: Data structures composed of a piece of code and an evaluation environment, called *closures*, are a canonical model for implementing call by name and all those situations in which a function must be passed as a parameter or returned as a result.
- *Exceptions*: Exceptional conditions which can be detected and handled in high-level languages using *protected blocks* and a *handler*. When an exception is detected, the appropriate handler is found by ascending the dynamic chain.

7.5 Bibliographical Notes

The principal modes for parameter passing originate in the work of the ALGOL committee and were subsequently explored in other languages such as ALGOL W and Pascal. The original definition of ALGOL 60 [1] is a milestone in programming language design. The preparatory work on ALGOL W can be seen in [2] and in its mature form in the reference manual [3]. ALGOL W included call by name (as

default), call by value, by result and by value-result, as well as pointers and garbage collection. Pascal, which adopts as default call by reference, was first defined in [4]; the reference manual for the ISO Standard is [5]. Call by object reference (“by assignment”) was introduced in CLU [6] (and originally called “call by sharing” [7]).

The problems with determining the environment in the case of higher-order functions are often known as the *funarg problem* (the *functional argument problem*). The *downward* funarg problem refers to the case of functions passed as arguments and therefore to the necessity of handling deep binding. The *upward* funarg problem refers to the case in which a function can be returned as a result [8]. The relations between binding policy and scope rules are discussed in [9].

One of the first languages with exceptions was PL/1 which used resumption handling (see [10]). More modern handling with termination (where the text of the program specifies which handler is responsible for any protected block) descends from Ada, which, in its turn, was inspired by [11].

7.6 Exercises

1. On Sect. 7.1, commenting on Fig. 7.1, it can be seen that the environment of the function `foo` includes (as a nonlocal) the name `foo`. What purpose does the presence of this name serve inside the block?
2. State what will be printed by the following code fragment written in a pseudo-language permitting reference parameters (assume `Y` and `J` are passed by reference).

```
int X[10];
int i = 1;
X[0] = 0;
X[1] = 0;
X[2] = 0;
void foo (reference int Y,J){
    X[J] = J+1;
    write(Y);
    J++;
    X[J]=J;
    write(Y);
}
foo(X[i],i);
write(X[i]);
```

3. State what will be printed by the following code fragment written in a pseudo-language which allows *value-result* parameters:

```
int X = 2;
void foo (valueresult int Y){
    Y++;
    write(X);
    Y++;
```

```

}
foo(X);
write(X);

```

4. The following code fragment, is to be handed to an *interpreter* for a pseudo-language which permits constant parameters:

```

int X = 2;
void foo (constant int Y){
    write(Y);
    Y=Y+1;
}
foo(X);
write(X);

```

What is the most probable behaviour of the interpreter?

5. Say what will be printed by the following code fragment which is written in a pseudo-language allowing *name* parameters:

```

int X = 2;
void foo (name int Y){
    X++;
    write(Y);
    X++;
}
foo(X+1);
write(X);

```

6. Based on the discussion of the implementation of deep binding using closures, describe in detail the case of a language with dynamic scope.
7. Consider the following fragment in a language with exceptions and call by value-result and call by reference:

```

{int y=0;
  void f(int x){
    x = x+1;
    throw E;
    x = x+1;
  }
  try{ f(y); } catch E {};
  write(y);
}

```

State what is printed by the program when parameters are passed: (i) by value-result; (ii) by reference.

8. In a pseudo-language with exceptions, consider the following block of code:

```

void ecc() throws X {
    throw new X();
}
void g (int para) throws X {
    if (para == 0) {ecc();}
    try {ecc();} catch (X) {write(3);}
}

```

```

}
void main () {
    try {g(1);} catch (X) {write(1);}
    try {g(0);} catch (X) {write(0);}
}

```

Say what is printed when `main()` is executed.

9. The following is defined in a pseudo-language with exceptions:

```

int f(int x){
    if (x==0) return 1;
    else if (x==1) throw E;
        else if (x==2) return f(1);
            else try {return f(x-1);} catch E {return x+1;}
}

```

What is the value that is returned by `f(4)`?

10. The description of the implementation of exceptions in Sect. 7.3.1 assumes that the compiler has access (direct or through the linkage phase) to the entire code of the program. Propose a modification to the implementation scheme based on the handler table for a language in which separate compilation of program fragments is possible (an example is Java, which allows separate compilation of classes).

References

1. J.W. Backus, F.L. Bauer, J. Green, C. Katz, J. McCarthy, A.J. Perlis, H. Rutishauser, K. Samelson, B. Vauquois, J.H. Wegstein, A. van Wijngaarden, M. Woodger, Report on the algorithmic language ALGOL 60. Commun. ACM **3**(5), 299–314 (1960)
2. N. Wirth, C.A.R. Hoare, A contribution to the development of ALGOL. Commun. ACM **9**(6), 413–432 (1966)
3. R.L. Sites, ALGOL W reference manual. Technical Report STAN-CS-71-230, Computer Science Department, Stanford University, Stanford, CA, USA (1972)
4. N. Wirth, The programming language Pascal. Acta Informatica **1**(1), 35–63 (1971)
5. K. Jensen, N. Wirth, *Pascal User Manual and Report* (Springer, Berlin, 1991)
6. B. Liskov, A history of CLU, in *The Second ACM SIGPLAN Conference on History of Programming Languages*, HOPL-II (Association for Computing Machinery, New York, NY, USA, 1993), pp. 133–147
7. B. Liskov, R. Atkinson, T. Bloom, E. Moss, C. Schaffert, B. Scheifler, A. Snyderw, CLU reference manual. Technical Report MIT-LCS-TR-225, MIT-LCS (1979)
8. J. Moses, The function of FUNCTION in LISP or why the FUNARG problem should be called the environment problem. SIGSAM Bull. **15**, 13–27 (1970)
9. T.R. Virgilio, R.A. Finkel, Binding strategies and scope rules are independent. Comput. Lang. **7**(2), 61–67 (1982)
10. M.D. MacLaren, Exception handling in PL/I, in *Proceedings of an ACM Conference on Language Design for Reliable Software* (1977), pp. 101–104
11. J.B. Goodenough, Exception handling: issues and a proposed notation. Commun. ACM **18**(12), 683–696 (1975)

Each programming language provides constructs and mechanisms for structuring data. Instead of just the simple sequences of bits in the physical machine, a high-level language provides complex, structured data which more easily describes the structure of the problems to be solved. These constructs and mechanisms are shaped by what is called the *type system* of a language. Far from being an auxiliary aspect, types represent one of the salient characteristics of a programming language, which substantially differentiates one language from another.

In this chapter, we will examine type systems in the general sense, discussing primitive types and the mechanisms used to define new ones. Central to our presentation will be the concept of *type safety*, which will be introduced in Sect. 8.2. We will then tackle the questions of type equivalence and compatibility of types, that is mechanisms allowing to use a value of some type in a context requiring another type. We will then discuss polymorphism and overloading. We will conclude the chapter with some questions about storage management (*garbage collection*), which is not, strictly speaking, a topic about data types but which well complements the examination of pointers which we must undertake.

8.1 Data Types

Data types are present in programming languages for at least three different reasons:

1. At the design level, as support for the conceptual organisation;
2. At the program level, as support for correctness;
3. At the translation level, as support for the implementation.

Before entering into a detailed discussion of these aspects, which we will do in the coming sections, we give a definition which, as is often the case with programming

languages, is not formally precise but suffices to explain the phenomena which we intend studying.

Definition 8.1 (*Data Type*) A *data type* is a homogeneous collection of values, effectively presented, equipped with a set of effective operations which manipulate those values.

Let us clarify the various elements of this definition. A type is a collection of values, like the integers, or an integral interval. The adjective “homogeneous”, for all its informality, suggests that these values must share any structural property which makes them similar to each other. Next, such values “come with” the operations which manipulate them. For example, together with the integers, we can consider the usual operations of addition, subtraction, multiplication and division; or, we may add also operations such as remainder after integer division and raising to a power. According to our definition, the same set of values, equipped with two distinct sets of operations, forms another data type. Finally, values must be “effectively presented”, and operations must be “effective.” Since we are speaking of languages for describing algorithms, we are interested in values which it is possible to present (write, name) in a finite manner. Real numbers (the “true” ones in mathematics, that is the only complete archimedean ordered field) are *not* effectively presentable because there are real numbers with infinite decimal expansion which cannot be obtained by means of any algorithm. Their approximations in programming languages (`real` or `float`) are only subsets of the rationals. This presentation is usually fixed, for a given type in a specific language (e.g., floating point numbers are presented as finite sequences of digit with a “point”, or with an exponential notation). Finally, operations must be effective, meaning that there is a mechanical, concrete way of performing them on the presentation of values.

8.1.1 Types as Support for Conceptual Organisation

The solution to every complex problem has a conceptual structure which often reflects that of the problem. The presence of different types allows the designer to use the type that is most appropriate to each class of concept. For example, a program handling hotel reservations, will handle concepts such as client, date, price, rooms, etc. Each of these concepts can be described as a different type, with its own set of operations. The designer can define new types and associate them with different concepts, even if they are represented by the same values. For example, rooms and prices could be both represented by integers within specified intervals, but their representation as distinct types makes their conceptual difference explicit.

The use of distinct types can be seen both as a design and a documentation tool. When reading a program, it may be clear from their declaration that a variable of type “room” has a different role from that of a variable of type “price”. In this sense, types are similar to comments, with the important difference that—under certain

assumptions and in certain programming languages—we are dealing with *effectively checkable* comments, as we will see in the next section.

8.1.2 Types for Correctness

Every programming language has its own type-checking rules which regulate the use of types in a program. For instance, it may be forbidden to add integers and records, or to call (that is, transfer control to) an object which is not a function or a procedure. In languages where variables (names) are declared, a common rule is the one regulating the assignment. For a command of the form $x := \text{exp}$ to be correct, the majority of languages require that the type of exp coincides (or better, is compatible) with the (declared) type of x .

Such constraints are present in languages both to avoid runtime hardware errors (for example, a call to an object that is not a function might cause an address error), and, more likely, to avoid the kinds of logic error frequently hidden beneath type-rule violations. The sum of an integer and a string rarely corresponds to anything sensible.

Programming languages, then, assume that the violation of a type constraint corresponds to a possible semantic error (a design error). Many languages require that type constraints must all be satisfied before the execution (or the code generation) of a program. This is the role of the *type checker*, a very important component of the static semantic checking phase of a compiler (Sect. 2.3). We referred to types (in Sect. 8.1.1) as checkable comments. As it is the case for comments, the programmer uses types to communicate the legal ways with which the given objects can be used; but (unlike comments), the compiler (or the language's abstract machine) may detect and signal attempted incorrect use of these objects.

It is clear that a program that is correct with respect to the type rules can still be logically incorrect. Types ensure a minimal correctness, which, however, is of considerable help during the development phase of a program. A powerful analogy is that of dimensional control in physics: when a physicist writes a formula, before verifying its correctness using laws, they verify that the dimensions are correct. If the formula is to express velocity, there must be distance over a time; if there is an acceleration, there must be distance over time squared. If the formula is dimensionally incorrect, no more time should be spent on it because it is certainly incorrect. If, on the other hand, it is dimensionally correct, it *might* be incorrect and must be handled semantically.

Sometimes, however, the type rules can appear too restrictive. A C programmer is used to the free handling of pointers (which are, in C, actual memory locations). They may find the restrictions on performing arbitrary pointer arithmetic such as those in Java unnecessarily restrictive. In this case, the reply by the designer of Java is that the benefits of strict control over types considerably outweighs the loss of expressiveness and conciseness.

A more apt example of the restrictions which could be imposed by a type-system is that of a sorting subprogram. In some languages, because of the presence of types,

it will be necessary to write one routine to order an integer sequence, another to order sequences of characters, and still another for sequences of reals, and so on. All these functions are identical as far as algorithm goes and differ only in the declaration of the types of the parameters and variables. The way out, in this case, is to adopt more sophisticated typing rules, which, without renouncing any control, allow one to write a single function which is parameterised by type. We will see below that languages which allow this kind of *polymorphism* are becoming common.

A different approach is taken from those languages which postpone at run time almost all type checks, the notable example being Python. In this way one may obtain greater flexibility, to the expenses of the amount of static correctness that could be ensured.

Let us finally observe that the rules pertaining to types are not always sufficient to guarantee that the constraints they express are satisfied. We give just a single example here. In a language that permits the explicit deallocation of store in a heap, it is possible that references (pointers) are generated that refer to memory that is no longer allocated to the program (*dangling references*). An attempted access using such a reference is an error that can be classified as a type error, but it is not guaranteed that it will be detected and reported by the abstract machine. Let us therefore classify programming languages as existing somewhere between *secure* and *insecure* with respect to types, according to how possible it is that there can be type-constraint violations during program execution that go undetected by the abstract machine.

8.1.3 Types and Implementation

The third motivation for the use of types in programming languages is that they are important sources of information for the abstract machine. The first kind of information is about the amount of memory to be allocated to various objects. The compiler can allocate one word for an integer, one byte for a boolean value, n words for a sequence of n integers, and so on. A type makes this information available.

Moreover, if types are statically determined, this information could be used for compilation and does not change during execution. As a consequence, we may optimize the access to an object. In Sect. 5.3, we discussed how access to an allocated variable in an activation record is performed using an offset from the pointer to an activation record, without a runtime search by name. This form of optimisation is possible because the information carried by types allows the static determination of the allocation sizes for almost every object, even for heap-allocated objects. We will soon see that a record is formed from a collection of fields, each of which is characterised by its own name and its own type. For example, using the same notation as in C, the following declaration introduces the `Professor` type, a record with two fields:

```
struct Professor{
    char Name[20];
    int Course_code;
}
```

If now we have a variable, `p`, of type `Professor`, we can access its fields using either the name `p.Name` or the name `p.Course_Code`. However the object `p` is allocated (in the heap or on the stack), access to its fields will always be possible through the use of offsets from the start address of `p` in memory.

8.2 Type Systems

We first introduce a little terminology which we will illustrate in detail in the following sections. Every programming language has its own *type system*—the complex of information and rules which govern the types in that language. More precisely, a type system consists of the following¹:

1. The set of predefined types of the language.
2. The mechanisms which permit the definition of new types.
3. The mechanisms for the control of types, among which we distinguish the following:
 - Equivalence rules which specify when two formally different types correspond to the same type.
 - Compatibility rules specifying when a value of a one type can be used in a context in which a different type would be required.
 - Rules and techniques for type inference which specify how the language assigns a type to a complex expression based on information about its components.
4. The specification as to whether (or which) constraints are statically or dynamically checked.

A type system (and, by extension, a language) is *type safe*² when no program can violate the distinctions between types defined in that language. In other words, a type system is safe when no program, during its execution, can generate an unsignalled error derived from a type violation. Once more, it is not always clear what a type violation is, at least in general. We have already given some examples, such as access to memory that is not allocated to the program, or the call of a non-functional value. We will see below other examples of errors of this kind.

We have defined a type as a pair composed of a set of values and a set of operations. In any particular language, the values of a type can correspond to different syntactic entities (constants, expressions, etc.). Having fixed a programming language, we can

¹ “A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute” [1].

² Much of the literature uses the term *strongly typed* in place of type safe.

classify its types according to how the values of a type can be manipulated and the kinds of syntactic entity that corresponds to these values. Following the classification that we have already seen in the box on Sect. 6.2.2, we have values:

- *Denotable*, if they can be associated with a name.
- *Expressible* if they can be the result of a complex expression (that is different from a simple name).
- *Storable* if they can be stored in a variable.

Let us give some examples. Values of type integer are in general denotable (they can be associated with constants), expressible, and storable. The values of the type of functions from `int` to `int` are denotable in almost all languages because a name can be given using a declaration. For example:

```
int succ (int x) {
    return x+1;
}
```

This assigns a name `succ` to the function which computes the successor. Functional values are not always expressible in common imperative languages because there are no complex expressions returning a function as the result of their evaluation. In the same way, they are not, in general, storable values because it is not possible to assign a function to a variable. The situation is different in languages from other paradigms, for example functional languages (Scheme, ML, Haskell, etc.) in which functional values are both denotable, and expressible or, in some languages, storable. In Python, any value is denotable, expressible and may be assigned at run-time to a name.

8.2.1 Static and Dynamic Checking

A language has *static typing* if its checking of type constraints can be conducted on the program text at compile time. Otherwise, it has *dynamic typing* (that is if checking happens at runtime).

Dynamic type checking assumes that every object (value) have a runtime descriptor specifying its type. The abstract machine is responsible for checking that every operation is applied only to operands of the correct type. Often, this can be done by making the compiler generate appropriate checking code that is executed before each operation. It is not difficult to see that dynamic type checking locates type errors but is not efficient, given that operations are intermixed with type checking. In addition, a possible type error is revealed only during execution when the program might be in operation with its end user.

In static type checking, on the other hand, checks are made during compilation. In this scheme, checks are performed and reported to the programmer before the program is sent to the user. When checking is completely static, moreover, the explicit maintenance of type information at execution time is useless because correctness is guaranteed statically *for every execution sequence*. Execution is therefore more

efficient, given that checks are not required at runtime. There is, clearly, a price to pay. In the first place, the design of a statically-typed language is more complex than that of a dynamic language, especially if, together with static checking, guaranteed type safety is also desired. In the second place, compilation takes longer and is more complex, a price that one pays willingly, given that compilation takes place only a few times (with respect to the number of times that the program will be executed) and, above all because type checking shortens the testing and debugging phases.

There is, in the end, a third price to pay. This is less evident than the others but is intimately connected with the nature of static type checking. Static types can decree as erroneous, programs that, in reality, do not cause a runtime type error. By way of a simple example, let us consider the following fragment in our pseudocode:

```
int x;  
if (0==1) x = "pippo";  
else x = 3+4;
```

The first branch of the conditional assigns the integer variable, `x`, to a value that is incompatible with its type but the execution of the fragment causes no error because the condition is never satisfied. However, every static type checker will signal that this fragment is incorrect because the types of the two conditional branches are not the same. Static checking is therefore more *conservative* than dynamic checking. The motivation for this statement is found in the considerations of Chap. 3 on the existence of undecidable problems. In addition to the halting problem, the problem of determining whether a program causes a type error at execution time is undecidable (see the box on “Type errors are undecidable”). It follows from this that there exists no static check that can determine all and only those programs that produce errors at runtime. Static typing therefore adopts a prudential position: that of excluding more programs than strictly necessary with the justification that it can therefore guarantee correctness.

As we have already seen more than once in other contexts, static and dynamic type checking represent the two extremes of a spectrum of solutions in which the two methods coexist. Almost every high-level language combines static and dynamic type checks. We promise to return to this topic in due course; meanwhile let us just give a simple example from Pascal, a language traditionally classified as one that uses static type checking. Pascal allows the definition of interval types (see Sect. 8.3.9). For example, `1..10` is the type of the integers between 1 and 10 (inclusive). An expression of an interval type must be checked dynamically to guarantee that its value is properly contained within the interval. More generally, if a language with arrays want to check that the index of an array lies between the bounds of that array, must perform checks at runtime.

8.3 Scalar Types

Scalar (or simple) types are those types whose values are not composed of aggregations of other values. In this section, we will undertake a quick review of the

Type Errors are Undecidable

It is not difficult to prove that the problem of determining whether a program will cause a type error during its execution is undecidable. Let us, in fact, make use of what we already know, the undecidability of the halting problem.

Let us consider the following fragment, where *P* is an arbitrary (type-correct) program:

```
int x;
P;
x = "pippo";
```

Under what conditions will this fragment produce a type error as it runs? Only if *P* terminates, in which case it will try to execute the assignment:

```
x = "pippo";
```

which clearly violates the type system. If, on the other hand, *P* does not terminate, it will produce no error because control remains always inside *P* without ever reaching the critical assignment. Therefore, the fragment generates a type error if and only if *P* terminates. If there now existed a general method for deciding whether an arbitrary program causes a type error while executing, we could apply this method to our fragment. Such a method, though, would also be a method for deciding the termination of the (arbitrary) program *P*, something we know to be impossible.

main scalar types found in most common programming languages, while, in the next section, we will be concerned with the types that result from aggregating different values. The details (which we will not give) clearly depend on specific languages. In order to fix our notation, let us assume that we have in our pseudo-language the following way of defining (or declaring) new types:

```
type newtype = expression;
```

This introduces the name of the type, *newtype*, whose structure is given by *expression*. In C, we would write, for the same meaning:

```
typedef expression newtype;
```

8.3.1 Booleans

The type of logical values, or booleans, is composed of:

- *Values*: The two truth values, *true* and *false*.
- *Operations*: an appropriate selection from the main logical operations: conjunction (and), disjunction (or) and negation (not), equality, exclusive or, etc.

If present, its values can be stored, expressed and denoted. For reasons of addressing, the memory representation does not consist of a single bit but of a byte (or possibly more if alignment is required).

8.3.2 Characters

The character type is composed of:

- *Values*: a set of character codes, fixed when the language is defined; the most common of these are ASCII and UNICODE.
- *Operations*: strongly dependent upon the language; we always find equality, comparison and some way to move from a character to its successor (according to the fixed encoding) and/or to its predecessor.

Values can be stored, expressed and denoted. The representation in store will consist of a single byte (ASCII) or of a variable numbers of bytes or words (UNICODE, with its character encodings UTF-8, UTF-16 or UTF-32).

8.3.3 Integers

The type of integer numbers is composed of:

- *Values*: In most cases a finite subset of the integers, usually fixed when the language is defined (but there are cases in which it is determined by the abstract machine, which can be the cause of some portability problems). Due to representation issues, an interval of the form $[-2^t, 2^t - 1]$ is commonly used. Python (together with a handful of other languages) assumes that the values of the `int` type are the true mathematical integers (hence an infinite set).
- *Operations*: the comparisons and an appropriate selection of the main arithmetic operators (addition, subtraction, multiplication, integer division, remainder after division, exponentiation, etc.).

The values can be stored, expressed and denoted. In most languages the representation in memory consists of an even number of bytes (usually 2, 4, or 8), in two's complement form. Python (and the languages which support arbitrary-length integers) must use a different representation, spreading a single integer value onto a variable number of memory words.

8.3.4 Reals

The so-called real (or floating point) type is composed of:

- *Values*: an appropriate subset of the rational numbers, usually fixed when the language is defined (but there are case in which it is fixed by the specific abstract machine, a matter that deeply affects portability); the structure (size, granularity, etc.) of such a subset depends on the representation adopted.

Empty or Singleton

At first sight, one might be confused by the statement that `void` has one (a single) element rather than none. Let us think a little. We are used to defining a function which “returns nothing” as:

```
void f ( . . . ) { . . . }
```

If `void` were the empty set, we could not write a function such as `f`. There exist no functions with an empty codomain, with the unique exception of the function that is everywhere divergent. It is, instead, sensible to assume that in `void`, there is a single element and that this (implicitly) is returned by `f`. Since such an element is unique, we have no (and we must not have any) interest in explicitly saying what it is. Python makes this explicit, with the object `None`, the unique value of type `NoneType`, which is returned by any function which does not (explicitly) returns another value.

- *Operations*: comparisons and an appropriate selection of the main numeric operations (addition, subtraction, multiplication, division, exponentiation, square roots, etc.).

The values can be stored, expressed and denoted. The memory representation consists of four, eight and also ten bytes, in floating point format as specified by the IEEE 754 standard.

8.3.5 Fixed Point

The so-called fixed point type for reals is composed of:

- *Values*: an appropriate subset of the rational numbers, usually fixed when the language is defined; the structure (size, granularity, etc.) of such a subset depends on the representation adopted.
- *Operations*: comparisons and an appropriate selection of the main numeric operations (addition, subtraction, multiplication, division, exponentiation, extraction of square roots, etc.).

The values can be stored, expressed and denoted. The representation in memory consists of four or eight bytes. Values are represented in two’s complement, with a fixed number of bits reserved for the decimal part. Reals in fixed point permit compact representation over a wide interval with few precision places.

Enumerations in C

In C, our definition of `Dwarf` takes the form:

```
enum Dwarf {Bashful, Doc, Dopey, Grumpy, Happy, Sleepy, Sneezzy};
```

Apart from notational variants, the essential point is that in C (but not in C++), such a declaration is substantially equivalent to the following:

```
typedef int Dwarf;  
const Dwarf Bashful=0, Doc=1, Dopey=2,  
          Grumpy=3, Happy=4, Sleepy=5, Sneezzy=6;
```

The type equivalence rules for the language, in other words, allow an integer to be used in place of a `Dwarf` and vice versa. Type checking does not distinguish between the two types and, therefore, better documentation is all that is obtained by the use of an enumeration; stronger type checking is not obtained. In languages derived from Pascal, on the other hand, enumerations and integers are different types.

8.3.6 Complex

The so-called complex type is composed of:

- *Values*: an appropriate subset of the complex numbers, usually fixed by the definition of the language; the structure (size, granularity, etc.) of this subset depends on the adopted representation.
- *Operations*: comparisons and an appropriate selection of the main numerical operations (sum, subtraction, multiplication, division, exponentiation, taking of square roots, etc.).

The values can be stored, expressed and denoted. The representation consists of a pair of floating-point values.

8.3.7 Unit

In some languages, there exists a primitive type whose semantics is that of having a single value. It is sometimes denoted `void` (even if, semantically, it would be better to call it `unit`, given that it is not the empty set but a singleton):

- *Values*: only one, which can be written as `()`, or `None`.
- *Operations*: none.

It is used to denote the type of operations that modify the state but return no value. For example, in some languages, (but not in C, Java or Python), assignments (are also expressions and) have type `void`.

8.3.8 Enumerations

Some languages allow the user to define new scalar types, the most common example being enumerations and intervals.

An enumeration type consists of a fixed set of constants, each characterized by its own name. In our pseudo-language, we could write the following definition

```
type Dwarf = {Bashful, Doc, Dopey, Grumpy, Happy, Sleepy, Sneezy};
```

which introduces a new type with the name `Dwarf` and is a set of seven elements, each one denoted by its own name.

The operations available over an enumeration consist of comparisons and of a mechanism to move from a value to its successor and/or predecessor value (this should be compared with what was said about the character type).

From a pragmatic viewpoint, enumerations permit the creation of highly legible programs insofar as the names for values constitute a fairly clear form of self documentation of the program. Type checking moreover can be exploited to check that a variable of an enumeration type assumes only the correct values.

Introduced for the first time in Pascal, enumeration types are present in many other languages; the box discusses those in C.

A value of an enumeration type is typically represented by a one-byte integer. The individual values are represented by contiguous values, starting at zero. Some languages (C and Ada for example) allow the programmer to choose the values corresponding to the different elements of an enumeration.

8.3.9 Intervals

The values of an interval (or range) type form a contiguous subset of the values of another scalar type (the *base type* of the interval). Two examples in Pascal (which was the first language to introduce intervals as well as enumerations) are:

```
type Bingo = 1..90;  
SomeDwarves = Grumpy..Sleepy;
```

In the first case, the `Bingo` type is an interval of 90 elements whose base type is the integer type. The interval `SomeDwarves` is formed from the values `Grumpy`, `Happy` and `Sleepy` and has `Dwarf` as its base type.

As in the case of enumerations, the advantage of using an interval type rather than the corresponding base type is both that it is better for documentation and because it provides a stronger type check. It can be seen that verifying that a value of a certain expression really belongs to the interval must necessarily be made dynamically, even in those languages whose type system is designed for the static checking of type constraints.

As far as representation goes, a compiler can represent a value of an interval type as a one- or two-byte integer according to the number of elements in the interval.

In reality, usually an abstract machine will represent the values of an interval in the same way (and in the same number of bytes) in which the base type is represented.

8.3.10 Ordered Types

The boolean, character, integer, enumeration and interval types are examples of *ordered types* (or *discrete types*). They are equipped with a well-defined concept of total order and, above all, possess a concept of predecessor and successor for every element, except for the extreme values. Ordered types are a natural choice for array indices and for control variables of bounded iterations (see Sect. 6.3.3).

8.4 Composite Types

Non-scalar types are said to be *composite* since they are obtained by combining other types using appropriate *constructors*. The most important and common composite types are:

- *Record* (or structure), an ordered collection of values in general of different type.
- *Union* and *tagged union* (or variant), a collection resulting from the union of other types.
- *Array* (or vector), an ordered collection of values of the same type.
- *Set*: subsets of a base type, generally ordinal types.
- *Pointer*: l-values which permit access to data of another type.
- *Sequence*: finite sequences of values (of any type).
- *Recursive type*: types defined by recursion, using constants and constructors; particular cases of recursive types are lists, trees, etc.

In the following subsections, we will analyse these types in the same order as in the above list.

8.4.1 Records

A record is a collection formed from a finite number of (in general ordered) elements called *fields*, which are distinguished by name. Each field can be of a type that is different from the others (records are a *heterogeneous* data structure). In the majority of imperative languages, each field behaves like a variable of the same type.³ The

³ In many non-imperative languages, a record is, instead, a finite set of values each of which is accessed by name. Ignoring the question of name-based access, a record type, in such a case, is the cartesian product of the types of its fields.

terminology is not, as usual, universal. In Pascal, one talks about records, in C (C++, ALGOL 68, etc.), one talks about *structures* (`struct`); in Java or Python, they do not exist because they are subsumed by the concept of class.

A simple example in our pseudo-language (inspired by C's notation) is the following:

```
type Student = struct{
    int year;
    float height;
};
```

Each record of type `Student` is composed of a pair whose first component is an integer, while the second is a real (float).

The only operation usually permitted on a value of a record type is the selection of a component, which can be obtained using the name of the fields. For example, if `s` is a name that refers to a `Student`, we can assign values to its fields as follows:

```
s.year = 12345;
s.height = 1.85;
```

In many languages, records can be nested, that is the field of a record is permitted to be of a record type.

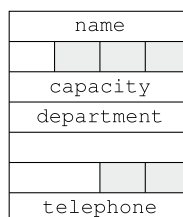
```
type Hall = struct{
    char name[5];
    int capacity;
    struct{
        char department[10];
        int telephone;
    } manager;
};
```

The third field (`manager`) of a record of type `Hall` is a record with two fields (whose type, anonymous, is therefore also a `struct`; the convention is observed that the type occurs before the name being declared). The selection of fields is performed by an obvious extension of the dot notation: if `h` refers to a `Hall`, with `h.manager.telephone`, we can denote the second field of the third field (`manager`) of `h`.

Equality of records is not always defined (Ada permits it but Pascal, C, C++ do not). Similarly, assignment of entire records is not always permitted. In the case in which these operations are not permitted, the language user must explicitly program them, for example by comparing (or assigning) one field at a time.

The order of fields is, in general, significant and is reflected in the way in which records are stored. The fields of a record are stored in contiguous locations, even if reasons of alignment can insert a hole between one field and another. For example, in a 32-bit architecture a record of type `Hall` could be represented by as shown in Fig. 8.1. The name field is represented by 5 bytes. The following field, however, being an integer, must be aligned on a word boundary and therefore between name and capacity, 3 empty bytes must be left. A similar thing happens with fields `department` and `telephone`. A record of type `Hall` is therefore represented

Fig. 8.1 Possible storage arrangement for a record of type `Hall`



by 7 words each of 4 bytes, even though only 23 bytes are significant.⁴ To avoid these problems, some languages do not ensure that the order of fields will be maintained by the abstract machine, thus allowing the compiler to re-organise fields in order to minimise the number of holes required to enforce alignment. Our record of type `Hall` could be represented in 6 words, with the waste of a single byte. This is a good implementation if the language guarantees a significant level of abstraction between the user and the implementation. In some applications, on the other hand, there is the need to allow users to manipulate the implementation of types. Languages such as C are designed also to allow this kind of manipulation—in such cases, the reorganisation of fields would not be a good design decision.

8.4.2 Unions

When runtime central-memory was an expensive resource, it was useful to have types akin to records, but where fields would be mutually exclusive, to save memory space. Called *variant records* in Pascal, the concept survives in C *union* types. An (untagged) union is analogous to a record (`struct`) in definition and in selection of its fields, with the fundamental difference that only one of the fields of a union can be active at any particular time, given that the fields (which can be declared of different types) share the same area of storage. We will be content with a simple example. In the scope of the following C definition

```
union Example{
    int lastyear;
    struct{
        float height;
        int year;
    } student;
};
```

any variable `ex` of type `Example` would take only two memory words, see Fig. 8.2. At any single time of the execution, either the first of these two words is significant, or both. In the former case, only the field `ex.lastyear` makes sense, as an integer. In

⁴ The presence of holes is often the reason for which some languages do not permit equality between entire records. A bit-by-bit comparison of two records would distinguish two records which are in reality equal apart from the irrelevant information present in the holes.

Secure Union: ALGOL 68

Is it possible to design secure variant records such that they do not threaten the very roots of the language’s type system? The reply is certainly positive, provided we are willing to pay the price, both linguistically and in terms of efficiency. Let us discuss here in summary form the ALGOL 68 solution. ALGOL 68 allows the definition of union types and requires the abstract machine to follow the evolution of the type of any variable of union type.

```
union (int, bool, char) tmp; # tmp of union type #  
...  
tmp := true; # now tmp is of type boolean #  
...  
tmp := 123; # now tmp is an integer #
```

For every variable of union type, the abstract machine maintains a hidden type tag which is implicitly set when an assignment occurs. The crucial point is that a union can be used only through a “conformity clause” (a case) which specifies what to do with this variable in all cases. For example:

```
case tmp in  
  (int a) : a := a+1,  
  (bool b) : b := not b,  
  (char c) : print(c)  
esac
```

A conformity clause is the only construct in the language that generate type checks for dynamic types.



Fig.8.2 The two possible storage allocations for a union type

the latter, the first word would be interpreted as the float for `ex.student.height`, while the second word would be the integer for `ex.student.year`. Which of the two cases is significant at any single moment is sole responsibility of the programmer—the language does not (and cannot) provide any support for type-checking.

Union types in this form are a significant threat to type safety, since there is no way to connect a reference to a union value to its actual type and representation, even at run-time. On the other hand, nowadays unions are of little use to save on memory usage. For this reason, untagged union types (variant records) gradually disappear from major languages, while an alternative, safe design choice is discussed in the following section.

8.4.3 Tagged Unions

A tagged union is the union of several other types, where, however, each value maintains trace of the original type it comes from. It is the modern evolution of ALGOL 68's unions (see the box on Sect. 8.4.2) and Pascal's enumerations and variant records. Popular first in functional languages, tagged unions are now available in most modern languages. Let us take the following as a simple example

```
type Three =  n of int |  b of bool |  st of char;
```

A value of type `Three` is either an integer, a boolean value, or a character, together with a type tag (a name) which discriminates the three cases. The definition above makes explicit that `n` is the integer tag, `b` is the boolean tag, and `st` is the character tag. After the declaration

```
Three V = st 'A'
```

the name `V` is bound to the value `st 'a'` of type `Three`. Whenever a value of `Three` is used, it must be checked against the possible tags it may have. The following fragment updates `V`, incrementing it of one if it is bound to an integer, negating it if it is a boolean, or finally leaving it unchanged if it is a character:

```
V = case V in
    n x : n (x+1)
    b x : b (not x)
    st x : st x
end case
```

Tagged unions are the programming language version of what type theory calls sum types. Enumerations, as discussed in Sect. 8.3.8, are a degenerate case of tagged unions, built over unit types (see Sect. 8.3.7)—the only interesting information are the very tags, not the corresponding values. Our enumeration type

```
type Dwarf = {Bashful, Doc, Dopey, Grumpy, Happy, Sleepy, Sneezy};
```

is shorthand for the tagged union

```
type Dwarf = Bashful of unit | Doc of unit | ... | Sneezy of unit;
```

Values of a tagged union are always represented storing also the tag, which is exploited at run time whenever a discriminating `case` is used.

Option Types

Besides enumerations, a particularly interesting case of tagged unions are the so-called *option types*, as present, e.g., in Scala. Given a type `T`, the type

```
Option[T]
```

could be seen as shorthand for the tagged union

```
None of unit | Some of T
```

The rationale for the introduction of option types into a language is the opportunity to treat in a coherent way the result types of functions with special (or error) cases.

Consider a function `find(S, e)`, returning the first index where the element `e` occurs in the sequence `S`. What value should be returned if `e` is not present in `S`? With option types we could declare

```
Option[int] find(sequence S, int e){
    ... }
```

and make `find` return `None` when `e` is not in `S`; if, on the other hand, `e` occurs as `S[i]`, we make `find` return `Some(i)`. Anytime `find` is called, we may match its result against the tags `None` or `Some`, discriminating in this way the possible “error” case.

8.4.4 Arrays

An array (or vector) is a finite collection of *elements* of the same type, indexed by an interval of ordinal type.⁵ Each element behaves as if it were a (modifiable) variable (see Sect. 6.2.1) of the same type. The ordinal interval of the indices is the index type, or the type of the array indices. The type of the elements is the *component type*, or even, with a little imprecision, the *array type*. Since all elements are of the same type, arrays are homogeneous data types.

Arrays are among the most common composite type found in programming languages; they first appeared in FORTRAN, the progenitor of all high-level languages. The syntax and various characteristics of arrays, however, vary considerably from language to language. The fundamental ingredients of array declarations are its name, its index type and the type of its elements. Let us begin with one of the simplest examples we can produce in C:

```
int v[10];
```

The square brackets after the name of the variable indicate that we are dealing with an array formed from 10 elements, each of which is a variable of type integer (from now on, we will simply say: an array of 10 integers). The element type is therefore `int`, while the index type is an interval of 10 elements; using the convention adopted in many languages (C, C++, Java, etc.), this interval starts at 0. In this case, therefore, the index type is the interval 0–9. In general, we can assume that a language allows the declaration of the index type to be an arbitrary interval of an ordinal type, for example:

```
int W[21..30];
type Dwarf = {Bashful, Doc, Dopey, Grumpy, Happy, Sleepy,
              Sneezzy};
float Z[Dopey..Sneezzy];
```

In the first declaration, `W` is an array of 10 integers, with integer indices from 21 to 30. The last line declares `Z`, an array of 4 reals, with indices taken from the `Dwarf` type that run `Dopey` to `Sneezzy`.

⁵ From a semantic viewpoint, an array is a function which has, as domain, an interval for the indices and, as codomain, a type for the array elements.

All languages permit the definition of *multidimensional* arrays, that is arrays indexed by two or more indices:

```
int V[1..10,1..10];
char C[Dopey..Sleepy,0..10,1..10];
```

The array *V* is a 10×10 square integer matrix, with row and column indices running from 1 to 10; *C* is a matrix whose elements are characters and whose bounds are $4 \times 11 \times 10$.

In some languages, a multidimensional array can be obtained by declaring that the type of the array elements is, at the same time, an array. A possible syntax in our pseudo-language could be the following, so we can declare the above as:

```
int V[1..10][1..10];
char C[Dopey..Sleepy][0..10][1..10];
```

In some languages, the two models (multidimensional array and array of array) are equivalent (one is an abbreviation for the other). In other languages, only one of the two possibilities is admitted. In C, a multidimensional array is an array of arrays and must be declared as such. Other languages, finally, allow both models, but arrays of arrays may support additional operations (e.g., *slicing*, as will shortly be seen) that are not defined over multidimensional arrays.

Operations on Arrays

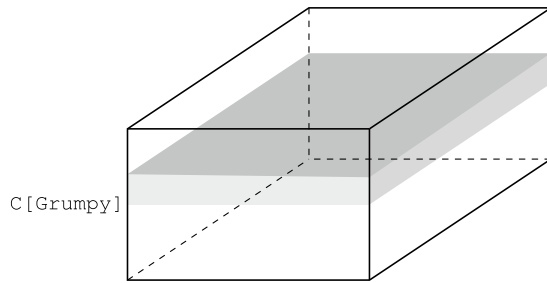
The simplest operation permitted on an array is *element selection*. It is performed using an index variable. The most common notation uses square brackets: $W[e]$ indicates the element of *W* corresponding to the index denoted by the value of the expression *e*. For multidimensional arrays, it is possible to find $C[\text{Grumpy}, 1, 2]$ or $C[\text{Grumpy}][1][2]$, depending on whether the emphasis is on multidimensional or arrays of arrays.

Some languages, furthermore, allow operations on arrays in their entirety: assignment, equality, comparisons and also arithmetic operations (performed, in general, elementwise). In other languages, these global operations are just one particular case of operations that allow the selection of interior parts of an array, which is operated on in a global fashion. An *array slice* is a portion of an array composed of (in general contiguous, but with exceptions) elements. With the declarations above, $V[3]$ could indicate the third row of the matrix *V*, and $C[\text{Grumpy}]$ the plane of the 3-dimensional matrix *C* which is obtained by selecting only its first component (Fig. 8.3). Languages may allow the selection of more sophisticated slices: diagonals, frames, etc.

Checking

The specification of the index type of an array is an integral part of its definition. Type checking in the language, therefore, would have to verify that every access of an element of a vector really happens “between the bounds” of the array and that there is no attempt to access elements that do not exist. With the exception of some special cases, this check can occur only at runtime. A secure language, therefore,

Fig. 8.3 A slice through an array



will have to ensure that the compiler generates appropriate checks for *every access*. Since this checking will affect the efficiency of the program, some languages, while permitting such checks to be generated, allow their deactivation.

Let us observe, by the way, that we are dealing with a matter of not insignificant importance to the security of a system, understood not just as “type safety”, but as “security” in a real sense. One of the most common attacks, which is also one of the most serious for the security of a system, is called *buffer overflow*. A malicious agent sends messages across the network with the aim of having them read into the buffers of the destination. If the destination does not check that the length of messages does not exceed the capacity of the buffer, the malicious sender can write into an area of memory that is not allocated to the buffers. If the buffer is allocated in an activation record and the sender succeeds in writing into the area the activation record reserves for saving the procedure’s return address, a “return” to any instruction can be made, in particular it can be made to jump to malicious code loaded by the attacker on purpose (for example in the same buffer that caused the overflow). In almost all cases, a buffer overflow attack can be stopped by an abstract machine that checks that every access to an array happens “between the bounds”.

Storage and Calculation of Indices

An array is usually stored as a contiguous portion of memory (see Exercise 2 for a non-contiguous allocation technique). For a single-dimensional array, allocation follows the order of the indices. In the case of a multidimensional array, there are two alternative techniques, referred to as storage in *row-major order* and in *column-major order*. In row-major order, two elements are contiguous when they differ in their last index (except in the case of elements at the extremes of a row, or on a plane, etc.). In column-major order, two elements are contiguous if they differ in the first index (except in the case of extremal elements). Row-major order is a little more common than column-major order (which is the storage standard of FORTRAN and, more recently, MATLAB). The fact that the elements of a row are stored contiguously makes the selection of a row slice easier (this is the slicing operation that is most often provided).

The two storing techniques are not equivalent as far as efficiency is concerned, particularly when there is a cache. Programs that manipulate large multidimensional arrays are often characterised by nested loops that run over such arrays. If the array

cannot be stored entirely in a cache, it is important that its elements are accessed “along” the cache, such that the first miss brings into the cache the elements that will be accessed immediately after. If the loop works by row, the cache must also be loaded by row, that is, row-major order is more convenient; if the loop is columnwise, column-major order should be preferred.

Once an array is stored in a contiguous fashion, the calculation of the address corresponding to an arbitrary element is not difficult, even if it requires a little arithmetic. Let us consider the generic array of n dimensions, with elements of type T .

$T \quad \forall [L_1 \dots U_1] \dots [L_n \dots U_n] ;$

Let S_n be the number of the addressable units (typically: bytes) necessary to store a single element of type T . Starting from this value, we can successively calculate a series of values which express the quantity of memory required to store increasingly larger slices of $\forall$. Let us assume we are working in row-major order, so we have:

$$\begin{aligned} S_{n-1} &= (U_n - L_n + 1)S_n, \\ &\dots \\ S_1 &= (U_2 - L_2 + 1)S_2. \end{aligned}$$

For example, for $n = 3$, S_3 is the amount of memory required to store a single element, S_2 is the amount required to store a row, while S_1 is the amount required to store an entire plane of $\forall$. The values S_j are usually called the *strides* (or stride factors) of the array.

The address of element $\forall[i_1, \dots, i_n]$ is now obtained by adding to the first address used to store $\forall$ the quantity:

$$(i_1 - L_1)S_1 + \dots + (i_n - L_n)S_n.$$

If the number of dimensions (that is, n), and the value of the L_j and U_j are all known at compile time (that is the *shape* of the array is static, see next section), it is convenient to rewrite the addressing expression as:

$$i_1 S_1 + \dots + i_n S_n - (L_1 S_1 + \dots + L_n S_n). \quad (8.1)$$

Here, the second part (in parentheses), and *a fortiori*, all the S_j are constants that can be determined at compile time. Making use of this formulation, an address is calculated (dynamically) with n multiplications and n additions. The final subtraction disappears whenever all the L_j are zero (which explains why some languages use zero as lower bound for indices).

Shape of an Array: Where an Array is Allocated

The *shape* of an array is given by the number of its dimensions and by the interval within which each of them can vary. An important aspect of the definition of a language is the decision about when the form of an array is fixed. Corresponding to the three principal cases, we also have three different models for allocating arrays in memory:

- *Static shape*. Everything is decided at compile time; the array can be stored in the activation record of the block in which its declaration appears (or in the static

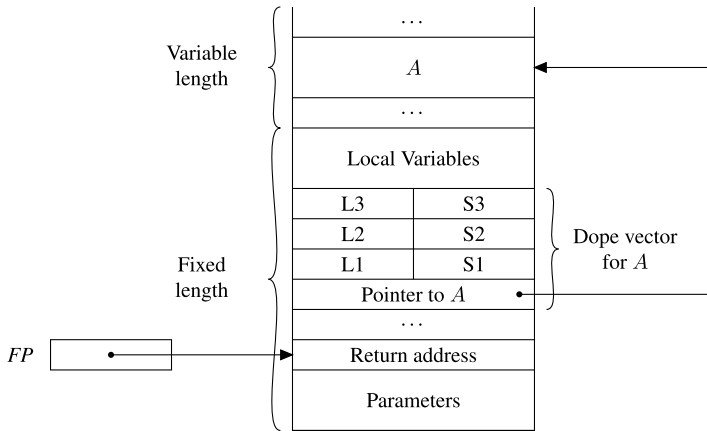


Fig. 8.4 Structure of an activation record with a dope vector

memory allocated for globals, if used). Let us observe that in this case, the total dimension required for an array is a constant that is known at compile time. The offset from the address of the array's start and the fixed point in the activation record to which the activation record pointer points is also therefore a constant (this was explained in Sect. 5.3.3). Access to an element of the array differs from access to an ordinary (scalar) variable only by the quantity determined by (8.1).

- *Shape fixed at declaration time.* In this case, the shape is not defined at compile time but is recorded and fixed when execution reaches the array declaration (for example, the interval of the index depends on the value of a variable). In this case, too, the array can be allocated in the activation record of the block in which its declaration appears, but the compiler has no way of knowing what the offset is between the start of the array and the fixed point of the activation record to which the frame pointer refers. This is an unpleasant position because it could have repercussions for other, completely static, data structures which were allocated in the activation record *after* such an array whose shape is fixed only at declaration time. To avoid this problem, an activation record is divided into two parts: one part for fixed-length data, and the other for variable-length data. Access to all the data allocated in the fixed-length part is performed in the usual fashion. On the other hand, when accessing a variable-length data item, an indirection through a data descriptor is performed. The descriptor is contained in the fixed-length part which is accessed by offset and contains, amongst other things, a pointer to the start of the data structure (in our case, an array). Figure 8.4 shows a scheme for allocation inside an activation record for this. The descriptor for an array is known as a *dope vector*. We will describe dope vectors in a little more detail in a short while.
- *Dynamic shape.* In this case, an array can change shape after its creation as a result of the effects of execution. Stack allocation is no longer possible because the size and structure of an activation record would have to be dynamically modified. Dynamic arrays must therefore be allocated on the heap, while a pointer to the start of the array remains stored in the fixed-length part of the activation record.

Arrays in Java

In Java, an array is not created at the same time as the declaration of a variable of array type. The declaration `int [] v;` introduces only the name `v`. It is not associated with any data structure (more precisely, this is the same as for any declaration of a variable of class type). The data structure is created on the heap using the predefined `new` operation: `v = new int[10];` creates in the heap a new array of 10 integers and assigns a reference to them to the name `v`. Multidimensional arrays are not stored contiguously. Using a technique similar to that discussed below for dynamic sequences, they are stored using multiple layers of indirect pointers (see Sect. 8.4.7).

An array in Java is an object in the strict sense of the term as understood for that language. It has unlimited lifetime.

To conclude the argument, let us argue that, in addition to the static or dynamic nature of the shape, the decision on allocation of an array must also take its lifetime into account. An array allocated in an activation record has a lifetime limited to that of the block in which it is declared. In languages that allow the creation of arrays with unlimited lifetimes (for example, Java) they must be allocated on the heap.

Dope Vectors

The descriptor of an array whose shape is not known statically (and which can either be fixed at execution time or be completely dynamic) is called a *dope vector*. A dope vector, which is usually allocated in the fixed-length part of an activation record contains the following components:

- A pointer to the first location at which the array is stored;
- All the dynamic information required to perform the calculation in expression (8.1): the number of dimensions (also called the *rank* of the array), the value of the lower limit of each dimension (the L_j), the occupation of each dimension (the S_j).

If some of these values are known statically, they are not explicitly stored. The dope vector is initialised when the array is created (using appropriate calculations of the values S_j). To access an element of the array, the dope vector is accessed (by an offset from the frame pointer), expression (8.1) is computed and is added (via an indirect access) to the address of the start of the array.

8.4.5 Sets

Some programming languages allow the definition of set types, whose values are composed of subsets of a base type (or universe), usually restricted to an ordinal type. For example, we may have:

```
set of char S;
set of Dwarf N;
```

We have a variable `S` which is a subset of the characters and a variable `N` which will contain a subset of the type `Dwarf`, as defined on Sect. 8.3.8. Languages allowing

sets provide appropriate syntax for assigning a specific subset to a variable, for example:

```
N = {Grumpy, Sleepy};
```

The possible operations on values of set type are the membership test (the test that an element belongs to a set) and the usual set-theoretic operations of union, intersection and difference; complement is not always provided.

A set may be represented as a bit vector (the *characteristic* vector of the set) whose length is the same as the cardinality of the base type. For example, a subset of `char`, in an implementation that uses the ASCII 7-bit code, would be represented as 128 bits. If, in the characteristic vector, the j th bit is set, it indicates that the j th element of the base type (in the standard enumeration) is in the set; otherwise, a bit equal to zero indicates that the corresponding element is not in the set. Although this representation permits the highly efficient execution of set-theoretic operations (as bitwise operations on the physical machine), it is clear that it is completely inappropriate for subsets of types whose base type has a cardinality greater than a few hundred. To obviate this problem, languages often limit the types that can be used as base for set types. Or rather, they select other representations (for example, hash tables), which will support the representation of the set in a more compact fashion.

8.4.6 Pointers

Some languages permit the direct manipulation of l-values. The corresponding type is called a pointer type—if T is a type, it is possible to define the type of “pointers to (variables/values of type) T ”. It should be noted that, in this context, we will be interested in pointers as far as they are explicitly present in the language, in contrast to the obvious presence of addresses (pointers) in the abstract machine.

In a language with a reference model for variables (see Sect. 6.2.1), pointer types are not necessary (or required). Every variable is always a reference, or rather it is considered as an l-value, even if this value cannot explicitly be manipulated. In languages with modifiable variables, on the other hand, pointers provide a way of referring to an l-value without automatically dereferencing it. In such languages, one of the uses for pointers is to construct values of recursive type as linked structures.

In our pseudo-language, we use T^* to indicate a pointer type to objects of type T and therefore we declare a pointer as⁶:

```
T* p;
```

⁶ The mother tongue C programmer would write the example with a different distribution of spaces: `T *p`, which is read as `*p` is a pointer to `T`. In C, `*` is a modifier of the variable and not of the type (which is apparent when you introduce two pointers in the same declaration: C requires `int *p, *q`, while we would write `int* p, q`; for uniformity with the other types). Although the C approach seems simpler (particularly when considered with the dereferencing operator), it is semantically more correct (and more uniform with respect to other languages) to see `*` as a type modifier and not a variable modifier.

The values of type T^* are pointers (from an implementation viewpoint: addresses) to memory locations (that is modifiable variables) which contain values of type T . It is not always the case such pointers can refer to arbitrary locations; some languages require that pointers point only to objects allocated in the heap (this is the case with Pascal, in its descendants and in some earlier versions of Ada). Other languages, on the other hand, allow pointers also to point to locations on the system stack or into the global area (this is the case with C++ and later versions of Ada).

There usually exists a canonical value, which is an element of type T^* , for every T , which indicates the null pointer, that is which points to no value. We write `null` for this value. The operations permitted on values of pointer type are usually creation, equality (in particular, equality to `null`), dereferencing (access to the object begin pointed to).

The commonest way of creating a value of pointer type is to use a predefined construct or a library function which allocates an object of an appropriate type on the heap and also returns a reference to the object. This is the purpose of `malloc` in C:

```
int* p;  
p = (int *) malloc (sizeof (int));
```

For heap management in the presence of explicit allocation, we refer to our discussion in Sect. 5.4.

In some languages, it is possible also to create pointers by applying special operators which return the storage address of an object stored in memory. Again we use C as the model for our examples. C uses the `&` operator to obtain the address of an entity:

```
float r = 3.1415;  
float* q;  
q = &r;
```

The pointer `q` now points to the location that contains the variable `r`. This is also an example of a pointer pointing to a location on the stack and not in the heap.

Dereferencing a pointer is often indicated by some operator, for example, the `*` in C. Continuing the example above:

```
*p = 33;  
r = *q + 1;
```

Here, the first line assigns the value 33 to the object pointed by `p` (which is in the heap). The second line assigns to `r` the value 4.1415 (on the stack). It can be seen that, as a side-effect, the second assignment also modifies the value pointed by `q` (but not `q` itself). It can also be observed that the l-value/r-value distinction for variables in an assignment still remains valid even for a dereferenced pointer. When `*p` is on the left of an assignment, it indicates the l-value that is obtained by dereferencing `p` (that is, the assignment will work on the address contained in the pointer); when `*p` is on the right of an assignment, it indicates the r-value of the same location.

From the pragmatic viewpoint, pointers play a key role in the definition of so-called *recursive* structures such as lists, trees, etc. Some languages directly allow

the definition of recursive types, as will be seen in Sect. 8.4.9. In languages with pointers, recursive data structures can be defined in a natural fashion. For example, the type for the lists of integers⁷ can be defined as:

```
type int_list = element*;
type element  = struct {int val;
                        int_list next;};
```

It can be seen that, in order for such a definition to be legal in a hypothetical language, problems related to the use of a name before its definition (which we mentioned in Sect. 4.3.3) must be solved.

Pointer Arithmetic

In C and in its descendants, in addition to the operations that we have just discussed, it is possible to perform some arithmetic operations on pointers. It is possible to increment a pointer, subtract one pointer from another (to get an offset constant), add an arbitrary value to a pointer. This complex of operations goes under the name of pointer arithmetic. The semantics of these operations, although denoted by the standard arithmetic operators, must be understood with reference to the pointer type on which they operate. By means of an example, let us consider the following fragment:

```
int* p;
int* c;
p = (int *) malloc(sizeof(int));
c = (char *) malloc(sizeof(char));
p = p+1;
c = c+1;
```

If the two `mallocs` return addresses (i for `p` and j for `c`), at the end of the fragment, the value of the two pointers is *not* $i + 1$ and $j + 1$, but, rather, $i + \text{sizeof}(\text{int})$ and $j + \text{sizeof}(\text{char})$. And thus, analogously, for other increment and decrement operations.

It should be clear that pointer arithmetic destroys every hope of type safety in a language. There is no guarantee whatsoever that, at any time during the execution of a program, a variable declared as a pointer to an object of type `T` will still point to an area of store in which a value of type `T` is actually stored.

Deallocation

We have already seen that one of the most common methods for creating values of a pointer type is to use a function defined as standard in the language to allocate an object on the heap and also return a pointer to it. The deallocation of memory can be explicit or implicit.

⁷ As we saw in Sect. 4.3.3, a list is a variable-length data structure formed from a possibly empty ordered sequence of elements of some type, in which it is possible to add or remove elements and where only the first element can be directly accessed.

In the case of implicit deallocation, the language does not provide the programmer with mechanisms to reclaim memory. The programmer continues to request allocations while there is still available heap. When the heap memory is full, however, the computation need not abort: already allocated values might not be accessible so they can be reused. Consider indeed the following fragment:

```
int* p = (int *) malloc(sizeof(int));
*p = 5;
p = null;
```

The last command, which assigns null to `p`, destroys the only pointer through which this previously allocated area can be accessed. It is possible to equip the abstract machine with a mechanism to recoup such pieces of memory. This technique, which is called *garbage collection*, is a subject that has been studied a great deal when implementing programming languages. We will examine it in Sect. 8.12.

In the case of explicit deallocation, the language makes available a mechanism with which the programmer can release the memory referred to by a pointer. In C, for example, if `p` points to an object on the heap that was previously allocated by `malloc`, calling the function `free` on `p`, will deallocate the object pointed to by `p` (the store will be put back on the free list if the technique described in Sect. 5.4 is used). It is good practice to assign the value `null` to `p` when this is done. It is a semantic error (with unpredictable result) to invoke `free` on a pointer which does not refer to an object allocated using `malloc` (this can happen if either pointer arithmetic has been used or if the pointer was created using the operator `&`).⁸ As we saw in Chap. 4, the most important problem that is raised by deallocation is that it is possible to generate dangling references; that is pointers with a value other than `null` which point to storage that is no longer allocated. The simplest example is probably the following:

```
int* p;
int* q;
p = (int *) malloc(sizeof(int));
*p = 5;
q = p;
free(p);
p = null;
...           //
...           // series of commands not modifying p or q
...           //
print(*p);    // error that may be trapped
print(*q);    // error that cannot be trapped
```

Figure 8.5 shows the situation in memory immediately after line 5 (left) and after line 7 (right). The pointer `q` has a value that is not `null`. It points to an area of

⁸ The point is that `malloc` does not only allocate the space required for the object required, but also a descriptor to the data. The descriptor contains the size of the allocated block and possibly other information. It is by accessing such data that `free` (which takes a single pointer as parameter) can determine what is to be deallocated.



Fig. 8.5 Dangling references

memory that could by now be allocated to other data. We can certainly assume that the abstract machine checks and signals an error if we try to de-reference a `null` pointer (as on line 11).⁹ However, the de-referencing of `q` in line 12 cannot be signalled and can be the cause of errors that are as devastating as they are difficult to find.

In languages which allow pointers to refer to objects on the stack, dangling references can also be generated without explicit deallocation. It is sufficient to store the address of a variable local to a function in a nonlocal environment.

```
{ int* p;
  void foo() {
    int n;
    p = &n;
  }
  ...
  foo();
  ...    // here p is a dangling reference
}
```

If a language allows dangling references to happen, it is obvious that it cannot be type safe. If a language does not allow pointers to data on the stack, dangling references can be avoided by not having deallocation, both at the level of program design (by using a language with implicit pointer deallocation and garbage collection), or by deallocating nothing even if the programmer requests it: this is the case with some early implementations of Pascal, where the deallocation function `dispose` was implemented as a function with an empty body.

If, though, a language provides explicit deallocation, or pointers to the stack, some techniques are discussed in Sect. 8.11 with which dangling pointers can be rendered harmless, thereby restoring type safety.

8.4.7 Sequences

Some modern languages provide types for extensible sequences of values, akin to arrays with dynamic shape that we briefly discussed at the end of Sect. 8.4.4. Sequences in Python (lists and tuples) or Ruby (arrays) are heterogenous, in the sense that the elements of a sequence may be of different types. An element of a

⁹ This can happen often without affecting efficiency, by making use of mechanisms for protecting addresses on the underlying physical machine.

Arrays and Pointers in C

In C, arrays and pointers are considered under certain circumstances to be equivalent. The declaration of an array introduces a name which can be used as a pointer to the whole array. Conversely, if an integer pointer refers to an array, it can also be used as the name of an array:

```
int V[10];
int* W;
W = V;           // W points to the start of array V
V[1] = 5;
W[1] = 5;
*(W+1) = 5;
*(V+1) = 5;
```

The last four assignments in this example are all equivalent. They assign the value 5 to the second element of the array pointed to by both *V* and *W*. (It can be seen how pointer arithmetic is natural here.) In the case of a multidimensional array (stored in row-major order), pointer arithmetic allows various combinations of operation. In the scope of the declaration `int Z[10][10]`, the following expressions are equivalent: `Z[i][j]`, `(* (Z+1)) [j]`, `* (Z[i]+j)`, and `* (* (Z+i)+j)`.

The equivalence of arrays and pointers is essential when passing parameters. It will be recalled that C has only call by value. When it passes a vector, it always passes a pointer (by value), never the array itself. The formal parameter can be declared as an array (`int A[]`) or as a pointer (`int *A`). In the case of multidimensional arrays, it is required only to specify the number of elements of the dimensions other than the first in the formal parameter (for example `A[][10]`, or even `(*A)[10]`). This information is necessary for the static generation of the correct code for accessing elements of the vector.

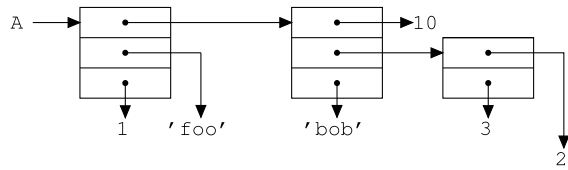
Finally, C permits storage of multidimensional array as a vector of pointers to arrays; for this organisation by row-pointer, see Exercise 2.

sequence can be a sequence, too, thus allowing for nested structures. Sequences share with arrays a fast access to an element *by index*, but they may allow also for deletion and addition of elements,¹⁰ and, more generally, dynamic shrinking and growing of the number of elements (the *length* of the sequence.) Moreover, ranges (Sect. 8.3.9) may be used to dynamically select slices out of a sequence.

Flexibility of use is obviously paid for in terms of efficiency. Any element of a sequence can be an arbitrary complex value, and the type of that value may change dynamically. It follows from the discussion on dynamic shapes that sequences cannot be directly allocated in an activation record and, moreover, values in a sequences cannot be stored into contiguous memory. What is stored contiguously is a vector of references (pointers) to the actual values, thus introducing several layers of indirection—see Fig. 8.6 for a possible storage layout for the sequence `A = [[10, [2, 3], 'bob'], 'foo', 1]`, where we write `[1, 'a']` for the two elements sequence whose elements are 1 and 'a'. Deletion or addition of an element at a given index (and, more generally, operations which change the length of a sequence) cannot be implemented in constant-cost. To insert a new element into an *n* elements sequence, all the *n* element references must be moved (copied),

¹⁰ Python's list are mutable, while tuples are immutable.

Fig. 8.6 Possible storage arrangement for the sequence
 $A = [[10, [2, 3], 'bob'], 'foo', 1]$



in order to restore the new sequence as a contiguous block of memory references. To avoid repeated allocations (and thus repeated copies) the abstract machine may over-allocate contiguous blocks for memory references, which may be used only later during execution. With this technique, the cost of adding an element at the end of a sequence (“appending an element”) may be reduced to constant cost under amortized analysis. Adding or deleting elements at a generic index, though, remains an operation whose cost is linear in the length of the sequence.

8.4.8 Recursive Types

A recursive type is a composite type in which a value of the type can contain a (reference to a) value of the same type. In many languages recursive types can be assimilated to (and in effect are) records in which a field is of the same type as the record itself. The simplest example is perhaps that of a list of integers, which we present in our pseudo-language (adapting Java’s notation):

```

type int_list = { int val;
                  int_list next; };
  
```

To end the recursion, the language can provide a special value, for example `null`, which belongs to any (recursive) type and which we can imagine as having the empty value. A value of type `int_list` is therefore a pair: the first element is an integer, the second is a value of type `int_list`, and so on, until one of these `int_list` values is `null` (see the example in Fig. 8.7).

Figure 8.8 shows the definition of a recursive type for binary trees of characters. It also shows a value and the usual graphical representation for trees.

As can be seen from these very simple examples, the possibility of defining recursive types makes the type system of a language flexible and powerful (see Exercise 4). The operations permitted on the values of recursive types are the selection of a component and equality test for `null`.

If the language admits modifiable objects (which is the case for all imperative languages), it is also possible to construct circular values. In purely functional languages, the values of a recursive type are always in the form a tree.

Recursive types are usually represented as structures in the heap. A value of a recursive type corresponds to a linked structure. Every element of the structure is

```

{2, {33, {1, {4, {3, {1, {21, null}}}}}} }
  
```

Fig. 8.7 A value of type `int_list`

```

type char_tree = {char val;
                  char_tree left;
                  char_tree right;};

{A, {B, {C, null, null}, {D, {E, null, null}, null}}, {F, null, null}}

```

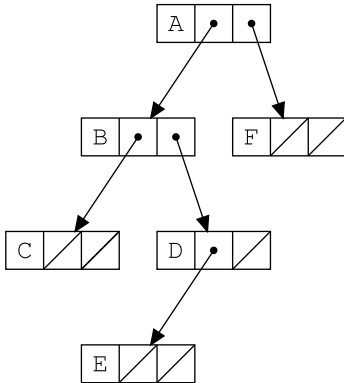


Fig. 8.8 Binary tree over char

composed of a record in which the recursive reference is implemented as an address to the next record (the one created in the previous recursive step). The representation in the heap of a binary tree shown in Fig. 8.8 corresponds exactly to its graphical representation.

The creation of values of a recursive type (so that they can be associated with a name) is an operation which varies considerably from language to language. In many functional languages, the values of a recursive type are expressible. There exist explicit syntactic constructs for designating their values. An example could be the expression in Fig. 8.7 or in Fig. 8.8 (see, also, the box on ML). In imperative languages, on the other hand, the values of recursive types are constructed by explicit allocation of their components in the heap. To continue our analogy with Java, we have a predefined constructor, `new`, which allows us to allocate instance of the type in the heap:

```
int_list l = new int_list();
```

The part on the right of the assignment symbol creates on the heap a record of the form specified by `int_list` and assigns the name `l` to a reference to this record. At this point, its fields can be initialised. For example:

```
l.val = 2;
l.next = null;
```

It remains to discuss the management of the heap when there are values of recursive type. They are in fact dynamically allocated (explicitly as in Java, or implicitly as in ML) but languages which admit recursive types as primitive do not in general permit explicit deallocation of the values thus created. As in the case of implicit allocation

Recursive Types in ML

ML is an important functional language. It is equipped with an extended type system. It is a type-safe language with many interesting properties. The language allows the definition of explicitly recursive types using *constructors*, which are introduced as part of the definition of the type. By way of an example, our definition of list of integers can be written as (ignoring the fact that ML has lists as a predefined type):

```
datatype int_list = Null | CONS of int * int_list;
```

In this definition, `Null` and `CONS` are constructors, that is function symbols to which there corresponds no code, but which serve only to construct, syntactically, terms of the desired type (in this case `int_list`). The vertical line is read as “or”. A value of type `int_list` is the constant `Null`, or is a pair (denoted by the constructor `CONS`) composed of an `int` and an `int_list`. Therefore `CONS (2, Null)` is a simple value of type `int_list`. Figure 8.9 shows the ML expression which corresponds to the value in Fig. 8.7.

The language permits the definition of ill-founded recursive types. For example the following definition introduces the type `empty`; it is so-called because it is impossible to construct values of this type:

```
datatype empty = Void of empty;
```

Indeed, there is no basic constructor which will allow us to construct a value of type `empty` without presupposing another such value.

```
CONS (2, CONS (33, CONS (1, CONS (4, CONS (3, CONS (1, CONS (21, Null))))))
```

Fig. 8.9 A value of type `int_list` in ML

that we discussed in the context of pointers, it is necessary to provide the abstract machine with mechanisms for garbage collection in order to reclaim the chunks of memory that can no longer be accessed.

8.4.9 Functions

Every high-level programming language supports the definition of functions (or procedures). Few traditional languages permit the denotation of function types (that is, give them a name in the language). Let f be a function defined as:

```
T f (S s) { ... }
```

It has the type $S \rightarrow T$; more generally, a function with header

```
T f (S1 s1, ..., Sn sn) { ... }.
```

is of type $S_1 \times \cdots \times S_n \rightarrow T$.

The values of a functional type are denotable in every language but not always they are expressible (or storable). In addition to definition, the principal operation permitted on a value of functional type is *application*, that is the invocation of a function on some arguments (actual parameters).

Relations of Preorder and Equivalence

A binary relation, $*$, on a set, D , is a subset of the cartesian product of D with itself: $* \subseteq D \times D$. We write $c * d$ for $(c, d) \in *$. A relation is *reflexive* when, for every $d \in D$, it is true that $d * d$. It is symmetric if, for all $c, d \in D$, if $c * d$ holds, then also $d * c$ holds. It is transitive if, for every $c, d, e \in D$, if $c * d$ and $d * e$, then $c * e$. It is antisymmetric if, for every $c, d \in D$, if $c * d$ and $d * c$, then c coincides with d .

If a relation is reflexive and transitive, it is said to be a *preorder*. If a preorder is also symmetric, it is an *equivalence*. If, on the other hand, a preorder is also antisymmetric, it is a *partial order*.

In most modern languages the type of functions has the same status of any other type: we may have expressions having as value a function (Sect. 7.2.3), it is possible to pass functions as arguments to other functions (Sect. 7.2.1), and to define functions which return functions as results (Sect. 7.2.2). In such *higher order* languages, a function is not just a piece of code equipped with its local environment, but may be handled like other data. We will see in Chap. 11, the outline of a possible abstract machine for doing this.

8.5 Equivalence

Having analysed the principal types that we find in programming languages, the time has come to discuss the rules which govern the correctness of programs with respect to types. The first set of these rules deals with the definition of when two types, which are formally different, are to be considered interchangeable. The rules define an equivalence relation between types. If two types are equivalent, every expression or value of one type is also an expression or value of the other, and vice versa. In the next section, we will discuss compatibility rules which specify when a value of one type can be used in a context in which the value of another type would be expected (but not conversely).

Let us recall that a type is defined in the following form (the details will vary from language to language):

```
type newtype = expression;
```

Languages interpret a definition like this in two different ways which lead to two distinct rules for type equivalence:

- The definition of a type of can be *opaque*, in which case we have equivalence *by name*.
- The definition of the type can be *transparent*, in which case we have *structural* equivalence.

8.5.1 Equivalence by Name

If a language uses opaque type definitions, each new definition introduces a new type which is different from every preceding one. In this case, then, we have equivalence by name, as defined below.

Definition 8.2 (*Equivalence by name*) Two types are equivalent by name only if they have the same name (so a type is equivalent only to itself).

By way of example, let us consider the following definitions of types:

```
type T1 = 1..10;  
type T2 = 1..10;  
type T3 = int;  
type T4 = int;
```

Four distinct types are introduced. There is no equivalence between them.

However, equivalence by name is too restrictive. Some languages adopt the rule of *weak* (or *lax*) equivalence by name. The simple renaming of a type does not generate a new type but only an alias for the same one. In the example above, using a weak equivalence by name, the types T3 and T4 are equivalent, but types T1 and T2 remain distinct.

Under equivalence by name, each type has a unique definition which occurs at a unique point in the program, a fact which simplifies maintenance. Despite its simplicity, from the programming point of view, equivalence by name is the choice that most respects the intentions of the designer. If the programmer has introduced two different names for the same type, they will have their reasons which the language respects by maintaining two different types.

Note that equivalence by name is defined with reference to a specific, fixed program and it does not make sense to ask whether two types are equivalent by name “in general”.

8.5.2 Structural Equivalence

A type definition is transparent when the name of the type is just an abbreviation for the expression defining the type. In a language with transparent declarations, two types are equivalent if they have the same structure, that is if, substituting names for the relevant definitions, identical types are obtained. If a language uses transparent type definitions, the equivalence between types that is obtained is said to be *structural equivalence*.

We can give a more precise recursive definition of structural equivalence.

Definition 8.3 (*Structural Equivalence*) Structural equivalence of types is the (least) equivalence relation satisfying the following properties:

- The name of a type is equivalent to itself;
- If a type `T` is introduced with the definition `type T = expression`, `T` is equivalent to `expression`;
- If two types are constructed by applying the same type constructor to equivalent types, then the two types are equivalent.

By way of example, consider the following definitions:

```
type T1 = int;  
type T2 = char;  
type T3 = struct{  
    T1 a;  
    T2 b;  
}  
type T4 = struct{  
    int a;  
    char b;  
}
```

`T3` and `T4` are structurally equivalent. Some aspects of the definition of equivalence that we have given are deliberately vague or imprecise. For example, it is not clear whether the three following types are equivalent:

```
type S = struct{  
    int a;  
    int b;  
}  
type T = struct{  
    int n;  
    int m;  
}  
type U = struct{  
    int m;  
    int n;  
}
```

They result from the application of the same constructors, but they have been given names (or field order) that are different. A further subtle question concerns structural equivalence and recursive types:

```
type R1 = struct{  
    int a;  
    R2 p;  
}  
type R2 = struct{  
    int a;  
    R1 p;  
}
```

Intuitively, we can think that `R1` and `R2` equivalent, but the type checker will be unable to solve the mutual recursion involved in these definitions (showing the

equivalence between R1 and R2 requires a mathematical argument of some sophistication).

Structural equivalence allows us to speak about equivalent types in general, not just those in a specific program. In particular, two equivalent types can always be substituted for each other in any context without altering the meaning of the program in which the substitution occurs (this general property of substitution is often known as *referential transparency*).

It will come as no surprise to the reader to learn that existing languages almost always use some form of combination or variant of the two equivalence rules that we have just defined. Java uses equivalence by name, except for arrays, for which it uses structural equivalence; C uses structural equivalence for arrays and for types defined using `typedef`, but not when it involves records (`structs`) and unions, for which it always uses equivalence by name; C++ uses equivalence by name (except when it inherits from C); ML uses structural equivalence, except for types defined using `datatype`; Modula-2 inherits from Pascal its use of equivalence by name, but Modula-3 uses structural equivalence; and so on.

8.6 Compatibility and Conversion

The relationship of compatibility between types, which is weaker than equivalence, allows a value of one type to be used in a context where another type has been requested.

Definition 8.4 (*Compatibility*) We say that type T is compatible with type S , if a value of type T is permitted in any context in which a value of type S would be admissible.

In many languages, it is the rule of compatibility (and not that of equivalence) which controls the correctness of assignment (the type of the right-hand component must be compatible with that of the left-hand one), parameter passing (the type of the actual parameter must be compatible with that of the formal parameter), etc. It is clear that two equivalent types are also compatible with each other but in general the relation of compatibility is not symmetric: the canonical example is the compatibility that exists in many languages (but not in all: ML and Java, for example, do not admit it) between `int` and `float` (but not vice versa). The compatibility relation is therefore a pre-order (that is a reflexive and transitive relation) that is not symmetric, but almost never an ordering (that is it is not symmetric, but not anti-symmetric either). In fact, in a language which admits structural equivalence, two types that are structurally equivalent are compatible but not equal.

The compatibility relation, more than that of equivalence, varies enormously between languages. Without pretending to be complete, let us list some of the possible concepts that we can encounter in order of generality (that is, of the “generosity” of the compatibility relation). A type T can be compatible with S when:

1. Types T and S are equivalent; it is the most restrictive version of compatibility.
2. The values of T form a subset of the values of S : this is a case of an interval type, contained in its own base type (and therefore compatible with it). Usually, the values of T and values of S have the same internal representation.
3. All the operations on the values of S are also permitted on values of T . This is probably the most interesting case of compatibility and can be declined in several different variants.

A first variant is *structural compatibility*¹¹ and is present in statically typed languages. We present it by means of a simple example on record types. Let us assume that we have declared two record types:

```
type S = struct{
    int a;
}
type T = struct{
    int a;
    char b;
}
```

The only possible operation on values of type S is the selection of the field with name a . This also makes sense for values of type T (on which also the selection of the field b makes sense). There is no inclusion relation between values of T and those of S ; still it makes perfect sense to use a value of type T whenever a value of type S is required. Therefore, something like

```
int f(x:S){
    return x.a
}
print(f({a=10,b=20}))
```

is correct under structural compatibility and prints 10. Scala allows structural compatibility and the following is correct code:

```
import reflect.Selectable.reflectiveSelectable
def f(x:{def a:Int}):Int={
    x.a
}
f(new {def a=20;
    def b=100}) // evaluates to 20: scala.Int
```

A second variant on this form of compatibility is called *duck typing*¹² and is present in some dynamically typed languages, for instance Python. When a value is used in a certain context, the value's type itself is not important. What it matters is that the value supports all operations that are used on it. We may recast the previous example in Python as follows:

¹¹ Structural compatibility is sometimes called *static duck typing*, to distinguish it from the similar mechanism in dynamically typed languages, which we will introduce next.

¹² From the old saying: "If it walks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck".

```
def f(x):
    return x.a
class T:
    a=20
    b=100
f(T()) # evaluates to 20
```

In duck typing there is no formal reference to the actual types of the involved values. There is only a (dynamic) check that all operations are defined on certain values.

A final variant is when compatibility is expressed as a *subtype relation*: T is a subtype of S and hence we may use any T value whenever an S value is required. We will discuss the case of subtyping in Sects. 8.7.3 and 10.2.4 (for object-oriented languages). Sometimes structural compatibility and subtypes can both be present and combined. In the context of subtyping, this form of compatibility takes the name of *Liskov substitution principle*.¹³ It is a form of *behavioural* subtyping: if T is a subtype of S , then we may use an object of type T wherever one of type S is present in a program, *without altering the properties of that program*.

4. The values of T correspond *in a canonical fashion* to values of S . In addition to the case outlined in the previous point, this is the case that we have already mentioned of `int` being compatible with `float`. (This is not an instance of point (2), above, because, as far as the implementation is concerned, the values of the two types are distinct).
5. Values of T can be made to correspond with some values of S . Once the requirement of the canonicity of correspondence has been dropped, every type can be made to be compatible with another defining a (conventional) way to transform one value of T into one of S . Using this very broad notion of compatibility, `float` can be made compatible with `int` by arbitrarily defining the conversion procedure (for example, rounding, truncation, etc.).

In order to manage this collection of different interpretations as a unified whole, we introduce the notion of *type conversion*. This notion will be stated in two distinct ways:

- *Implicit conversion* (also called *coercion* or forced conversion). This is the case in which the abstract machine inserts a conversion when there is no such indication in the high-level language;
- *Explicit conversion* (or *cast*). This is when the conversion is indicated in the text of the program.

¹³ From Barbara Liskov, the recipient of the Turing Award in 2008.

Casts that Do not Change the Representation

In some situations, especially in systems programming, it could be useful to change the type of an object without changing its representation in memory. This kind of conversion is obviously forbidden in type-safe languages, but is, though, permitted in many languages (these are often said to be “unchecked” (ADA) or “non-converting” type casts).

An example in C using the addressing (&) and de-reference (*) operators is:

```
int a = 233;
float b = *((float *) &a);
```

First of all, the address of the integer variable `a` is taken, then it is converted explicitly into a pointer to a `float` using the cast `(float *)`; finally, it is dereferenced and assigned to the real value `b`. All this machinery is merely annotation for the compiler which does not result in any conversion by the machine. After the assignments we have two locations containing identical bit strings, one interpreted in two's complement (and representing the integer 233), one interpreted as the IEEE 754 representation for a floating point number (provided that integer and `float` are represented by the same number of bytes).

It is clear that a conversion which does not modify the stored representation is always an extremely delicate operation, as well as being a potential source of hard-to-fix errors.

Coercions

In the presence of compatibility between the types `T` and `S`, the language allows a value of type `T` to occur where a value of type `S` is expected. When this happens, the compiler and/or abstract machines inserts a type conversion between `T` and `S`. We will call this a *type coercion*.

From a syntactic viewpoint, coercion has no significance other than annotating a compatibility. From the implementation viewpoint, however, a coercion can correspond to different things depending upon the notion of compatibility adopted:

1. Type `T` is compatible with `S` and have the same storage representation (at least for values of `T`). In such a case, coercion remains a syntactic matter and causes no code to be generated.
2. Type `T` is compatible with `S` but there exists a canonical way to transform values of `T` into values of `S`. In this case, the coercion is executed by the abstract machine, which applies precisely the canonical conversion. For example, in the case of `int` and `float`, the abstract machine inserts code to transform (at runtime) the representation of an integer (e.g., in two's complement) into one for a floating point (e.g., in IEEE 754).
3. The compatibility of `T` with `S` is based on an arbitrary correspondence between values of `T` with those of `S`. In this case, too, the abstract machine inserts code that performs the transformation. This is the case, also, of all those situations where `T` and `S` have the same representation, but `T` is a superset of `S`. For example, `T` is `int` and `S` is an integer interval¹⁴ (note that we are in the symmetric case to the “canonic” case, considered in point 1, in which $T \subseteq S$). Coercion in this

¹⁴ Almost all languages with interval types allow this kind of compatibility for the obvious reason of programming flexibility.

case does not transform the representation but dynamically checks (at least if the language is supposed to be type safe) that the value of T belong to S .

Languages with strong type checking tend to have few coercions (that is few compatibilities) of the last kind. On the other hand, in a language like C, the type system is designed to be by-passed and so permits numerous coercions (from characters to integers, from long reals to short, from long integers to short reals, and so on).

Explicit Conversions

Explicit conversions (or *casts*, using the name by which they are known in C and in other languages) are annotations *in the language* which specify that a value of one type must be converted to that of another type. In this case, as well, such a conversion can be either just a syntactic marking or can correspond to code that is executed by the abstract machine, using the scheme for coercions that we have already discussed. In our pseudo-language, we will follow the conventions of the C language family and will denote a cast using brackets:

```
S s = (S) t;
```

Here we are assigning a value t , having converted it into type S , to a variable of type S . Not every explicit conversion is permitted, only those for which the language knows how to implement the conversion. It is clear that we can insert a cast where a compatibility exists. In this case explicit conversion is syntactically useless but must be advised for documentation purposes. Languages with few compatibilities make available, in general, many explicit conversions, which the programmer can use to annotate the program where a change of type is necessary.

Generally, modern languages tend to prefer casts over coercions. They are more useful as far as documentation is concerned; they do not depend upon the syntactic context in which they appear, and, most importantly, they behave better in the presence of overloading and polymorphism (which we will consider shortly).

8.7 Polymorphism

A type system in which any language object (value, function, etc.) has a unique type is said to be *monomorphic*.¹⁵ Here we will be interested in the more general concept given in the next definition.

Definition 8.5 A type system in which the same object can have more than one type is said to be *polymorphic*.¹⁶ By analogy, we will say that the object is polymorphic when the type system assigns more than one type to it.

¹⁵ This is a word derived from the Greek and means “has only a single (*mono*) form (*morphos*)”

¹⁶ “Which has many (*poly*) forms.”

Even the more conventional languages contain limited forms of polymorphism. The name `+` in many languages is of type $\text{int} \times \text{int} \rightarrow \text{int}$, as well as of type $\text{float} \times \text{float} \rightarrow \text{float}$. The value `null` has the type T^* for every type T . The function `length` (which returns the number of elements of an array) is of type $T[] \rightarrow \text{int}$ for every type T . The list could be continued. In conventional languages, however, it is not in general permitted for the user to define polymorphic objects. Let us take as an example, a language with a type system that is fairly inflexible (for example, Pascal or Java but to some extent C also) and let us assume that we want to write a function that sorts a vector of integers. The resulting function could have the signature:

```
void int_sort(int A[])
```

If we now need to sort a vector of characters, we need to define *another* function

```
void char_sort (char C[])
```

which is wholly identical to the previous example except in the type annotations. A polymorphic language would allow the definition of a single function

```
void sort(<T> A[])
```

where `<T>` denotes a generic type which will be specified at a later stage.

In this section, we will analyse the phenomenon of polymorphism. We will start by distinguishing three different forms:

- *Ad hoc polymorphism*, also called *overloading*.
- *Universal polymorphism*, which can be further divided into:
 - Parametric polymorphism, and
 - Subtyping or inclusion polymorphism.

8.7.1 Overloading

Overloading, as its other name (*ad hoc* polymorphism) suggests, is really polymorphism in appearance only. A name is *overloaded* when it corresponds to more than one object and context information must be used to determine which object is denoted by a specific instance of that name. The most common examples of this are:

- The use of the name `+` to indicate either integer or real addition (and sometimes it also denotes concatenation of character sequences).
- The ability to define more than one function (or constructor) with the same name but whose instances differ in the number or type of their parameters.

In the case of overloading, therefore, a single name corresponds to more than one object (in fact, to a finite number of objects). If it is the name of a function, it will be associated with different pieces of code. The ambiguity of the situation is solved statically using type information that is present in the context. From the

Polymorphic Types and Universal Quantification

Rather than writing a polymorphic type between angle brackets, a more uniform notation (one which is more suggestive, as well as mathematically more accurate) is that which uses universal quantification. Instead of $\langle T \rangle \rightarrow \text{void}$, we write $\forall T. T[] \rightarrow \text{void}$.

This is a notation that is well-suited to the description of all the variations on polymorphism which have been proposed; in Sect. 8.7.3 (and again in Sect. 10.4.1) it will be used to analyse subtype polymorphism.

conceptual viewpoint, we can imagine a kind of pre-analysis of the program which solves cases of overloading by substituting a unambiguous name which uniquely denotes a unique object for each symbol that is overloaded. Overloading is therefore a sort of abbreviation at the syntactic level. It vanishes as soon as we introduce additional information.

Overloading should not be confused with coercion. They are two different mechanisms which solve different problems. On the other hand, in the presence of coercions, it might be not at all clear how a case of overloading should be solved. Consider for example the following expressions:

```
1 + 2
1.0 + 2.0
1 + 2.0
1.0 + 2
```

This quartet of expressions can be interpreted in several ways. First, $+$ is overloaded with four different meanings. Second, $+$ is overloaded with two meanings (integer and real) and coercion is inserted in the final two cases. Next, $+$ merely denotes real addition and in the other three cases, coercions are employed. It is up to the definition of the language to determine which of these interpretations is correct.

8.7.2 Universal Parametric Polymorphism

We begin with a definition that is relatively imprecise but adequate for our needs.

Definition 8.6 A value exhibits universal parametric polymorphism (or parametric polymorphism, more briefly) when it has is an infinite number of different types which can be obtained by instantiating a single schema of general type.

A universal polymorphic function is therefore composed of a single piece of code which operates uniformly on all the instances of its general type (because information about the type is not exploited by the algorithm which the function implements).

Among the examples that we discussed at the start of the section, there are some that fall into this category: the value `null`, which belongs to every type T^* (that is to every type that can be obtained by substituting an actual type for T); the function `void sort( $\langle T \rangle$  A[])`, which sorts an array of *any type whatsoever*.

Before presenting more examples, we need to extend the notation. Following on from what was suggested in Sect. 8.4.9, we can write the polymorphic type of `sort` as `<T>[] → void`. In this notation, we use angle brackets to indicate that `<T>` is not a usual type but a sort of parameter. By substituting any “concrete” type for the parameter, we obtain a specific type for `sort`. Each way to substitute a type for `<T>` corresponds to one of the infinite number of ways to apply the function: `int[] → void`, `char[] → void`, etc. With this notation, the type of `null` should be more correctly given as `<T>*`.

Let us give another example. A function which exchanges two variables of any type. As with `sort`, if it is written in a language without polymorphism, we would need a `swap` function for every possible variable type. Using universal polymorphism, and assuming that we have at our disposition call by reference, we can write:

```
void swap (reference <T> x, reference <T> y) {
    <T> tmp = x;
    x = y;
    y = tmp;
}
```

A polymorphic object can be *instantiated* to a specific type. The instantiation can happen in many different ways, according to the specific mechanism(s) for polymorphism provided by the language. The simplest model is that in which instantiation occurs automatically and is performed by the compiler or the abstract machine:

```
int* k = null;                                1
char v, w;
int i, j;                                      3
...
swap(v, w);                                   5
swap(i, j);
```

Without requiring additional annotations, on line 1, using information from the context (the assignment is performed on the variable of type `int*`), the type checker instantiates the type of `null` as `int*`. On line 5, `swap` is instantiated to character and, on line 6, to integer.

Parametric polymorphism is general and flexible. It is present in programming languages in two notationally different forms called explicit and implicit parametric polymorphism.

Explicit Polymorphism

Explicit polymorphism is the kind we have discussed thus far. In the program, there are explicit annotations (our `<T>`) which indicate the types to be considered parameters. This is the kind of polymorphism present in C++ (using the concept of “template”) and in Java (from version J2SE 5.0 using the “generic” notion).

Implicit Polymorphism

Other languages (in particular functional languages like ML) adopt implicit parametric polymorphism, in which the programmer need not provide any type indications

and instead the type checker (or rather, more properly, the module performing type inference—see Sect. 8.9) tries to determine for each object the most general type from which the other types can be obtained by instantiation. The simplest example of a polymorphic function is that of the identity which, using a notation as close as possible to that which we have used this far, can be written as:

```
fun Ide(x){return x;}
```

We will use the word `fun` to indicate that we are defining a function. The rest is totally analogous to the definitions we have already seen, except that there are no type annotations, either for parameters or results. Type inference will assign to the function `Ide`, the type $\langle T \rangle \rightarrow \langle T \rangle$. If the actual parameter is of type X , the result will be of type X , for any type X . The application of a polymorphic function obviously takes place without type indication. `Ide(3)` will be correctly typed as a value of type `int`, while `Ide(true)` be typed as `bool`.

Let us consider here a final example of implicit parametric polymorphism combined with higher order:

```
fun Comp(f,g,x){return f(g(x));}
```

`Comp` applies the composition of its first two parameters (the functions) to the argument x . What is the type that can be inferred for `Comp`? Here is the most general possible:

$$(\langle S \rangle \rightarrow \langle T \rangle) \times (\langle R \rangle \rightarrow \langle S \rangle) \times \langle R \rangle \rightarrow \langle T \rangle$$

The inferred type is perhaps more general than the reader would have thought.

8.7.3 Subtype Universal Polymorphism

Subtype polymorphism is typically present in object-oriented languages. It is a more limited form of universal polymorphism than parametric polymorphism.

Indeed, here as well, a single object can be assigned an infinite number of different types. They are obtained by instantiating a single most general type. In this case, too, since it is a case of true polymorphism (i.e., universally quantified polymorphism), there is (at least conceptually) a single algorithm which is *uniform in the type*—that is, it does not depend upon the particular structure of the type—that can be instantiated to each of the infinite number of possible types.

However, in the case of subtype polymorphism, not all of the possible instantiations of the schema for the most general type are admissible. Instantiations are limited by a notion of *structural* compatibility between types, that is, using the concept of subtype.

To be more precise, let us assume that a subtype relation is defined over the language's types. We denote this relation by the symbol $<:$. Therefore, we read $C <: D$ as “ C is a subtype of D ”. For the time being, we can be content with an abstract concept, but in Sect. 10.2.4, we will flesh out this concept in terms of relations between classes in object-oriented languages.

Templates in C++

Parametric polymorphism is obtained in C++ by means of *templates*. These are program schemata which include parameters (of class or of type). The function `swap` could take the following form:

```
template < typename T >
void swap (T& x, T& y) {
    T tmp = x;
    x = y;
    y = tmp;
}
```

The operator, `&`, attached to the type of a formal parameter indicates that the parameter is passed by reference, a primitive parameter-passing mode in C++ (in which it differs from C). The instantiation of a template is automatic. The call `swap(x, y)` is instantiated to the common type of `x` and `y`.

Definition 8.7 A value exhibits subtype (or bounded) polymorphism when there is an infinity of different types which can be obtained by instantiating a general type scheme, substituting for a parameter the subtypes of an assigned type.

To express subtype polymorphism, it is useful to make use of the notation with universal quantifiers that we introduced in the box on Sect. 8.7.1. A polymorphic function of type

$\forall T <: D. T \rightarrow \text{void}$

can be applied to all the values of any subtype of `D`. Polymorphism therefore is not general but is *limited to subtypes of D*.

8.7.4 Remarks on the Implementation

In this book, we cannot possibly explain the implementation of universal polymorphism in detail. We will limit ourselves to two paradigmatic examples of the problems which can be encountered.

The first way of handling polymorphism is that of solving it statically at link time¹⁷ (this is the case with C++). When a polymorphic function is called in two different instances, its code is instantiated in two different ways, one for each of the different instances required. Taking again the concrete example of `swap`, there is a local variable, `tmp`, which must be allocated in the activation record when `swap` is called. But the space to allocate for `tmp` depends on its type which is not known when

¹⁷ Some authors hold that in cases of this kind, we should talk of *generics* rather than polymorphism. We think, on the contrary, that the phenomenon of polymorphism is a syntactic phenomenon of a language, that can be obtained through several different implementation approaches.

Implicit Polymorphism in ML

The majority of compilers for ML are interactive. The user enters one expression (or definition) at a time. First of all, the system verifies the correctness of the typing of the input expression and then derives the most general type for it. If the types are correct, the expression is evaluated and the result is displayed (in the case of a definition, its evaluation will extend the environment with the new association). We can define the identity function as:

```
- fun Ide(x) = x;
val Ide = fn : 'a -> 'a
```

In this example, the first line is what is entered by the user (“-” is the prompt), while the following line is the system’s reply. The name `Ide` is added to the environment and associated with a functional value of type `'a -> 'a`. The identifiers prefixed by ‘`’` are *type variables* in ML, that is they can be instantiated. We can request the evaluation of `Ide` as follows:

```
- Ide(4);
val it = 4 : int
- Ide(true);
val it = true : bool
```

Now the functions:

```
- fun Comp(f,g,x) = f(g(x));
val Comp = fn : ('a->'b)*('c->'a)*'c -> 'b
- fun swap(x,y) = let tmp = !x in
                    (x:=!y; y=tmp);
val swap = fn : ('a ref)*('a ref) -> unit
```

In the case of `swap`, type inference finds that the two arguments cannot be generic values but *variables* (or *references*) of any type whatsoever (`'a ref`). The exclamation mark, “`!`”, is the explicit dereferencing operator. The identifier, `unit`, is the singleton type that is used for expressions with side effects.

```
- val v = ref 0;
val v = ref 0 : int ref
- val w = ref 2;
val w = ref 0 : int ref
- swap(v,w);
val it = () : unit
- !w;
val it = 0 : int
```

In the example, `v` and `w` are initialised, respectively, to a reference to 0 and 2. Type inference correctly deduces that they must be variables of type `int`. After the `swap`, the dereferencing of `w` obviously yields 0.

the template for `swap` is originally compiled. At link time, though, polymorphic functions can be identified and their code is modified (instantiated) so as to take into account the actual type(s) with which they are called. The resulting code can then be linked. It can be seen that there can be many copies of the same template (one per instantiation) in the executable code. On the other hand, the execution of a

template-using program is as efficient as a program that does not use them, because that templates no longer exist at runtime.

In the case of other polymorphic languages (and ML is one of them), a single version of the code for a polymorphic function is maintained. It is this single piece of code that is executed when one of its instances is required. What is done about the type-dependent information (in the case of `swap`, this is the amount of store to allocate for `tmp`)? It is necessary to change the whole data representation. Instead of directly allocating the data in the activation record, a pointer is maintained that points to the actual data. The data also includes its own descriptors (representing dimensions, structure, etc.). In the case of `swap`, when it is necessary to store a value in `tmp`, a pointer to the variable `x` is accessed and its size (as recorded in the descriptor) is accessed. At this point, the memory necessary for `tmp` is allocated in the heap and a pointer to it is stored in the activation record. The flexibility, uniformity and the conciseness of the code (there are no repeated instances of the same function) are paid for by decreased efficiency—an indirect reference is always required to access a piece of data. We will see more implementation details for subtype polymorphism in Sect. 10.4 when considering object-oriented languages.

8.8 Type Checking and Inference

We have already seen how a language's type checker is responsible for verifying that a program respects the rules (in particular, the compatibility rules) imposed by the type system. In the case of a language with static checking, the type checker is one of the compiler's modules; when type checking is dynamic, the type checker is a module in the runtime system. To perform its task, the type checker must determine the type of the expressions occurring in the program (for example, it must determine whether the type of the expression on the right-hand side of an assignment is compatible with the variable on the left-hand side). To do this, it uses the type information that the programmer has explicitly placed at critical points in the program (for example, declarations of local names, parameter declarations, explicit conversions, etc.), as well as information that is implicit in the program text (for example, the types of predefined constants, those of numerical constants and so on).

To determine the type of complex expressions, the type checker performs a simple traversal of the program's abstract syntax tree (see the start of Sect. 6.1.3). Starting at the leaves (which represent variables and constants whose types are known), it moves upwards through the tree towards the root, computing the type of the composite expressions on the basis of the information provided by the programmer and the information it obtains from the type system (for example, the type system could establish that `+` is an operator which, when applied to two expressions of type `int`, permits an expression also of type `int` to be derived, while `=`, when applied to two arguments of scalar type, gives an expression of type `bool`). In many cases, the information provided by the programmer can turn out to be redundant, for example:

```
int f(int n){return n+1;}
```

Starting with the fact that `n` is of type `int`, it can be easily inferred that the value returned by the function must be of type `int`. The explicit specification given by the programmer of the result type is a redundant specification that the language requires as a last resort so that it can report possible logical errors.

Instead of the simple type checking method we have merely outlined, some languages use a more sophisticated procedure which we might call *type inference*. The progenitor of this family of languages is ML (we have cited this language a number of times above), whose type system is sophisticated and refined. Some ideas presented in work on ML have inspired the design of many different languages, even those of a non-functional kind. In order to introduce the concept of inference in a simplified form, we will again use the definition of function `f`. It is clear that also the specification that the formal parameter, `n`, must be an `int` is redundant. This is because the constant `1` is an integer and `+` takes two integers and returns an integer, so `n` must be an integer. From a declaration of the form:

```
fun f(n){return n+1;}
```

it is certainly possible automatically to derive that `f` has type `int -> int`. Type inference is exactly this process of the attribution of a type to an expression in which explicit type declarations of its components do not occur. To perform this derivation, the algorithm also works on a syntax tree, again starting at the leaves but, this time, it must take into account the fact that it might not be immediately possible to determine a specific type for any atomic expression (`n`, in this case). In a case like this, the inference algorithm assigns a *type variable* such as `'a` (using ML notation). Climbing the tree again and using information present in the context, some constraints for type-variables will be collected. In our case, from the type of `1` and from that of `+` (which comes from looking up the table of predefined symbols), the constraint `'a = int` is derived.

This form of inference is more general and powerful than the simple type checking algorithm used in languages like Pascal, C or Java. It is capable of determining the most general type of a function, that is it can account for all the polymorphism implicit in an expression. This is done as follows:

1. Assign a type to each node in the syntax tree. For predefined names and constants, use the type stored in the symbol table. For new (i.e., programmer-defined) identifiers, and for every composite expression (which are stored as the internal nodes of the tree), use a type variable (one new variable for every expression or name);
2. Rewalk the syntax tree, generating an (equality) constraint between types at every internal node. For example, if we apply the function symbol, `f`, to which we have previously assigned the type `'a` to the argument `v` of type `'b`, the constraint `'a = 'b -> 'c` will be generated to indicate that `f` must really be a function and that its argument must be of the same type as `v` (`'c` is a new type variable).
3. Resolve the constraints thus gathered using the *unification* algorithm (a powerful, but conceptually simple, instrument for symbolic manipulation which we will discuss in the context of logic languages, in Sect. 12.3).

There are cases in which solving constraint equations does not remove all variables. If we apply the inference algorithm to

```
fun g(n){return n;}
```

we will obtain, as type, $'a \rightarrow 'a$. We already know that this is not an error but a positive characteristic: an expression whose most general type contains a type variable is a polymorphic expression.

8.9 Safety: An Assessment

We started our analysis of types in programming languages with the idea of safety based on types. This same notion has guided us in our examination of the various characteristics of type systems. The time has arrived to take into account what we have understood, classifying programming languages according to their safeness. Let us, then, distinguish between:

1. Non safe languages;
2. Locally safe languages;
3. Safe languages.

In the category of unsafe languages, we find all of those languages whose type system is a more or less just a methodological suggestion to the programmer, in the sense that the language allows the programmer to bypass type checking. Every language that allows access to the representation of a data type belongs to this category, as does every language that allows access to the value of a pointer (pointer arithmetic, that is). C, C++ and all the languages of their family are unsafe.

Locally unsafe languages are those languages whose type system is well regulated and type checked but which contain some, limited, constructs which, when used, allow insecure programs to be written. The languages ALGOL, Pascal, Ada and many of their descendants belong to this category, provided that the abstract machine really checks types that need dynamic checking, such as checking the limits of interval types (which, it should be recalled, can be used as an index type for arrays). The unsafe part results from the presence of unions (uncontrolled variants) and from the explicit deallocation of memory. Of these two constructs, it is the second that has the greater impact in practice (it is much more common to encounter programs which deallocate memory than those which make significant use of variant records), but it is the former that is the more dangerous. As we will see in the next section, it is possible to equip the abstract machine with appropriate mechanisms to allow it to detect and prevent dangling pointers, even if, for reasons of efficiency, they are almost never used. We have already seen different examples of type-system violations which are possible when using variant records. We have postponed until last the nastiest possibility:

that variant records can be used to access and manipulate the value of a pointer. In Pascal, we can write¹⁸:

```
var v : record
    case bool of
        true  : (i:integer);
        false : (p:^integer)
    end
```

Here, `^integer` is the Pascal notation for the type of pointers to integers. It is now possible to assign a pointer to the variant `p`, manipulate it as an integer using the variant `i` and then use it again as a pointer.

Finally, we have safe languages for which a theorem guarantees that the execution of a typed program can never generate a hidden error “induced by the violation of a type.” In this category are languages with dynamic type checking like Lisp and Scheme, languages with static checking such as ML, and languages with static checking but with many checks also performed at runtime (as in Java). Type safety—at it should be clear after the several examples of this chapter—is obtained as a delicate balance between the statics and the dynamics of a programming language.

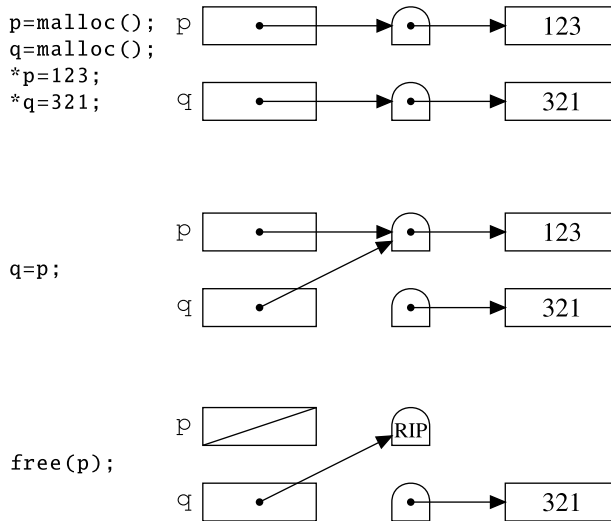
8.10 Avoiding Dangling References

In this section, we will tackle the question of which mechanisms can be included in an abstract machine that will dynamically prevent dereference of dangling pointers. This is a problem we first discussed in Sect. 8.4.6. We will first present a radical solution which works in the general case of pointers into the heap or into the stack. We will then consider a somewhat less demanding mechanism that, however, works only for pointers into the heap (and then under some probabilistic assumptions).

8.10.1 Tombstone

Using tombstones, an abstract machine can detect every attempt at dereferencing a dangling reference. Every time an object is allocated in the heap (to be then accessed by pointer), the abstract machine also allocates an extra word in memory. This word is called the *tombstone*. In a similar fashion, a tombstone is allocated every time that a stack-referencing pointer is allocated (that is, to a certain approximation, every time that the “indirection-to” operator (`&`) is used). The tombstone is initialised with the address of the allocated object and the pointer receives the address of the tombstone. When a pointer is dereferenced, the abstract machine inserts a second level of indirection, so that it first accesses the tombstone and then uses what it finds

¹⁸ In Pascal, pointers are allocated only on the heap by an allocation request (the `new` function). They can be assigned but there is no primitive method to access the value of a pointer.

**Fig. 8.10** Tombstones

there to access the object that is pointed to. When one pointer is assigned to another, it is the contents of the pointer (and not the tombstone) that is modified. Figure 8.10 shows this operation graphically.

On deallocation of an object, or when an indirection on the stack becomes invalid because it is part of an activation record that has been popped,¹⁹ the associated tombstone becomes invalidated, and a special value is stored in it to signal that the data to which the pointer refers is dead (this is where the name of this technique derives from). An appropriate choice of such a value must be made so that every attempt to access the contents of an invalid tombstone is captured by the address-protection mechanism of the underlying physical machine.

Tombstones are allocated in a particular area of memory in the abstract machine (this, appropriately enough, is called the *cemetery*). The cemetery can be managed more efficiently than the heap because all tombstones are of the same size.

For all its simplicity, the tombstone mechanism imposes a heavy cost in terms both of time and space. As far as time is concerned, we must consider the time required for the creation of tombstones, that for their checking (which can be reduced, as we have seen, if an underlying hardware protection mechanism can be used) and, most importantly, the cost of two levels of indirection. Tombstones are also expensive in terms of storage. They require a word of memory for every object allocated on the heap; a word is also required every time a pointer into the stack is created. If there are

¹⁹ The operation can be quite opaque. Consider the case of a variable passed by reference to a function. Inside the function, a pointer is created to point to the variable. The pointer will be created using an associated tombstone. The tombstone must be invalidated when the lifetime of the variable forming the actual parameter ends, not when the function terminates.

lots of allocations of small objects, the percentage of memory required by tombstones can be significant. Moreover, invalid tombstones always remain allocated, with the consequence that they might exhaust the space available in the cemetery, even though there may be plenty of space available on the heap. To prevent this last problem, it is possible to reuse those tombstones that are no longer in use (that is, those to which no pointer points) using a small reference-counting garbage collector (Sect. 8.11.1); this further increases the time cost of the mechanism.

8.10.2 Locks and Keys

An alternative to the tombstone technique is called *locks and keys*. This solves the problem of dangling references into a heap using a mechanism which does not suffer from the accumulated problems of tombstones.

Every time an object is created on the heap, the object is associated with a *lock* which is a word of memory in which an arbitrary value is stored. (Strictly speaking, it should be a value that can be sequentially incremented every time an object is allocated, but which avoids values such as 0 and 1, the code of frequently used characters, and so on.) In this approach, a pointer is composed of a pair: the address proper and a *key* (a word of memory that will be initialised with the value of the lock corresponding to the object being pointed to). When one pointer is assigned to another, the whole pair is assigned. Every time the pointer is dereferenced, the abstract machine checks that the key opens the lock. In other words, it verifies that the information contained in the key coincides with that in the lock. In the case in which the values are not the same, an error is signalled. When an object is deallocated, its lock is invalidated and some canonical value (for example, zero) is stored, so that all the keys which previously opened it now cause an error (see Fig. 8.11). Clearly, it can happen that the area of memory previously used as a key is reallocated (for another lock or for some other structure). It is statistically highly unlikely that an error will be signalled as a result of randomly finding an ex-lock that has the same value as it had before it was cleared.

Locks and keys also have a non-negligible cost. In terms of space, they cost even more than tombstones because it is necessary to associate an additional word with each pointer. On the other hand, locks as well as keys are deallocated at the same time as the object or the pointer of which they are a part. From an efficiency viewpoint, it is necessary to take into account both the cost of creation, and, more importantly, the high cost of pointer assignment and of determining whether the key opens a lock, something that happens every time a pointer is dereferenced.

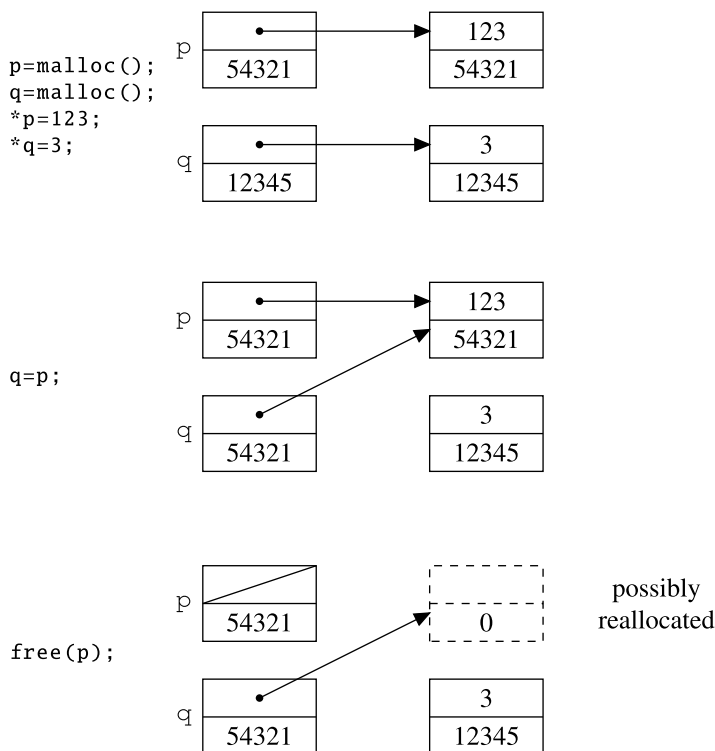


Fig. 8.11 Locks and keys

8.11 Garbage Collection

In languages without explicit memory deallocation, it is necessary to equip the abstract machine with a mechanism which could automatically reclaim the memory allocated on the heap that is no longer used. This is done by a *garbage collector*, introduced for the first time in LISP around 1960, and since then included in many languages. Java has a powerful and efficient garbage collector.

From the logical point of view, the operation of a garbage collector consists of two phases:

1. Distinguish those objects that are still alive from those no longer in use (*garbage detection*);
2. Collect those objects known no longer to be in use, so that the program can reuse their space.

In practice these two phases are not always temporally separate, and the technique for reclaiming objects essentially depends on the one in which the objects no longer in use are discovered. We will see, then, that the “no longer in use” concept in a

garbage collector is often a conservative approximation. For reasons of efficiency,²⁰ not all the objects that can be used again are, in reality, determined by the garbage collector to be so.

The educational aim of this text does not permit us to describe a garbage collector for a real language, or to provide an exhaustive overview of the different techniques available (there are bibliographic references to exhaustive treatments). We will limit ourselves to presenting in some detail the main points in the light of the most common techniques. Real garbage collectors are variations on these techniques. In particular the collectors that are of greatest interest today are based upon some form of incremental reclamation of memory and are beyond the scope of this book.

We can classify classical garbage collectors according to how they determine whether an object is no longer in use. We will have, therefore, collectors based on *reference counting*, as well as *marking* and *copying*. In the next few sections, we will present these collectors. We will discuss these techniques in terms of pointers and objects but the argument can equally be applied to languages without pointers which use the reference variable model.

Finally, we will see that all the techniques that we consider need to be able to recognize those fields inside an object that correspond to pointers. If objects are created as instances of statically-defined types (so it is statically known where pointers are inside instances), the compiler can generate a descriptor containing offsets to the pointers in each type. Each object in the heap is associated with the type of which it is an instance (for example, via a pointer to its type descriptor). When an object is to be deallocated, the garbage collector accesses the type descriptor to access the pointers inside the object. Similar techniques are used to recognise which words in an activation record are pointers. If types are only known dynamically, descriptors have to be completely dynamic and allocated together with the object.

8.11.1 Reference Counting

A simple way to tell whether an object is not in use is to determine whether there are any pointers to it. This is the reference counting technique. It defines what is probably the easiest way of implementing a garbage collector.

When an object is created on the heap, an integer is allocated at the same time. This integer is the *reference counter* or *reference count* for the object. The reference count is inaccessible to the programmer. For each object, the abstract machine must ensure that the counter contains the number of pointers to this object that are still active.

In order to do this, when an object is created (that is, a pointer for it is assigned for the first time), its counter is initialised to 1. When we have a pointer assignment:

```
p = q;
```

²⁰ As well as decidability.

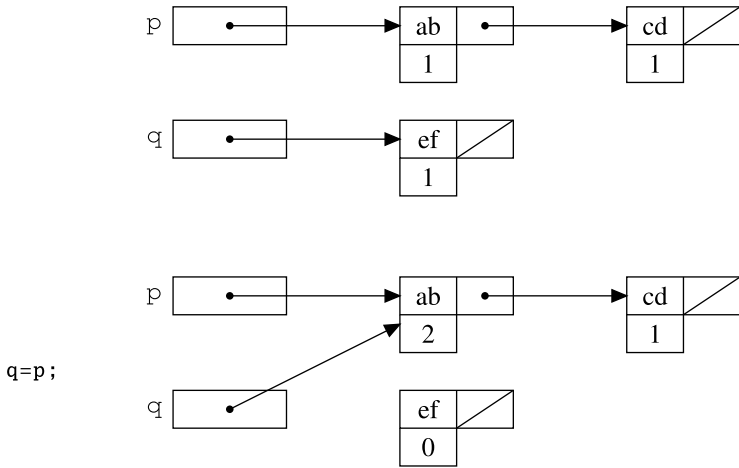


Fig.8.12 Reference counting

the counter belonging the object pointed to by *q* is incremented by one, while the counter of object pointed to by *p* is decremented by one. When a local environment is exited, the counters of all the objects pointed to by pointers local to the environment are decremented. Figure 8.12 shows operation of this technique in diagrammatic form.

When a counter returns to the value 0, its associated object can be deallocated and returned to the free list. This object, moreover, might contain internal pointers, so before returning the memory occupied by an object to the free list, the abstract machine follows the pointers inside the object and decrements by one the counters of the objects they point to, recursively collecting all the objects whose counter has reached 0.

From the viewpoint of the abstract division into two phases of a garbage collector, the update and checking of counters implement the garbage-detection phase. The collection phase occurs when the counter reaches zero.

A clear advantage of this technique is that it is *incremental* because checking and collection are interleaved with the normal operation of the program. With a few adjustments, real-time systems (that is systems where there are absolute deadlines to response time) can employ this technique.

The biggest defect, at least in principle, of this technique is in its inability to deallocate circular structures. Figure 8.13 shows a case in which a circular structure has no more access paths. However it cannot be collected because, clearly, its counters are not zero. It can be seen that the problem does not reside so much in the algorithm, as in the definition of what a useless object is. It is clear that all objects in a circular structure missing an external reference are not usable any more, but this is not at all captured by the definition of not being pointed at.

It could be argued that reference counting, despite its simplicity, is also fairly inefficient. Its cost is proportional to the combined work performed by the program

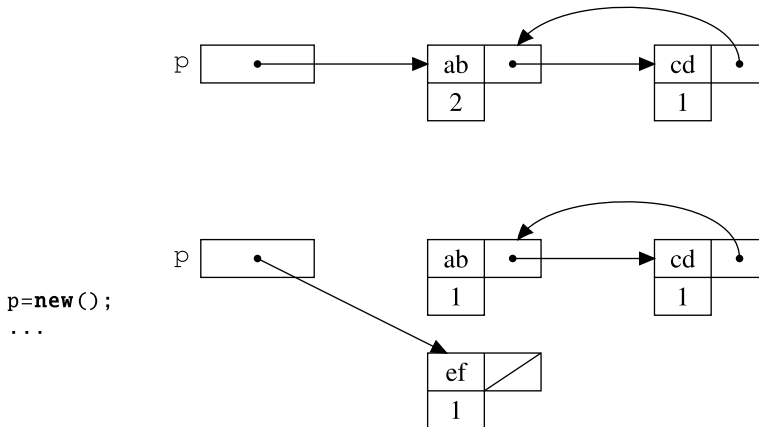


Fig. 8.13 Circular structures with reference counts

(and not to the size of the heap or to the percentage of it in use or not in use at any time). One particular case is that of updating the counters of parameters of pointer type which are passed to functions that execute only for a short time. These counters are allocated and, after a particularly short time, have their value returned to zero.

Of the major programming languages using automatic memory management, Python, in its reference implementation CPython, is the one with a basic garbage collector based on reference counting. It cannot be disabled and it is always at work. An additional, incremental collector (which, instead, may be disabled by the user) is available for reclaiming unusable circular structures.

8.11.2 Mark and Sweep

The *mark and sweep* method takes its name from the way in which the two abstract phases we mentioned at the start are implemented:

- *Mark*. To recognize that something is unused, all the objects in the heap are traversed, marking each of them as “not in use”. Then, starting from the pointers which are active and present on the stack (the *root set*), all the data structures present in the heap are traversed recursively (the search is usually performed depth-first) and every object that is encountered is marked as “in use”.
- *Sweep*. The heap is swept—all blocks marked “in use” are left alone, while those marked “not in use” are returned to the free list.

It can be seen that to implement both phases, it is necessary to be able to recognize allocated blocks in the heap. Descriptors might be necessary to give the size (and organisation) of every allocated block.

Unlike the reference-counting garbage collector, a mark and sweep collector is not incremental. It will be invoked only when the free memory available in the heap is close to being exhausted. The user of the program can therefore experience a significant degradation in response time while waiting for the garbage collector to finish.

The mark and sweep technique suffers from three main defects. In the first place, and this is also true for reference counting, it is asymptotically the cause of external fragmentation (see Sect. 5.4.2): live and no longer live objects are arbitrarily mixed in the heap which can make allocating a large object difficult, even if many small blocks are available. The second problem is efficiency. It requires time proportional to the total length of the heap, independent of the percentages of used and free space. The third problem relates to locality of reference. Objects that are “in use” remain in their place, but it is possible that objects contiguous with them will be collected and new objects allocated in their place. During execution, we find that objects with very different ages appear next to each other. This, in general, drastically reduces locality of reference, with the usual degradation in performance observed in systems with memory hierarchies.

8.11.3 Interlude: Pointer Reversal

Without some precautions, every marking technique runs the risk of being completely unusable in a garbage collector. The collector, indeed, goes into action when the memory is near to being exhausted, while the marking phase consists of the recursive traversal of a graph, which requires the use of a stack to store the return pointers.²¹ It is necessary, when marking a graph under these circumstances, to use carefully the unused space present in pointers, using a technique called *pointer reversal*.

As shown in Fig. 8.14, when visiting a chained structure, it is necessary to mark a node and recursively visit the substructures whose pointers are part of that node. In this recursive scheme, it is necessary to store the addresses of blocks when they are visited in a stack, so that they can be revisited when the end of the structure is reached. In the figure, using depth-first search, node E has been reached (marked blocks are indicated with a tick). The stack contains nodes A, B and C. After visiting E, it takes C from the stack and follows any pointers leading from this node; since there are none, it takes B from the stack and follows the remaining pointer, pushing B onto the stack, and visiting D. Using pointer reversal, this stack is stored in the pointers that form the structure, as shown in Fig. 8.15. When the end of a substructure is reached, (and a pop of the stack is required), the pointer is returned to its original value, so that at the end of the visit, the structure is exactly the same as it was at the start (apart from marking, clearly). In the figure, we are visiting node E. It can be seen how the

²¹ In many abstract machines, stack and heap are implemented in a single area of memory, with stack and heap growing in opposite directions starting from the two ends. In such a case, when there is no space in the heap, there is no space for the stack.

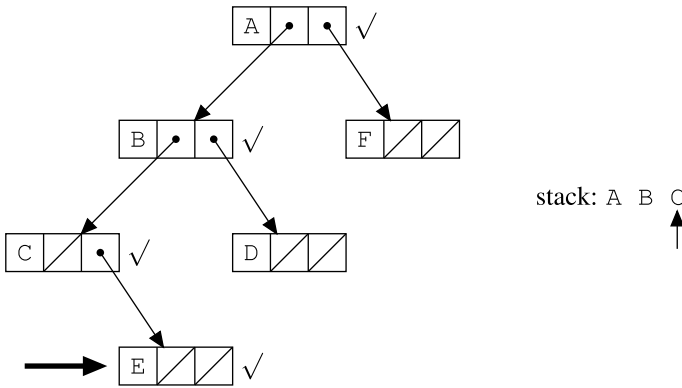
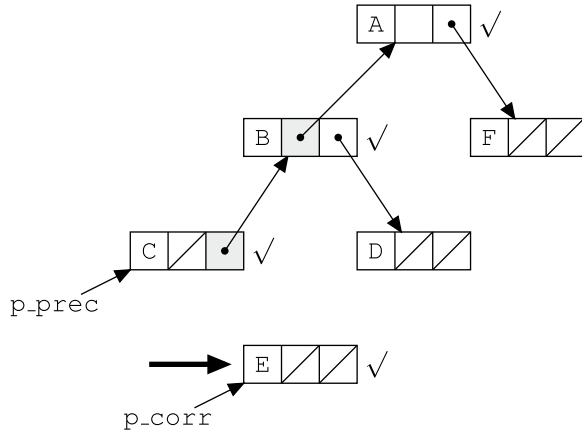


Fig. 8.14 A stack is necessary for depth-first traversal

Fig. 8.15 Pointer reversal



stack in Fig. 8.14 is stored in space internal to the structure itself (pointers marked with a grey background). Only two pointers (*p_prec* and *p_curr*) are required to perform the visit. After visiting E, using *p_prec*, the structure is retraversed in a single step. Using the reversed pointer, we return to B, resetting the correct value of the pointer in C using *p_curr*.

8.11.4 Mark and Compact

To avoid the fragmentation caused by the mark and sweep technique, we can modify the sweep phase and convert it into a compaction phase. Live objects are moved so that they are contiguous and thereby leave a contiguous block of free memory. Compaction can be performed by moving linearly across the heap, moving every live block encountered and making it contiguous with the previous block. At the end, all free blocks are contiguous, as are all unused blocks.

This is a technique which, like mark and sweep, requires more than one pass over the heap (and the time required is therefore proportional to the heap size). Compaction, on its own, requires two or three passes. The first is to compute the new position to be taken by the live blocks; a second updates the internal pointers and a third actually moves objects. It is, therefore, a technique that is substantially more expensive than mark and sweep if there are many objects to be moved.

On the other hand, compaction has optimal effect on fragmentation and on locality. It supports treating the free list as a single block. New objects are allocated from the free list by a simple pointer subtraction.

8.11.5 Copy

In garbage collectors based on copy there is no phase during which the “garbage” is marked. There is just the copying and compaction of live blocks. The lack of an explicit mark phase and the completely different way space is handled make its costs substantially different from those of algorithms based on marking.

In the simplest copy-base garbage collector (called a *stop and copy* collector), the heap is divided into two equally-sized parts (two *semi-spaces*). During normal execution, only one of the two semi-spaces is in use. Memory is allocated at one end of the semi-space, while free memory consists of a unique contiguous block which reduces its size every time there is an allocation (see Fig. 8.16). Allocation is extremely efficient and there is no fragmentation.

When the memory in the semi-space is exhausted, the garbage collector is invoked. Starting with pointers in the stack (the root set), it begins visiting the chain of structures held in the current semi-space (the *fromspace*), copying the structures one by one into the other semi-space (the *tospace*), compacting them at one end of the latter

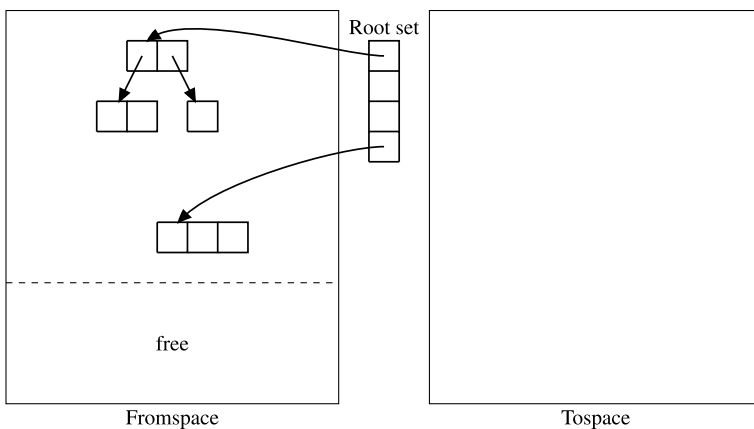


Fig. 8.16 Stop and copy before a call to the garbage collector

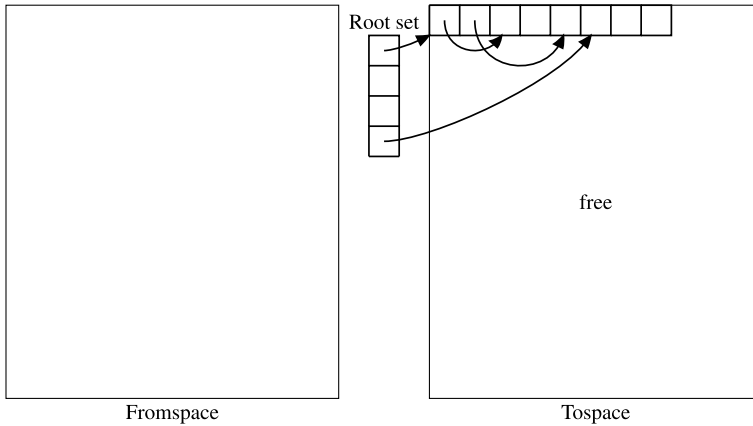


Fig. 8.17 Stop and copy after the execution of the garbage collector

(see Fig. 8.17). At the end of this process, the role of the two semi-spaces is swapped and execution returns to the user program.

The visit and copy of the live part can be executed in an efficient manner using the simple technique known as Cheney's algorithm (Fig. 8.18). Initially, all the objects which are immediately reachable from the root set are copied. This first set of objects is copied in a contiguous fashion into the tospace; it is handled as if it were a queue. Consider now the first of these objects, and add to the end of the queue (that is, copy into the tospace) the objects pointed to by the pointers present in the first object, while, at the same time, these pointers are modified. In this way, we have copied into the tospace all the first object's children. We keep processing the queue until it remains empty. At this point, in the tospace we have a copy of the live objects in the fromspace.²²

A stop and copy garbage collector can be made arbitrarily efficient, provided that there is enough memory for the two semi-spaces. In fact, the time required for a stop and copy is proportional to the number of live objects present in the heap. It is not unreasonable to assume that this quantity will be approximately constant at any moment during the execution of the program. If we increase the memory for the two semi-spaces, we will decrease the frequency with which the collector is called and, therefore, the total cost of garbage collection.

²² Some precautions must be taken to prevent objects accessible via multiple pointers being copied more than once.

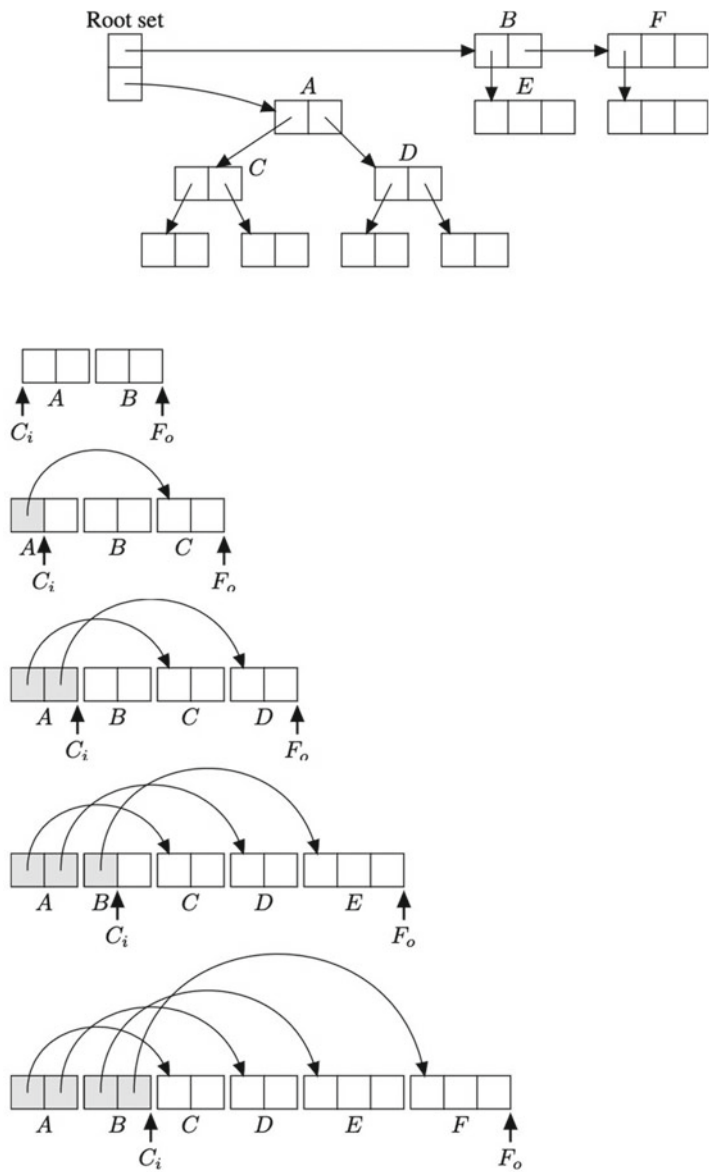


Fig.8.18 Cheney's algorithm

8.12 Summary

This chapter has dealt with a crucial aspect in the definition of programming languages: the organisation of data in abstract structures called data types. The main points can be summarised as follows.

- *Definition of type* as a set of values and operations and the role of types in design, implementation and execution of programs.
- *Type systems* as the set of constructs and mechanisms that regulate and define the use of types in a programming language.
- The distinction between *dynamic* and *static* type checking.
- The concept of type-safe systems, that is safe with respect to types.
- The primary *scalar types*, some of which are *discrete* types.
- The primary *composite types*, among which we have discussed *records*, *variant records* and *unions*, *arrays* and *pointers* in detail. For each of these types, we have also presented the primary storage techniques.
- The concept of *type equivalence*, distinguishing between equivalence by name and structural equivalence.
- The concept of *compatibility* and the related concepts of coercion and conversion.
- The concept of *overloading*, when a single name denotes more than one object and static disambiguation.
- The concept of *universal polymorphism*, when a single name denotes an object that belongs to many different types, finally distinguishing between parametric and subtype polymorphism.
- *Type inference*, that is mechanisms that allow the type of a complex expression to be determined from the types of its elementary types.
- Techniques for runtime checking for *dangling references*: tombstones and locks and keys.
- Techniques for *garbage collection*, that is the automatic recovery of memory, briefly presenting collectors based on reference counters, mark and sweep, mark and compact and copy.

Types are the core of a programming language. Many languages have similar constructs for sequence control but are differentiated by their type systems. It is not possible to understand the essential aspects of other programming paradigms, such as object orientation, without a deep understanding of the questions addressed in this chapter.

8.13 Bibliographical Notes

Types are a central notion in programming language design. Classic references are [2] and the more recent [3]. Review articles that introduce the mathematical formalisms are [4,5]. A larger treatment of the same arguments is to be found in [1]. On the

structural (behavioral) notion of compatibility, see [6] that introduces what we now call the Liskov substitution principle. On overloading and polymorphism in type systems, a good, clear review is [7] (which we have largely followed in this chapter).

Tombstones originally appeared in [8] (also see [9]). Fisher and Leblanc [10] proposed locks and keys, as well as techniques so that an abstract Pascal machine can make variant records secure.

The official definition of ALGOL 68 (which is quite a read) is [11]. A more accessible introduction can be found in [12]. The definition of ML can be found in [13], while an introductory treatment of type inference is to be found in [14].

There is a large literature on garbage-collection techniques. A detailed description of a mark and sweep algorithm can be found in several books on algorithms, for example [15], while [16] is a good summary of classical techniques. For a book entirely devoted to the problem, with pseudocode and a more-or-less complete bibliography (up to the time of publication), see [17].

8.14 Exercises

1. Consider the declaration of the multi-dimensional array:

```
int A[10][10][10]
```

We know that an integer can be stored in 4 bytes. The array is stored in row order, with increasing memory addresses (that is, if an element is at address i , its successor is at $i + 4$, and so on). The element $A[0][0][0]$ is stored at address 0. State the address at which element $A[2][2][5]$ is stored.

2. Instead of the contiguous multidimensional array allocation that we discussed in Sect. 8.4.4, some languages allow (e.g., C), or adopt (Java), a different organisation, called *row-pointer*. Let us deal with the case of a 2-dimensional array. Instead of storing rows one after the other, every row is stored separately in some region of memory (for example on the heap). Corresponding to the name of the vector, a vector of pointers is allocated. Each of the pointers points to a proper row of the array. (i) Give the formula for accessing an arbitrary element $A[i][j]$ of an array allocated using this scheme. (ii) Generalise this memory technique to arrays of more than 2 dimensions. (iii) Discuss the advantages and disadvantages of this organisation in the general case. (iv) Discuss the advantages and disadvantages of this technique for the storage of 2-dimensional arrays of characters (that is, arrays of strings).
3. Consider the following (Pascal) declarations:

```
type string = packed array [1..16] of char;
type string_pointer = ^string;
type person = record
    name : string;
    case student : Boolean of
        true: (year : integer);
        false: (socialsecno : string_pointer)
end;
```

and assume that the variable `C` contains a pointer to the string "LANGUAGES". Describe the memory representation for the person record after each of the following operations:

```
var pippo : person;
pippo.student := true;
pippo.year := 223344;
pippo.student := true;
pippo.socialsecno := C;
```

4. Show that the integer type can be defined as a recursive type starting with just the value `null`. Then write a function that checks if two integers defined in this way are equal.
5. Given the following type definitions in a programming language which uses structural type equivalence:

```
type T1 = struct{
    int a;
    bool b;
};
type T2 = struct{
    int a;
    bool b;
}
type T3 = struct{
    T2 u;
    T1 v;
}
type T4 = struct{
    T1 u;
    T2 v;
}
```

In the scope of the declarations `T3 a` and `T4 b`, it is claimed that the assignment `a = b` is permitted. Justify your answer.

6. Which type is assigned to each of the following functions using polymorphic type inference?

```
fun G(f,x){return f(f(x));}
fun H(t,x,y){if (t(x)) return x;
              else return y;}
fun K(x,y){return x;}
```

7. Using tombstones, it is necessary to invalidate a tombstone when its object is no longer meaningful. The matter is clear if the object is on the heap and is explicitly deallocated. It is less clear when the tombstone is associated with an address on the stack. In this case, indeed, it is necessary for the abstract machine to be able to determine all the tombstones that are potentially associated with an activation record. Design a possible organisation which makes this operation reasonably efficient (recall that tombstones are not allocated in activation records but in the cemetery).

8. Consider the following fragment in a pseudo-language with reference-based variables and which uses locks and keys (C is a class whose structure is of no importance):

```
C foo = new C(); // object OG1
C bar = new C(); // object OG2
C fie = foo;
bar = fie;
```

Give possible values *after* the execution of the fragment for all the keys and all the locks involved.

9. Under the same assumptions as in Exercise 8, consider the following code fragment in which `free(p)` denotes the explicit deallocation of the object referred to by the pointer `p`:

```
class C { int n; C next;}
C foo = new C(); // object OG1
C bar = new C(); // object OG2
foo.next = bar;
bar.next = foo;
free(bar);
```

For all the pointers in the fragment, give possible key values. For each object in the fragment, give possible lock values. In both cases, the values should be those *after* execution of the fragment terminates. After execution of the fragment, also execute the code `foo.n = 1; foo.next = 0;`. State what a possible result of this execution might be.

10. Consider the following fragment which is written in a pseudo-language with a reference-model for variables and a reference-counting garbage collector. If OGG is an arbitrary object in the heap, indicate by `OGG.cont` its (hidden) contents:

```
class C { int n; C next;}
C foo(){
    C p = new C(); // object OGG1
    p.next = new C(); // object OGG2
    C q = new C(); // object OGG3
    q.next = p.next;
    return p.next;
}
C r = foo();
```

State what are the values of the reference counters for the three objects after execution of line 6 and then of line 9.

11. Under the same assumptions as in Exercise 10, state what the values of the reference counters are for the two objects after the execution of the following fragment. Which of the two can be returned to the free list?

```
C foo = new C(); // object OG1
C bar = new C(); // object OG2
C fie = foo;
bar = fie;
```

12. Under the same assumptions as in Exercise 10, state what the reference-counter values for the three objects after execution of the following fragment are. Which of them can be returned to the free list?

```
class C { int n; C next;}
C foo = new C(); // object OG1
bar = new C(); // object OG2
foo.next = bar;
bar = new C(); // object OG3
foo = bar;
```

13. Using your favourite programming language, define the data structures required to represent a binary tree. Then write detailed code that performs a preorder traversal of a tree without using the system stack and instead using the pointer-reversal technique.

References

1. B. Pierce, *Types and Programming Languages* (MIT Press, Cambridge, MA, 2002)
2. D.A. Schmidt, *The Structure of Typed Programming Languages* (MIT Press, Cambridge, Mass., 1994)
3. R. Harper, *Practical Foundations for Programming Languages*, 2nd edn. (Cambridge University Press, Cambridge, 2016)
4. L. Cardelli, Type systems, in *The Computer Science and Engineering Handbook*. ed. by A.B. Tucker (CRC Press, Boca Raton, FL, 1997)
5. J.C. Mitchell, Type systems for programming languages, in *Handbook of theoretical computer science (vol. B): formal models and semantics*, ed. by J. van Leeuwen. (MIT Press and Elsevier Science Publishers, Cambridge-Amsterdam, 1990), pp. 365–458
6. B.H. Liskov, J.M. Wing, A behavioral notion of subtyping. *ACM Trans. Program. Lang. Syst.* **16**(6), 1811–1841 (1994). (Nov)
7. L. Cardelli, P. Wegner, On understanding types, data abstraction, and polymorphism. *ACM Comput. Surv.* **17**(4), 471–522 (1985)
8. D.B. Lomet, Scheme for invalidating references to freed storage. *IBM J. Res. Develop.* **19**(1), 26–35 (1975)
9. D.B. Lomet, Making pointers safe in system programming languages. *IEEE Trans. Softw. Eng.* **11**(1), 87–96 (1985)
10. C.N. Fisher, R.J. LeBlanc, The implementation of run-time diagnostics in Pascal. *IEEE Trans. Software Eng.* **6**(4), 313–319 (1980)
11. A. van Wijngaarden, B.J. Mailloux, J.E.L. Peck, C.H.A. Koster, M. Sintzoff, C.H. Lindsey, L.G.L.T. Meertens, R.G. Fisker, Revised report on the algorithmic language ALGOL 68. *Acta Inf.* **5**, 1–236 (1975)
12. F. Pagan. *A Practical Guide to Algol 68*, Wiley Series in Computing. (Wiley, London, 1976)
13. R. Milner, M. Tofte, R. Harper, D. MacQueen, *The Definition of Standard ML - Revised* (MIT Press, Cambridge, MA, 1997)
14. L. Cardelli, Basic polymorphic typechecking. *Sci. Comput. Progr.* **8**(2), 147–172 (1987)
15. E. Horowitz, S. Sahni. *Fundamentals of Data Structures in Pascal* (Freeman and Company, New York, 1994)

16. P.R. Wilson, Uniprocessor garbage collection techniques, in *Proceedings of the International Workshop on Memory Management*, LNCS 637 (Springer, Berlin, 1992)
17. R. Jones, R. Lins, *Garbage Collection: Algorithms for Automatic Dynamic Memory Management* (Wiley, Hoboken, 1996)